Universidad Nacional de Asunción
Facultad Politécnica

Ingeniería en Informática



Verificación de consenso e inclusión de transacciones en Ethereum con conocimiento cero aplicado a la transferencia de activos digitales

Javier Rafael López Cáceres
Alejandro Fabián Marín Insfran

San Lorenzo - Paraguay
Septiembre - 2026

Universidad Nacional de Asunción
Facultad Politécnica

Ingeniería en Informática



Verificación de consenso e inclusión de transacciones en Ethereum con conocimiento cero aplicado a la transferencia de activos digitales

Javier Rafael López Cáceres
Alejandro Fabián Marín Insfran

Asesor:
PhD Marcos Villagra

Proyecto de Trabajo de Grado presentado en conformidad a los requisitos para obtener el grado de Ingeniería en Informática

San Lorenzo - Paraguay
Septiembre - 2026

Dedico a mi familia,
y a mis amigos hghh.

AGRADECIMIENTOS

Verificación de consenso e inclusión de transacciones en Ethereum con conocimiento cero aplicado a la transferencia de activos digitales

Autor: Javier López - Alejandro Marín

Asesor: PhD Marcos Villagra

RESUMEN

Los puentes entre cadenas o popularmente conocidos como *cross-chain bridges*, permiten transferir activos entre distintas *blockchains* y constituyen una pieza central de la interoperabilidad. La mayoría se apoya en el patrón *lock-and-mint* operado por un *relayer* de confianza, un actor que tras observar el bloqueo de fondos en la cadena de origen acuña el equivalente en la cadena de destino. Como toda la seguridad recae en ese operador, que podría acuñar de forma fraudulenta, los puentes se han vuelto uno de los blancos más explotados del ecosistema. En este trabajo se diseña e implementa un puente de confianza reducida (*trust-minimized*) que sustituye esa confianza por verificación criptográfica. El contrato de destino solo acuña cuando valida *on-chain* una prueba de conocimiento cero (*zero-knowledge*). La prueba acredita que la transacción de bloqueo fue incluida en el bloque anclado y que el recibo de esa misma posición registra una ejecución exitosa. El receptor verifica que la transacción invoca `lock` en `Sender` y obtiene de los datos de la llamada el destinatario y el monto que se acuñará. Estas comprobaciones no acreditan la finalidad del bloque. La lógica de verificación se desarrolló en Noir y, de forma equivalente, en Rust para la máquina virtual de conocimiento cero SP1, con verificadores *on-chain* en Solidity, y ambas implementaciones se comparan empíricamente en rendimiento, costo de verificación y viabilidad práctica. Un *relayer off-chain* construye las pruebas de consenso y de inclusión de cada operación y las remite al contrato de destino, donde deja de ser un acuñador de confianza y pasa a ser un probador sin permisos, ya que cualquiera con pruebas válidas puede liberar los fondos y nadie puede acuñar sin ellas. De manera complementaria se diseña la validación de la firma del comité de sincronización (*sync committee*) de Ethereum como raíz de confianza sobre la finalidad del bloque. La solución se validó sobre redes de prueba reales, con Ethereum Sepolia como cadena de origen y Optimism Sepolia como destino. Adicionalmente incluye una interfaz web y un modo comparativo que evidencia de forma experimental cómo el esquema clásico admite la acuñación fraudulenta mientras que el esquema con verificación de conocimiento cero la rechaza.

Palabras Clave: puentes entre cadenas (*cross-chain bridges*); pruebas de conocimiento cero (*zk-SNARKs*); Noir; UltraHonk; SP1; verificación *on-chain*; interoperabilidad *blockchain*; prueba de inclusión (árbol de Merkle-Patricia); confianza reducida (*trust-minimization*); Ethereum.

Title

Author: full name

Advisor: full name

ABSTRACT

This is the abstract

Keywords:

ÍNDICE

Página

1. INTRODUCCIÓN **1**

1.1. Contexto y antecedentes 1
1.2. Planteamiento del problema 2
1.3. Justificación 2
1.4. Objetivos 3
1.4.1. Objetivo general 3
1.4.2. Objetivos específicos 3
1.5. Hipótesis 4
1.6. Delimitación 4

2. BLOCKCHAIN **5**

2.1. Arquitectura de blockchain 5
2.1.1. Estructura de bloques y encadenamiento por hash 5
2.1.2. Árboles de Merkle y pruebas de inclusión 6
2.1.3. Mecanismos de consenso 6
2.1.4. Roles de nodos y topología de red 7
2.2. Ethereum 8
2.2.1. Modelo de cuentas y máquina virtual 8
2.2.2. Gas y costo de ejecución 9
2.2.3. Arquitectura en dos capas 11
2.2.4. Estructura del bloque de ejecución 11
2.2.5. Árboles de Merkle-Patricia 12
2.2.6. Consenso y finalidad 13
2.2.7. Comité de sincronización y clientes ligeros 14
2.2.8. Cadena de verificación para el puente 16
2.3. Aztec 16
2.3.1. Rollups y pruebas de validez 17
2.3.2. Estado privado y estado público 17
2.3.3. Árboles de estado 18
2.3.4. Ejecución de transacciones y estructura de bloques 19
2.3.5. Secuenciadores, probadores y finalidad 20
2.3.6. Mensajería entre Ethereum y Aztec 21
2.4. Optimism 21
2.4.1. Arquitectura del OP Stack 22
2.4.2. Confirmación de bloques y finalidad 23
2.4.3. Compatibilidad con la EVM y modelo de tarifas 24
2.4.4. Pruebas de fraude 24
2.4.5. Puente nativo con Ethereum 25

2.5. Puentes 26

3. CONOCIMIENTO CERO **27**

3.1. Fundamentos 27
3.1.1. Concepto de conocimiento cero 27
3.1.2. Propiedades fundamentales 29
3.1.3. Pruebas interactivas y no interactivas 30
3.1.4. De una computación a una prueba 30
3.1.5. SNARKs y STARKs 33
3.2. Circom 35
3.2.1. Origen y propósito 35
3.2.2. Construcción de circuitos 36
3.2.3. Compilación, cálculo del testigo y generación de la prueba 37
3.2.4. Limitaciones y consideraciones de seguridad 38
3.3. Noir 40
3.3.1. Origen y propósito 40
3.3.2. Descripción de computaciones y generación de restricciones 41
3.3.3. Compilación, cálculo del testigo y generación de la prueba 43
3.3.4. Limitaciones y consideraciones de seguridad 46
3.4. SP1 48
3.4.1. Origen y propósito 49
3.4.2. Programas, entradas y valores públicos 50
3.4.3. Compilación, ejecución, generación y verificación de la prueba 53
3.4.4. Limitaciones y consideraciones de seguridad 55
3.5. Otros entornos y herramientas del ecosistema 57

4. ZKBRIDGE **59**

4.1. Implementación de un puente lock-and-mint 59
4.1.1. Arquitectura y componentes 59
4.1.2. Flujo operativo de la transferencia 61
4.1.3. Condiciones de aceptación y modelo de confianza 62
4.2. zkBridge de Berkeley RDI 63
4.3. Diseño e implementación en Noir 64
4.3.1. Programa de consenso 65
4.3.2. Programa de inclusión 66
4.3.3. Valores públicos y contratos verificadores 67
4.3.4. Generación de las pruebas e integración 68
4.4. Diseño e implementación en SP1 70
4.4.1. Programa de consenso 70
4.4.2. Programa de inclusión y ejecución exitosa 71

4.4.3. Valores públicos y verificación en Solidity 72
4.4.4. Generación de pruebas e integración 73

5. CONCLUSIÓN **75**

REFERENCIAS **76**

APÉNDICE **87**
A.1. A 87

PENDIENTES **88**
— Abstract (Preliminares, después del Resumen) [Necesario - Crítico] 88
— Agradecimientos (Preliminares, después de la Dedicatoria) [Sugerencia - Recomendado] 88
1.7 Estructura del documento (al final del Capítulo 1, después de Delimitación) [Sugerencia - Recomendado] 88
2.5.1 Interoperabilidad y concepto de puente (al inicio de la Sección 2.5 Puentes) [Necesario - Importante] 88
2.5.2 Modelos de transferencia de activos (después de Interoperabilidad y concepto de puente) [Necesario - Importante] 88
2.5.3 Modelos de verificación y supuestos de confianza (después de Modelos de transferencia de activos) [Necesario - Crítico] 88
2.5.4 Incidentes de seguridad (después de Modelos de verificación y supuestos de confianza) [Necesario - Crítico] 88
2.5.5 Puentes basados en pruebas de conocimiento cero (después de Incidentes de seguridad) [Necesario - Importante] 89
2.5.6 Requisitos del puente propuesto (al final de la Sección 2.5 Puentes) [Sugerencia - Recomendado] 89
4.5 Metodología de comparación (después de Diseño e implementación en SP1) [Necesario - Crítico] 89
4.6 Resultados y discusión (después de Metodología de comparación) [Necesario - Crítico] 89
5.1 Conclusiones (al inicio del Capítulo 5) [Necesario - Crítico] 89
5.2 Verificación de la hipótesis (después de Conclusiones) [Necesario - Crítico] 89
5.3 Trabajos futuros (después de Verificación de la hipótesis) [Necesario - Importante] 89
A.1 Código fuente de los circuitos (en el Apéndice, reemplazando la sección de plantilla) [Sugerencia - Recomendado] 89
A.2 Entorno y datos de las mediciones (en el Apéndice, después de Código fuente de los circuitos) [Sugerencia - Recomendado] 90

LISTA DE FIGURAS

Página

2.1. Distribución del pago de una transacción según EIP-1559. 10
2.2. Cadena de compromisos criptográficos que vincula un evento de bloqueo con la firma del comité de sincronización de Ethereum. 15
2.3. Recorrido de una transacción de Aztec desde su ejecución en el dispositivo del usuario hasta su asentamiento en Ethereum. 20
2.4. Componentes del OP Stack y su relación con los contratos desplegados en Ethereum. 23

3.1. Etapas de generación y verificación de una prueba. 32
3.2. Circuito Circom que expresa la relación $w^3 + w + 5 = y$. 37
3.3. Proceso de compilación, cálculo del testigo, generación y verificación de una prueba a partir de un circuito Circom. 38
3.4. Plantilla `IsZero` de circomlib, que combina un cálculo auxiliar con restricciones explícitas. 39
3.5. Programa Noir que expresa la relación $w^3 + w + 5 = y$. 42
3.6. Proceso de compilación, cálculo del testigo, generación y verificación de una prueba a partir de un programa Noir con el backend Barretenberg. 44
3.7. Cálculo no restringido de un inverso en Noir, seguido de la restricción que lo valida. 47
3.8. Programa invitado de SP1 que calcula $y = w^3 + w + 5$ con aritmética entera comprobada. 51
3.9. Programa anfitrión que prepara la entrada, genera y verifica una prueba comprimida y lee el valor público. 52
3.10. Proceso de compilación, ejecución, generación y verificación de una prueba con SP1. 53

4.1. Arquitectura del puente y recorrido de una transferencia de ida. Las líneas discontinuas indican eventos y los números, las etapas descritas en el texto. 61

LISTA DE TABLAS

Página

2.1. Campos principales del encabezado de un bloque de ejecución de Ethereum. . . 12
2.2. Árboles de estado de Aztec. 18
2.3. Estados de confirmación de un bloque de Optimism. 23

3.1. Diferencias conceptuales entre SNARKs y STARKs. 34
3.2. Herramientas y entornos del ecosistema de pruebas de conocimiento cero y su relación con el caso de estudio. 58

4.1. Valores públicos de los programas de consenso y de inclusión y uso que hacen de ellos los contratos de destino. 69

LISTA DE SÍMBOLOS

H	Función de hash criptográfica.
$\|\|$	Concatenación de cadenas de bits.
σ	Estado global de Ethereum.
Υ	Función de transición de estado de Ethereum.
T	Transacción de Ethereum.
g_{lim}	Límite de gas declarado por una transacción.
g_{usado}	Gas efectivamente consumido por una transacción.
g_{bloque}	Gas total consumido por un bloque.
g_{obj}	Gas objetivo por bloque según EIP-1559.
p_{base}	Tarifa base por unidad de gas.
p'_{base}	Tarifa base por unidad de gas del bloque siguiente.
p_{prop}	Propina efectiva por unidad de gas.
$p_{\text{máx}}$	Precio máximo por unidad de gas declarado por el emisor.
$p^{\text{prop}}_{\text{máx}}$	Propina máxima por unidad de gas declarada por el emisor.
$\mathcal{C}$	Comité de sincronización de la *Beacon Chain*.
$\mathcal{C}'$	Subconjunto de miembros del comité de sincronización que participaron en una firma.
pk_i	Clave pública BLS del miembro i del comité de sincronización.
pk_{agg}	Clave pública BLS agregada.
sig	Firma BLS agregada.
$\mathbb{G}_1$	Primer grupo de origen del emparejamiento bilineal.
$\mathbb{G}_2$	Segundo grupo de origen del emparejamiento bilineal.
$\mathbb{G}_T$	Grupo de destino del emparejamiento bilineal.
e	Emparejamiento bilineal $e\colon \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$.
g_1	Generador del grupo $\mathbb{G}_1$.
$H_{\mathbb{G}_2}$	Función de hash que transforma mensajes en puntos de $\mathbb{G}_2$.
v	Valor almacenado en una hoja de un árbol de Merkle indexado.
i_{sig}	Índice de la hoja que contiene el siguiente valor mayor en un árbol de Merkle indexado.

v_{sig}	Siguiente valor mayor almacenado en un árbol de Merkle indexado.
$\mathcal{P}$	Probador (*prover*) de un sistema de prueba.
$\mathcal{P}^*$	Probador potencialmente deshonesto.
$\mathcal{V}$	Verificador (*verifier*) de un sistema de prueba.
x	Enunciado (*statement*); entrada pública.
w	Testigo (*witness*); entrada privada del probador.
π	Prueba (*proof*) generada por el probador.
$\mathcal{R}$	Relación que define los pares válidos de enunciado y testigo.
$\mathbb{F}_p$	Cuerpo finito de p elementos, con p primo.
A, B, C	Combinaciones lineales de señales en una restricción cuadrática $A \cdot B + C = 0$ de Circom.
T_{Ethereum}	Tiempo de ejecución del anfitrión de SP1 en el modo que genera la prueba Groth16 destinada a Ethereum.
$T_{\text{comprimido}}$	Tiempo de ejecución del anfitrión de SP1 en el modo que genera la prueba comprimida.

CAPÍTULO 1
INTRODUCCIÓN

1.1. Contexto y antecedentes

Ethereum se ha consolidado como la plataforma de referencia para el despliegue de contratos inteligentes y constituye la base sobre la que se sostiene una porción significativa del ecosistema de finanzas descentralizadas [1]. Este crecimiento no se produjo de forma aislada, sino que estuvo acompañado por la proliferación de redes de *Layer 2* y cadenas alternativas que replican o extienden su modelo de ejecución, cada una optimizada según distintos criterios de costo, rendimiento o descentralización. La fragmentación resultante de este ecosistema plantea la necesidad de que los activos digitales y los datos puedan trasladarse entre las distintas cadenas que lo componen, requisito que se conoce como interoperabilidad. Los puentes entre cadenas, o *cross-chain bridges*, constituyen la infraestructura que hace posible dicho traslado, de modo que la seguridad de estos mecanismos determina, en gran medida, la seguridad de los fondos que a través de ellos se transfieren.

Paralelamente al desarrollo de estas redes, las pruebas de conocimiento cero (*Zero-Knowledge Proofs*) evolucionaron desde ser un objeto de estudio predominantemente teórico hasta convertirse en una herramienta de ingeniería aplicable a problemas concretos. Esta tecnología permite comprimir la verificación de un cómputo arbitrariamente extenso en una prueba sucinta, cuya validación puede realizarse en una fracción del tiempo y del costo que exigiría reejecutar dicho cómputo. Aplicada al contexto de Ethereum, esta propiedad habilita la verificación liviana, desde otra cadena, tanto del consenso alcanzado por los validadores —a través de la firma emitida por el comité de sincronización que certifica la finalidad de un bloque— como de la inclusión efectiva de una transacción particular dentro de ese bloque, lo cual se logra mediante una prueba sobre su árbol de Merkle-Patricia. La combinación de ambas verificaciones, comprimidas en una única prueba de conocimiento cero, constituye el fundamento técnico sobre el que resulta

posible construir un puente entre cadenas cuya seguridad no dependa de la honestidad de un intermediario, sino de una demostración matemáticamente verificable. En el presente trabajo, este fundamento se materializa mediante el diseño e implementación de un puente del tipo *lock-and-mint*, en el cual el bloqueo de un activo en la cadena de origen habilita su acuñación equivalente en la cadena de destino únicamente cuando dicha prueba ha sido validada *on-chain*.

1.2. Planteamiento del problema

La verificación directa del consenso de Ethereum y de la inclusión de una transacción, sin comprimir dicho cómputo mediante una prueba de conocimiento cero, resulta impracticable cuando se pretende realizar fuera de la propia cadena. Reproducir la agregación de las firmas del comité de sincronización y recorrer un árbol de Merkle-Patricia dentro de un contrato inteligente desplegado en otra cadena implica un volumen considerable de operaciones criptográficas, fundamentalmente sobre curvas elípticas, cuyo costo en gas resulta, en la práctica, prohibitivo. Esta limitación conduce a que los sistemas de interoperabilidad existentes adopten un compromiso distinto, en el cual la verificación criptográfica del estado de la cadena de origen es sustituida por la delegación de dicha verificación en un comité de validadores externos sobre el cual se deposita confianza. Este diseño traslada el problema de seguridad del propio protocolo hacia el comportamiento honesto de un conjunto reducido de actores, y esta delegación de confianza ha constituido, de manera sistemática, el punto de falla explotado en los incidentes de seguridad más costosos sufridos por puentes entre cadenas hasta la fecha [2]. Persiste, en consecuencia, una tensión no resuelta entre dos objetivos igualmente necesarios: por un lado, alcanzar garantías criptográficas equivalentes a las del consenso de la cadena de origen y, por otro, mantener un costo de verificación *on-chain*, tanto en gas como en tiempo de generación de la prueba, que resulte viable para un sistema de uso real.

1.3. Justificación

El ecosistema de herramientas orientadas al desarrollo de circuitos de conocimiento cero capaces de abordar este volumen de cómputo criptográfico se encuentra, a su vez, en un proceso activo de maduración, y ofrece actualmente dos paradigmas de diseño claramente diferenciados. El primero está representado por los lenguajes de dominio específico, orientados exclusivamente a la escritura de circuitos, entre los cuales se encuentra Noir [3], que expone al desarrollador primitivas y un modelo de programación concebidos desde su diseño para la generación eficiente de pruebas. El segundo paradigma corresponde a las máquinas virtuales de conocimiento cero de propósito general, cuyo exponente más reciente es SP1 [4], las cuales permiten probar la ejecución de programas escritos en un lenguaje de propósito general, en este caso Rust, compilados a un conjunto de instrucciones que la máquina virtual ejecuta y prueba sin que el desarrollador deba razonar directamente en términos de restricciones aritméticas. Si bien ambos paradigmas

persiguen resolver el mismo tipo de problema, lo hacen mediante compromisos de diseño distintos entre la expresividad del lenguaje, la facilidad de desarrollo y depuración, y el rendimiento final del circuito resultante. Dado que ambas herramientas son de adopción reciente y se encuentran en desarrollo activo, no existe todavía evidencia empírica consolidada que permita fundamentar la elección de uno u otro paradigma para la implementación de un caso de uso concreto y computacionalmente exigente, como lo es la verificación de consenso e inclusión de transacciones en Ethereum. El presente trabajo se propone generar dicha evidencia mediante la implementación de un mismo problema en ambas tecnologías, bajo condiciones comparables, y la medición sistemática de su comportamiento. Ambas implementaciones se integran, además, dentro del mismo puente *lock-and-mint*, de manera que la comparación entre Noir y SP1 no se limita a un ejercicio aislado de generación de pruebas, sino que se valida en el contexto de una transferencia real de activos digitales entre cadenas.

1.4. Objetivos

1.4.1. Objetivo general

Diseñar, implementar y comparar empíricamente la verificación de consenso de Ethereum y de inclusión de transacciones mediante Noir y SP1, integrando ambas implementaciones en un puente *lock-and-mint* de confianza reducida, con el propósito de determinar cuál de ambas tecnologías ofrece un mejor rendimiento, un menor costo de verificación y una mayor viabilidad práctica para la transferencia segura de activos digitales entre cadenas.

1.4.2. Objetivos específicos

- Investigar los mecanismos de prueba y las arquitecturas de circuitos que subyacen a Noir, en tanto lenguaje de dominio específico, y a SP1, en tanto máquina virtual de conocimiento cero de propósito general basada en Rust.
- Implementar, en ambas tecnologías, la lógica de verificación de encabezados de bloque de Ethereum, de la firma del comité de sincronización y de las pruebas de inclusión de transacciones.
- Medir y comparar las métricas de rendimiento relevantes entre ambas implementaciones, entre las que se incluyen el tiempo de generación de la prueba, el uso de recursos computacionales y de memoria, el costo de verificación *on-chain* en gas, y la complejidad relativa del proceso de desarrollo.

1.5. Hipótesis

El presente trabajo se orienta por la siguiente pregunta de investigación: ¿presenta el paradigma de máquina virtual de conocimiento cero de propósito general, representado por SP1, ventajas significativas en términos de flexibilidad de desarrollo y de tiempos de ejecución frente a un lenguaje de conocimiento cero de dominio específico como Noir, al implementar la verificación de consenso e inclusión de transacciones en Ethereum?

1.6. Delimitación

El presente trabajo se circunscribe a la arquitectura de consenso de Ethereum posterior a *The Merge*, particularmente al mecanismo del comité de sincronización empleado por los clientes ligeros para certificar la finalidad de un bloque, así como a las pruebas de inclusión sobre su árbol de Merkle-Patricia. Las implementaciones desarrolladas y las comparativas de rendimiento realizadas se validan sobre redes de prueba, empleando Ethereum Sepolia como cadena de origen y Optimism Sepolia como cadena de destino, bajo una infraestructura de cómputo estandarizada que permite que las mediciones obtenidas para Noir y para SP1 resulten comparables entre sí. Quedan fuera del alcance de este trabajo tanto la implementación sobre redes principales como la verificación de mecanismos de consenso correspondientes a cadenas distintas de Ethereum.

CAPÍTULO 2

BLOCKCHAIN

2.1. Arquitectura de blockchain

Una *blockchain* es un registro distribuido (*distributed ledger*) de transacciones que se mantiene de forma replicada entre los nodos de una red *peer-to-peer*, sin depender de una autoridad central que valide o custodie su contenido. Su propiedad fundamental es que, una vez que un dato se incorpora al registro, modificarlo o eliminarlo requiere rehacer todo el trabajo criptográfico posterior a ese punto, lo cual se vuelve computacionalmente inviable a medida que la cadena crece. Esta sección describe los cuatro componentes que en conjunto sostienen esa propiedad: la estructura de bloques encadenados por hash, los árboles de Merkle que resumen su contenido, los mecanismos de consenso que deciden qué bloque es válido, y los roles que los distintos tipos de nodos cumplen dentro de la red. Estos conceptos son la base sobre la que se apoyan tanto el diseño de Ethereum (Sección 2.2) como el esquema de verificación de conocimiento cero desarrollado en este trabajo.

2.1.1. Estructura de bloques y encadenamiento por hash

Definición 2.1. Un **bloque** es una estructura de datos compuesta por un *encabezado* (*header*) y un cuerpo. El encabezado contiene, como mínimo, el hash criptográfico del encabezado del bloque anterior, una marca de tiempo, un valor que resume el conjunto de transacciones del bloque (la raíz de un árbol de Merkle, ver Definición 2.2), y metadatos propios del mecanismo de consenso utilizado (por ejemplo, un *nonce* en *Proof-of-Work*). El cuerpo contiene la lista de transacciones incluidas en el bloque.

La **blockchain** propiamente dicha es la secuencia de bloques $B_0, B_1, \ldots, B_n$ donde B_0 es el *bloque génesis* y cada bloque B_i, con $i > 0$, referencia a su predecesor incluyendo $H(B_{i-1})$

en su propio encabezado, siendo H una función de hash criptográfica (típicamente SHA-256 o Keccak-256). Esta construcción encadenada es lo que le da su nombre a la estructura y es la responsable directa de su inmutabilidad. Como el hash de un bloque depende del hash del bloque anterior, alterar el contenido de B_i cambia su hash $H(B_i)$, lo que invalida la referencia almacenada en B_{i+1}, y por extensión la de todos los bloques posteriores hasta la cabeza de la cadena. Un atacante que quisiera modificar una transacción confirmada tendría entonces que recalcular el trabajo de consenso de todos los bloques desde ese punto en adelante, y hacerlo más rápido que el resto de la red honesta le suma nuevos bloques a la cadena legítima.

2.1.2. Árboles de Merkle y pruebas de inclusión

Incluir el hash de cada transacción individual en el encabezado del bloque sería costoso en espacio y haría que verificar la pertenencia de una transacción específica implicara descargar el bloque completo. Los árboles de Merkle [5] resuelven ambos problemas.

Definición 2.2. Un **árbol de Merkle** es un árbol binario en el que cada hoja contiene el hash de un elemento de datos (en este contexto, una transacción) y cada nodo interno contiene el hash de la concatenación de sus dos hijos: $h_{padre} = H(h_{izq} \| h_{der})$. El hash almacenado en la raíz, llamado **raíz de Merkle**, es un compromiso criptográfico sobre el conjunto completo de hojas: cambiar cualquier transacción altera su hash hoja y, por propagación, la raíz.

Esta estructura permite generar una **prueba de inclusión** (*Merkle proof*) de que una transacción t forma parte de un bloque sin necesitar el bloque completo: basta con entregar t junto con los hashes hermanos de cada nivel del camino desde su hoja hasta la raíz. Quien verifica recalcula los hashes de ese camino y comprueba que el resultado coincide con la raíz de Merkle publicada en el encabezado del bloque. Para un árbol con n hojas, la prueba tiene tamaño $O(\log n)$, en contraste con el $O(n)$ que implicaría transmitir todas las transacciones. Ethereum extiende esta idea con *Merkle-Patricia tries*, que además de las transacciones permiten probar el estado de una cuenta o el resultado de una ejecución (Sección 2.2); esta capacidad de generar pruebas de inclusión compactas y verificables es, precisamente, el mecanismo que este trabajo utiliza para certificar *on-chain*, mediante una prueba de conocimiento cero, que una transacción de bloqueo fue efectivamente incluida en la cadena de origen.

2.1.3. Mecanismos de consenso

Que cada nodo mantenga una copia del registro no es suficiente: en una red sin autoridad central, distintos nodos pueden recibir transacciones en distinto orden o proponer bloques en simultáneo, y es necesario que todos converjan a una única secuencia de bloques válida. Este es un caso particular del problema de los generales bizantinos [6]: lograr acuerdo entre participantes distribuidos cuando una fracción de ellos puede ser maliciosa o fallar arbitrariamente. Un **mecanismo de consenso** es el protocolo que resuelve este problema definiendo, ante bloques o cadenas en conflicto, cuál es la versión canónica.

Los dos mecanismos dominantes son:

- **Proof-of-Work (PoW)**: introducido por Bitcoin [7], exige que quien propone un bloque resuelva un problema computacional costoso (encontrar un *nonce* tal que $H(\text{header})$ sea menor a un umbral de dificultad). La cadena válida es la que acumula mayor trabajo computacional total, lo que hace que reescribir el historial requiera superar la capacidad de cómputo de toda la red honesta. Su finalidad es *probabilística*: la confianza en que un bloque no será revertido crece con la cantidad de bloques confirmados sobre él, pero nunca es absoluta.

- **Proof-of-Stake (PoS)**: sustituye el costo computacional por un costo económico. Los participantes (*validadores*) bloquean una cantidad de la criptomoneda nativa como garantía y son seleccionados para proponer y atestiguar bloques en proporción a su participación bloqueada (*stake*). Un comportamiento malicioso detectable (como firmar dos bloques en conflicto para la misma posición) resulta en la pérdida de parte o la totalidad de esa garantía (*slashing*). A diferencia de PoW, las variantes modernas de PoS —como la que adoptó Ethereum tras *The Merge*— pueden alcanzar **finalidad determinística**: una vez que un bloque acumula suficientes atestiguaciones, revertirlo es no solo improbable sino que exige destruir económicamente a los validadores responsables. Este mecanismo de finalidad, y el comité que la certifica en Ethereum, se retoma en la Sección 2.2 y es la raíz de confianza sobre la que se construye el puente propuesto en este trabajo.

2.1.4. Roles de nodos y topología de red

No todos los participantes de una red blockchain cumplen la misma función ni requieren los mismos recursos:

- **Nodos completos** (*full nodes*): descargan y validan de forma independiente cada bloque y cada transacción desde el bloque génesis, manteniendo una copia íntegra del estado de la red. No confían en ningún tercero: verifican por sí mismos que se respetaron las reglas de consenso y de validez de transacciones.

- **Mineros o validadores**: además de validar, participan activamente en la producción de nuevos bloques (resolviendo el rompecabezas de PoW o siendo seleccionados y atestiguando en PoS).

- **Clientes ligeros** (*light clients*): no almacenan la cadena completa ni reejecutan todas las transacciones; en su lugar, confían en los encabezados de bloque —en particular en su raíz de Merkle y, en redes PoS, en la firma del comité que certifica su finalidad— para verificar de forma eficiente si una transacción específica fue incluida, mediante una prueba de inclusión como la descrita en la Sección 2.1.2. Este es precisamente el modelo de confianza que adopta el puente diseñado en este trabajo, cuyos contratos en la cadena de

destino no reejecutan la cadena de origen, sino que verifican pruebas de conocimiento cero de la firma del comité de sincronización de Ethereum sobre un bloque y de la inclusión de la transacción en ese bloque. El *relayer off-chain* se limita a generar dichas pruebas y a remitirlas a esos contratos.

Todos estos roles se comunican mediante una red *peer-to-peer* de propagación por *gossip*, en la que cada nodo retransmite a sus pares los bloques y transacciones que recibe y considera válidos, sin que exista un servidor central que coordine la difusión de la información.

2.2. Ethereum

Ethereum es una plataforma *blockchain* propuesta por Buterin en 2014 [1] y especificada formalmente por Wood en el denominado *Yellow Paper* [8]. A diferencia de Bitcoin, cuyo lenguaje de *scripts* se limita deliberadamente a la validación de transferencias, Ethereum incorpora una máquina virtual capaz de ejecutar programas arbitrarios, denominados *contratos inteligentes* (*smart contracts*), cuyo código y estado forman parte del propio registro. Esta capacidad convierte a la cadena en una máquina de estados replicada de propósito general y es la que permite desplegar en ella la lógica de un puente entre cadenas. La presente sección describe el modelo de estado de Ethereum, la estructura de sus bloques, los árboles que resumen su contenido y el mecanismo de consenso que certifica su finalidad, con especial atención a los elementos que el puente desarrollado en este trabajo debe verificar.

2.2.1. Modelo de cuentas y máquina virtual

Ethereum adopta un modelo basado en cuentas, en contraposición al modelo de salidas no gastadas (UTXO) de Bitcoin. El **estado global** σ es una correspondencia entre direcciones de 160 bits y estados de cuenta, y cada cuenta se describe mediante cuatro campos, que son el *nonce*, el saldo expresado en wei, la raíz del árbol que almacena su espacio de almacenamiento persistente (*storageRoot*) y el hash de su código ejecutable (*codeHash*) [8]. Se distinguen dos clases de cuentas. Las *cuentas de propiedad externa* (*externally owned accounts*) están controladas por una clave privada y carecen de código, mientras que las *cuentas de contrato* carecen de clave privada y se rigen exclusivamente por el código que tienen asociado.

Una transacción es una instrucción firmada por una cuenta de propiedad externa que transfiere valor, invoca una función de un contrato o despliega un contrato nuevo. Su aplicación sobre el estado se describe mediante una función de transición $\sigma_{t+1} = \Upsilon(\sigma_t, T)$, donde T denota la transacción y Υ la función de transición de estado definida en el *Yellow Paper*. La ejecución de contratos tiene lugar en la **Ethereum Virtual Machine** (EVM), una máquina de pila con palabras de 256 bits cuyo comportamiento es completamente determinista, de modo que todos los nodos que procesan la misma secuencia de transacciones obtienen el mismo estado resultante. El costo de dicha ejecución se mide en *gas*, concepto que se desarrolla en la Sección 2.2.2.

Los contratos pueden emitir **eventos** (*logs*) durante su ejecución. Un evento consta de la dirección del contrato emisor, una lista de hasta cuatro *tópicos* de 32 bytes, el primero de los cuales identifica habitualmente la firma del evento, y un campo de datos de longitud arbitraria. Los eventos no forman parte del estado ni pueden ser leídos por otros contratos, pero quedan registrados de forma permanente en el recibo de la transacción que los originó. Este mecanismo es el que utiliza el contrato de bloqueo del puente para dejar constancia verificable de cada depósito realizado en la cadena de origen.

2.2.2. Gas y costo de ejecución

Una máquina virtual capaz de ejecutar programas arbitrarios enfrenta el problema de que no es posible determinar de antemano si un programa concluirá, lo que constituye una consecuencia del problema de la detención. Si la red ejecutara sin restricción cualquier código recibido, un único contrato con un ciclo infinito bastaría para detener a todos los nodos. Ethereum resuelve este problema asociando un costo a cada operación que realiza la EVM [8].

Definición 2.3. El **gas** es la unidad que mide el esfuerzo computacional requerido para ejecutar una transacción en la EVM. Cada instrucción tiene asignado un costo fijo en gas, proporcional a los recursos de cómputo, memoria y almacenamiento que consume, y toda transacción declara un **límite de gas** g_{lim}, que es la cantidad máxima de gas que su emisor está dispuesto a consumir. El gas efectivamente consumido durante la ejecución se denota g_{usado} y satisface $g_{\text{usado}} \leq g_{\text{lim}}$.

El gas es independiente del precio de la moneda nativa. La cantidad de gas que requiere una operación depende exclusivamente de su complejidad, mientras que el precio que se paga por cada unidad de gas se determina por la demanda de espacio en los bloques. Toda transacción consume un costo intrínseco de 21 000 unidades, al que se suma el costo de cada instrucción ejecutada. Las operaciones aritméticas elementales cuestan pocas unidades, mientras que las que modifican el almacenamiento persistente se encuentran entre las más costosas, puesto que su efecto debe ser conservado por todos los nodos de la red. Los precios se expresan habitualmente en *gwei*, equivalente a 10^9 wei, siendo un ether igual a 10^{18} wei.

Desde la propuesta EIP-1559, el precio por unidad de gas se compone de dos partes [9]. La primera es la **tarifa base** p_{base}, que el protocolo fija para cada bloque y que se destruye en lugar de transferirse a algún participante. La segunda es la **propina** p_{prop}, que se transfiere al validador que propone el bloque y actúa como incentivo para que incluya la transacción. Al enviar una transacción, el emisor no fija estos valores directamente, sino que declara un precio máximo por unidad de gas $p_{\text{máx}}$ y una propina máxima $p_{\text{máx}}^{\text{prop}}$. La propina efectiva y el costo total de la transacción quedan determinados por las expresiones

$$p_{\text{prop}} = \min\!\left(p_{\text{máx}}^{\text{prop}},\; p_{\text{máx}} - p_{\text{base}}\right), \tag{2.1}$$

$$\mathit{costo} = g_{\text{usado}} \cdot \left(p_{\text{base}} + p_{\text{prop}}\right). \tag{2.2}$$

Una transacción solo es válida si $p_{\text{máx}} \geq p_{\text{base}}$. Antes de ejecutarla, el protocolo descuenta del saldo del emisor la cantidad $g_{\text{lim}} \cdot p_{\text{máx}}$, que representa el costo máximo posible, y una vez concluida la ejecución reintegra la diferencia entre dicha reserva y el costo de la Ecuación (2.2). La Figura 2.1 representa esta distribución como un área, en la que el eje horizontal corresponde a la cantidad de gas y el eje vertical al precio por unidad.

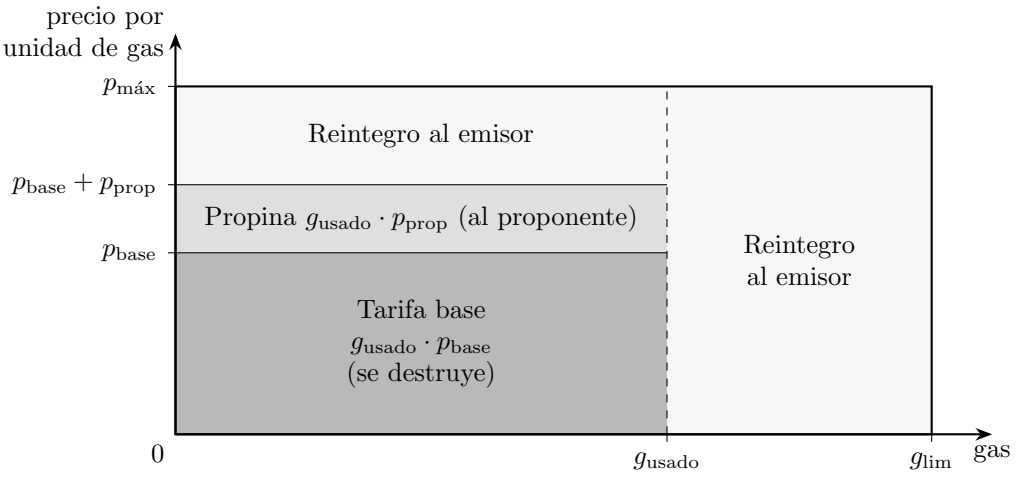


Figura 2.1: Distribución del pago de una transacción según la propuesta EIP-1559. El rectángulo completo corresponde a la reserva inicial $g_{\text{lim}} \cdot p_{\text{máx}}$.

La tarifa base se ajusta automáticamente de un bloque a otro en función de la congestión de la red. Cada bloque tiene un límite de gas total y un *objetivo* g_{obj} equivalente a la mitad de dicho límite. Si el gas consumido por un bloque g_{bloque} supera el objetivo, la tarifa base del bloque siguiente aumenta, y si es inferior, disminuye, según la regla

$$p'_{\text{base}} = p_{\text{base}} \left(1 + \frac{1}{8} \cdot \frac{g_{\text{bloque}} - g_{\text{obj}}}{g_{\text{obj}}}\right), \tag{2.3}$$

de modo que la variación entre dos bloques consecutivos no excede el 12,5 % [9]. Este mecanismo hace que el costo de una misma operación, expresado en ether, varíe con la demanda, aunque su consumo de gas permanezca constante. Por esta razón, el costo de verificación *on-chain* se reporta en este trabajo en unidades de gas, que constituyen una medida reproducible e independiente de las condiciones del mercado.

Si durante la ejecución el gas consumido alcanza el límite declarado, la EVM detiene la ejecución con una excepción de *gas agotado* (*out of gas*). En ese caso, todas las modificaciones del estado realizadas por la transacción se revierten, pero el gas consumido no se reintegra, ya que el trabajo de los nodos que la procesaron ya fue realizado. La transacción queda igualmente incluida en el bloque y su recibo registra el fallo mediante el código de estado descrito en la Sección 2.2.5. Este comportamiento impide que un emisor obtenga cómputo gratuito de la red y garantiza que toda ejecución concluya en una cantidad finita de pasos.

Además del conjunto ordinario de instrucciones, la EVM dispone de *contratos precompilados*, que son direcciones reservadas cuya funcionalidad se implementa de forma nativa en el cliente

y se cobra a un costo de gas muy inferior al que tendría la misma operación programada con instrucciones ordinarias. Entre ellos se encuentran la suma, la multiplicación escalar y la comprobación de emparejamientos sobre la curva BN254, incorporadas mediante las propuestas EIP-196 y EIP-197 [10, 11]. La propuesta EIP-1108 redujo posteriormente su costo, de modo que la comprobación de un producto de k emparejamientos requiere $34\,000 \cdot k + 45\,000$ unidades de gas [12]. Para una prueba Groth16, cuya verificación involucra cuatro emparejamientos, esta comprobación cuesta 181 000 unidades de gas, cifra a la que se añaden las operaciones necesarias para procesar las entradas públicas. Estas operaciones son las que hacen económicamente viable verificar dentro de un contrato inteligente una prueba de conocimiento cero de tipo SNARK, aspecto que se desarrolla en el Capítulo 3. Para la curva BLS12-381, empleada por el consenso de Ethereum, la propuesta EIP-2537 incorporó precompilados análogos recién en la actualización Pectra de 2025 [13]. Su disponibilidad no está garantizada en otras cadenas que pudieran actuar como destino de un puente y, aun con ellos, la verificación completa de una actualización del comité de sincronización conserva un costo elevado, según se expone en la Sección 2.2.8.

2.2.3. Arquitectura en dos capas

Desde su lanzamiento en 2015 y hasta septiembre de 2022, Ethereum utilizó un mecanismo de consenso de tipo *Proof-of-Work*. La actualización conocida como *The Merge*, especificada en la propuesta EIP-3675, sustituyó dicho mecanismo por un consenso de tipo *Proof-of-Stake* provisto por la *Beacon Chain*, una cadena que operaba en paralelo desde diciembre de 2020 [14]. Como consecuencia de esta transición, cada nodo de Ethereum se compone actualmente de dos clientes que cooperan entre sí. El **cliente de ejecución** mantiene el estado global, ejecuta las transacciones en la EVM y construye los bloques de ejecución, mientras que el **cliente de consenso** gestiona el conjunto de validadores, determina quién propone cada bloque y decide qué bloques se consideran finalizados.

Esta separación se refleja en la estructura de los datos. Cada bloque de la *Beacon Chain* contiene, dentro de su cuerpo, un *execution payload* que corresponde al bloque de ejecución tradicional [15]. En consecuencia, la autenticidad de un bloque de ejecución no se deriva de una prueba de trabajo, sino de su inclusión en un bloque de consenso que los validadores han atestiguado. Ambas capas emplean, además, funciones de hash distintas. La capa de ejecución utiliza Keccak-256 y la codificación RLP (*Recursive Length Prefix*), mientras que la capa de consenso utiliza SHA-256 y la serialización SSZ (*Simple Serialize*), cuya función `hash_tree_root` organiza los campos de cada estructura en un árbol de Merkle binario [16]. Un verificador que pretenda enlazar ambas capas debe, por lo tanto, calcular los dos tipos de hash.

2.2.4. Estructura del bloque de ejecución

El encabezado de un bloque de ejecución agrupa los compromisos criptográficos sobre el estado y el contenido del bloque, junto con los metadatos necesarios para validarlo. Su identificador, denominado *block hash*, es el hash Keccak-256 de la codificación RLP del encabezado

completo. La Tabla 2.1 resume los campos más relevantes para este trabajo.

Tabla 2.1: Campos principales del encabezado de un bloque de ejecución de Ethereum.

Campo	**Descripción**
`parentHash`	Hash Keccak-256 del encabezado del bloque predecesor.
`beneficiary`	Dirección que recibe las propinas de las transacciones del bloque.
`stateRoot`	Raíz del árbol de estado tras ejecutar todas las transacciones del bloque.
`transactionsRoot`	Raíz del árbol que contiene las transacciones del bloque.
`receiptsRoot`	Raíz del árbol que contiene los recibos de dichas transacciones.
`logsBloom`	Filtro de Bloom de 2048 bits sobre las direcciones y tópicos de los eventos emitidos.
`number`	Altura del bloque, es decir, cantidad de bloques que lo preceden.
`gasLimit`, `gasUsed`	Límite de gas admitido por el bloque y gas efectivamente consumido.
`timestamp`	Marca de tiempo Unix del bloque.
`prevRandao`	Valor pseudoaleatorio aportado por la capa de consenso, que reemplaza al antiguo `mixHash` [17].
`baseFeePerGas`	Tarifa base por unidad de gas vigente en el bloque [9].

Tras *The Merge*, los campos asociados a la prueba de trabajo, como la dificultad y el *nonce*, se mantienen en el encabezado con valores constantes a fin de preservar la compatibilidad del formato [14]. Las actualizaciones posteriores del protocolo añadieron campos nuevos, entre ellos la raíz del bloque de consenso predecesor (`parentBeaconBlockRoot`), que permite a los contratos de la propia cadena acceder a los compromisos de la capa de consenso [18]. Del conjunto de campos descritos, el puente desarrollado en este trabajo se apoya en `transactionsRoot`, que permite demostrar la inclusión de la transacción de bloqueo, y en `receiptsRoot`, que permite demostrar que esa transacción se ejecutó con éxito.

2.2.5. Árboles de Merkle-Patricia

Los compromisos del encabezado no se construyen con árboles de Merkle binarios como los descritos en la Sección 2.1.2, sino con **árboles de Merkle-Patricia** (*Merkle-Patricia tries*), una estructura que combina un árbol de prefijos (*Patricia trie*) con el direccionamiento por hash propio de los árboles de Merkle [8]. Las claves se interpretan como secuencias de *nibbles* de 4 bits y cada nodo pertenece a uno de tres tipos. Un *nodo rama* contiene dieciséis referencias, una por cada valor posible del siguiente *nibble*, además de un valor opcional. Un *nodo extensión* comprime un tramo de la clave compartido por todos sus descendientes, y un *nodo hoja* almacena el sufijo restante de la clave junto con el valor asociado. Cada nodo se codifica

en RLP y es referenciado por su padre mediante el hash Keccak-256 de dicha codificación, salvo cuando esta ocupa menos de 32 bytes, en cuyo caso se incrusta directamente. De esta forma, la raíz del árbol constituye un compromiso sobre el conjunto completo de pares clave-valor.

Ethereum mantiene cuatro clases de árboles de este tipo. El **árbol de estado** asocia el hash Keccak-256 de cada dirección con el estado de la cuenta correspondiente. Cada cuenta de contrato posee, a su vez, un **árbol de almacenamiento** cuya raíz figura en su campo *storageRoot*. Los **árboles de transacciones** y **de recibos** se construyen por bloque y utilizan como clave la codificación RLP del índice de cada transacción dentro del bloque. Mientras que el árbol de estado persiste y se actualiza de un bloque a otro, los dos últimos se reconstruyen para cada bloque y solo su raíz queda registrada en el encabezado.

Una prueba de inclusión en un árbol de Merkle-Patricia consiste en la secuencia de nodos que recorre el camino desde la raíz hasta la hoja que contiene el valor buscado. El verificador comprueba que el hash de cada nodo coincide con la referencia almacenada en su predecesor y que los *nibbles* consumidos a lo largo del recorrido reproducen la clave esperada. La longitud de la prueba crece de forma logarítmica con el número de elementos del árbol, aunque cada nodo rama puede ocupar hasta 532 bytes, lo que hace que estas pruebas sean considerablemente más extensas que las de un árbol binario. Su verificación exige, además, decodificar RLP y calcular Keccak-256, operaciones que resultan costosas de expresar como circuito, según se analiza en la Sección 3.2.4.

El **recibo** de una transacción registra el resultado de su ejecución. Contiene un código de estado que indica si la ejecución concluyó con éxito o fue revertida [19], el gas acumulado consumido en el bloque hasta esa transacción, un filtro de Bloom y la lista de eventos emitidos. La distinción entre la inclusión de una transacción y su ejecución exitosa es relevante para un puente, ya que una transacción revertida figura igualmente en el árbol de transacciones a pesar de no haber modificado el estado. Por esta razón, la prueba que necesita el puente no puede limitarse al árbol de transacciones y debe abarcar también el árbol de recibos, en el que puede comprobarse el código de estado de la transacción de bloqueo.

2.2.6. Consenso y finalidad

El consenso de Ethereum, denominado Gasper, combina una regla de selección de bifurcación derivada de GHOST, que determina la cabeza de la cadena en cada instante, con el mecanismo de finalidad Casper FFG (*Friendly Finality Gadget*) [20, 21]. El tiempo se divide en *slots* de 12 segundos, en cada uno de los cuales un validador seleccionado de forma pseudoaleatoria puede proponer un bloque, y los slots se agrupan en *épocas* de 32 slots, equivalentes a 6,4 minutos [15]. Para participar como validador es necesario depositar 32 ETH en garantía, y durante cada época todos los validadores activos se distribuyen en comités que emiten *atestaciones* sobre la cabeza de la cadena y sobre el primer bloque de la época, llamado *punto de control* (*checkpoint*).

Casper FFG opera sobre estos puntos de control. Cuando validadores que representan al menos dos tercios del saldo total en garantía atestiguan un enlace entre un punto de control

justificado y otro posterior, este último pasa a estar *justificado*, y cuando dos puntos de control consecutivos quedan justificados, el primero de ellos se considera *finalizado*. En condiciones normales, un bloque alcanza la finalidad tras aproximadamente dos épocas, es decir, cerca de 12,8 minutos. El protocolo garantiza que revertir un bloque finalizado exige que al menos un tercio de los validadores firme mensajes contradictorios, conducta que es detectable y se sanciona con la pérdida de su garantía [21]. Esta propiedad, conocida como *seguridad económica*, es la que permite tratar un bloque finalizado como irreversible salvo un costo económico de gran magnitud.

El encabezado de un bloque de consenso es una estructura compacta que contiene el slot, el índice del validador proponente, la raíz del bloque predecesor (`parent_root`), la raíz del estado de consenso (`state_root`) y la raíz del cuerpo del bloque (`body_root`) [15]. Puesto que el *execution payload* forma parte del cuerpo, cualquier campo del bloque de ejecución, incluido `receiptsRoot`, puede vincularse con el encabezado de consenso mediante una prueba de Merkle SSZ respecto de `body_root`.

2.2.7. Comité de sincronización y clientes ligeros

Verificar la finalidad a partir de las atestaciones individuales exigiría conocer el conjunto completo de validadores, que supera el millón de participantes, y procesar sus firmas en cada época. Para hacer posible la existencia de clientes ligeros, la actualización Altair introdujo el **comité de sincronización** (*sync committee*) [22]. Se trata de un subconjunto $\mathcal{C}$ de 512 validadores, seleccionados de forma pseudoaleatoria y ponderada por su saldo, que permanece fijo durante un *período* de 256 épocas, aproximadamente 27 horas. En cada slot, los miembros del comité firman la raíz del encabezado de consenso del slot anterior, y las firmas se agregan en una única estructura `sync_aggregate` que se incluye en el bloque siguiente. Esta estructura contiene un vector de 512 bits que indica qué miembros participaron y la firma agregada correspondiente.

Las firmas utilizan el esquema BLS [23] sobre la curva BLS12-381, en el que las claves públicas son elementos del grupo $\mathbb{G}_1$ y las firmas son elementos del grupo $\mathbb{G}_2$. La principal propiedad de este esquema es la agregación, pues las firmas de varios participantes sobre un mismo mensaje m pueden combinarse en una sola mediante la suma de puntos de la curva, y su verificación se reduce a comprobar una única igualdad de emparejamientos. Si pk_i denota la clave pública del miembro i y *sig* la firma agregada, la verificación consiste en calcular la clave agregada y comprobar la ecuación

$$e\big(g_1, sig\big) = e\Big(pk_{\text{agg}},\, H_{\mathbb{G}_2}(m)\Big), \qquad pk_{\text{agg}} = \sum_{i \in \mathcal{C}'} pk_i, \tag{2.4}$$

donde $\mathcal{C}' \subseteq \mathcal{C}$ es el conjunto de miembros que participaron, g_1 es el generador de $\mathbb{G}_1$, $H_{\mathbb{G}_2}$ es una función de hash que transforma mensajes en puntos de $\mathbb{G}_2$ y $e\colon \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ es el emparejamiento bilineal de la curva. El mensaje m no es directamente la raíz del encabezado, sino su combinación con un *dominio de firma* que incorpora la versión del protocolo y la raíz

de los validadores del bloque génesis, lo que impide reutilizar una firma en otra red o en otro contexto.

El protocolo de cliente ligero especifica cómo un nodo que solo conoce el comité vigente puede avanzar de forma segura [24]. Una actualización (*light client update*) contiene el encabezado atestiguado, el agregado de firmas del comité, el encabezado finalizado junto con una prueba de Merkle de su pertenencia al estado de consenso y, en el último tramo de cada período, el comité siguiente acompañado de su propia prueba de pertenencia. El cliente acepta un nuevo encabezado finalizado cuando al menos dos tercios de los miembros del comité han firmado y todas las pruebas de Merkle son válidas. Desde la actualización Capella, el encabezado del cliente ligero incluye también el encabezado del bloque de ejecución y una prueba de su pertenencia a `body_root`, lo que permite obtener la raíz de recibos sin descargar el bloque completo [25].

Conviene precisar que el modelo de confianza del comité de sincronización es distinto del de la finalidad plena. El cliente ligero no comprueba que dos tercios del total de validadores hayan atestiguado el bloque, sino que confía en que la mayoría de los 512 miembros del comité actúe honestamente, ya que el protocolo no contempla sanciones para quienes firman encabezados incorrectos en este rol. Dado que los miembros se seleccionan de forma aleatoria entre todos los validadores, la probabilidad de que un adversario controle una supermayoría del comité sin controlar una fracción comparable del conjunto total es despreciable. Este supuesto es el mismo que adoptan los clientes ligeros de Ethereum y constituye la raíz de confianza del puente desarrollado en este trabajo.

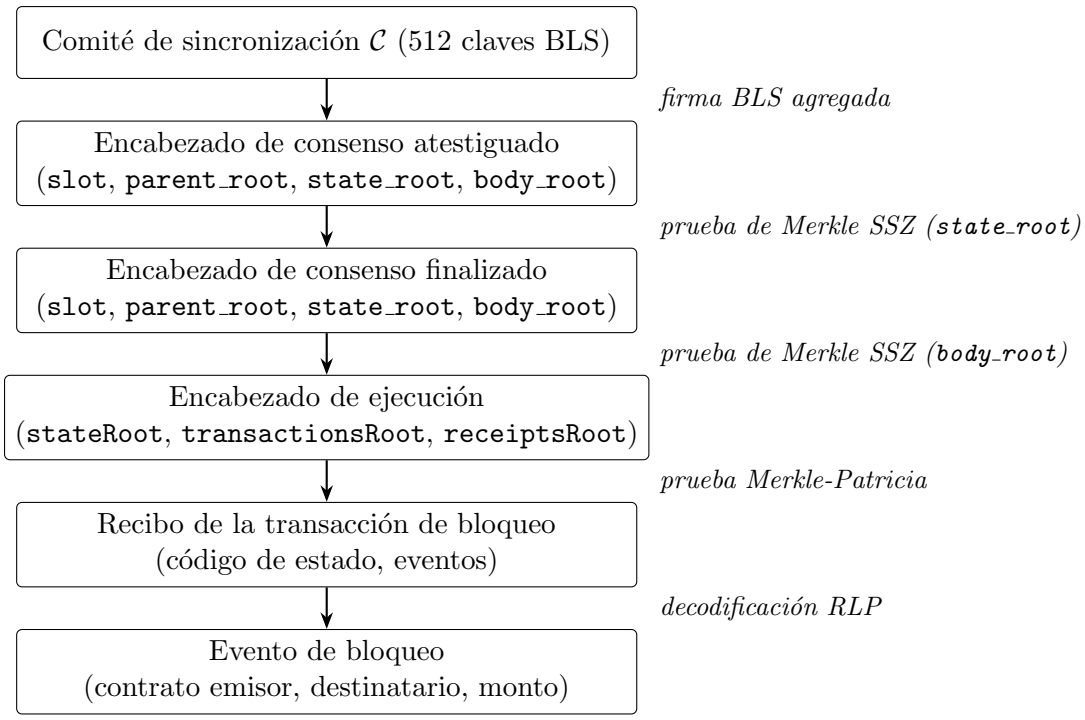


Figura 2.2: Cadena de compromisos criptográficos que vincula un evento de bloqueo con la firma del comité de sincronización de Ethereum.

2.2.8. Cadena de verificación para el puente

Los elementos descritos en esta sección permiten construir una cadena de compromisos criptográficos que vincula un evento emitido por un contrato con la firma del comité de sincronización, como se ilustra en la Figura 2.2. El punto de partida es un comité conocido, cuya firma agregada certifica la raíz del encabezado atestiguado. Una prueba de Merkle SSZ respecto del `state_root` de ese encabezado vincula con él el encabezado finalizado [24]. Desde este último, una prueba de Merkle SSZ respecto de `body_root` conduce al encabezado de ejecución y, en particular, a su campo `receiptsRoot`. Una prueba de inclusión en el árbol de Merkle-Patricia de recibos demuestra, finalmente, que el recibo de la transacción de bloqueo pertenece al bloque, que su ejecución fue exitosa y que contiene el evento emitido por el contrato del puente con los parámetros del depósito. El prototipo desarrollado en este trabajo implementa una variante de esta cadena, descrita en la Sección 4.1, que omite la prueba que vincula el encabezado finalizado y parte directamente del encabezado atestiguado por el comité, demuestra la inclusión de la transacción de bloqueo en el árbol de transacciones y su ejecución exitosa en el árbol de recibos, y obtiene el destinatario y el monto de los datos de la llamada en lugar del evento.

Cada eslabón de esta cadena es verificable de manera determinista, de modo que la validez del evento queda reducida a la validez de la firma del comité. Sin embargo, ejecutar esta verificación completa dentro de un contrato inteligente desplegado en otra cadena implicaría evaluar emparejamientos sobre BLS12-381, agregar cientos de claves públicas, calcular numerosos hashes SHA-256 y Keccak-256 y decodificar estructuras RLP, todo ello con un costo en gas que resulta prohibitivo en la práctica. El enfoque adoptado en este trabajo consiste en realizar este cómputo fuera de la cadena y comprimirlo en una prueba de conocimiento cero sucinta, de manera que el contrato de destino solo deba verificar dicha prueba. Las herramientas que lo hacen posible se presentan en el Capítulo 3.

2.3. Aztec

Aztec es una red de capa 2 (*Layer 2*) construida sobre Ethereum cuyo objetivo principal es ofrecer contratos inteligentes con estado y ejecución privados [26]. Su desarrollo está a cargo de Aztec Labs, la misma organización que creó el lenguaje Noir y el sistema de pruebas Barretenberg descritos en la Sección 3.3. La red principal de Aztec, denominada *Ignition Chain*, comenzó a producir bloques sobre Ethereum en noviembre de 2025 [27]. El estudio de Aztec resulta pertinente para este trabajo por dos razones. En primer lugar, constituye el entorno para el cual se concibieron las herramientas de Noir, de modo que su arquitectura explica muchas de las decisiones de diseño de ese lenguaje. En segundo lugar, Aztec es un sistema en el que la validez de las transiciones de estado de una cadena se certifica ante otra mediante una prueba de conocimiento cero verificada por un contrato inteligente, que es precisamente el principio sobre el que se construye el puente propuesto. La presente sección describe el modelo de *rollup* en el que se inscribe Aztec, su representación del estado, la estructura de sus bloques,

sus mecanismos de consenso y el protocolo de mensajería que lo comunica con Ethereum.

2.3.1. Rollups y pruebas de validez

Un **rollup** es una cadena que ejecuta transacciones fuera de una cadena principal, a la que se denomina capa 1, y publica en ella tanto los datos necesarios para reconstruir su estado como un compromiso sobre el estado resultante. La capa 1 actúa así como fuente de verdad y como garante de la disponibilidad de los datos, mientras que el cómputo se realiza fuera de ella. Los rollups se clasifican según el mecanismo con el que la capa 1 llega a aceptar un nuevo estado.

En los **rollups optimistas**, los nuevos estados se publican sin demostración y se presumen válidos, salvo que algún participante presente una *prueba de fraude* durante un período de impugnación, que en las implementaciones habituales se extiende por siete días. Optimism, cuya red de prueba Optimism Sepolia actúa como cadena de destino en este trabajo, pertenece a esta categoría [28]. En los **rollups de validez**, también llamados *zk-rollups*, cada nuevo estado se acompaña de una prueba sucinta de que resulta de aplicar correctamente las reglas del protocolo a un conjunto de transacciones, y un contrato verificador desplegado en la capa 1 solo acepta el nuevo estado si la prueba es válida. Esta segunda categoría elimina el período de impugnación y reemplaza la suposición de que algún participante honesto vigila la cadena por una garantía criptográfica. Aztec pertenece a la categoría de rollups de validez y añade a ella una propiedad que los demás rollups no ofrecen, que es la posibilidad de ocultar a los propios operadores de la red el contenido de las transacciones privadas [26].

2.3.2. Estado privado y estado público

A diferencia de Ethereum, cuyo estado es enteramente público, Aztec mantiene dos clases de estado con modelos de datos distintos [29]. El **estado público** funciona de manera análoga al de Ethereum, ya que cada contrato dispone de posiciones de almacenamiento cuyos valores son visibles para todos los participantes y pueden ser modificados directamente por la ejecución pública. El **estado privado**, en cambio, se representa mediante un modelo de salidas no gastadas semejante al de Bitcoin, en el que la unidad de información es la *nota*.

Definición 2.4. Una **nota** es un registro cifrado que contiene una porción del estado privado de un contrato, como el saldo de un token perteneciente a un usuario. La red no almacena la nota, sino un compromiso criptográfico sobre ella, denominado **hash de nota**, que se calcula a partir de su contenido, su propietario, la posición de almacenamiento a la que pertenece, la dirección del contrato y un valor aleatorio. Solo el propietario de la nota conoce su contenido y puede descifrarla.

Puesto que modificar un valor existente revelaría qué registro se actualizó, el estado privado nunca se modifica ni se elimina. Para consumir una nota, su propietario publica un **anulador** (*nullifier*), que es un valor derivado de la nota y de una clave secreta del propietario, calculado de forma que ningún observador pueda vincularlo con el hash de nota correspondiente. Una

transferencia privada consiste, en consecuencia, en anular las notas del emisor y crear notas nuevas para el receptor y para el cambio, de manera similar al uso de dinero en efectivo. La red rechaza toda transacción que intente publicar un anulador ya existente, lo que impide gastar dos veces la misma nota sin necesidad de identificarla [29].

Aztec adopta, además, la abstracción de cuentas de forma nativa. Toda cuenta es un contrato inteligente que define sus propias reglas de autorización, de modo que el esquema de firma, la gestión del *nonce* y la forma de pago de las tarifas quedan a criterio de quien programa la cuenta [26]. Por ejemplo, una cuenta puede verificar firmas ECDSA sobre la curva secp256k1, que es la misma que emplea Ethereum, u otro esquema distinto.

2.3.3. Árboles de estado

El estado de Aztec se organiza en cinco árboles de Merkle binarios, cuyas raíces conforman el compromiso sobre el estado global de la red. A diferencia de Ethereum, que anida un árbol de almacenamiento dentro de cada cuenta (Sección 2.2.5), Aztec almacena los valores directamente como hojas de árboles globales [29]. Los árboles emplean funciones de hash algebraicas, como Poseidon2, que se expresan como circuito con un número de restricciones muy inferior al de Keccak-256 o SHA-256. La Tabla 2.2 resume su función y su altura, según los parámetros vigentes del protocolo [30].

Tabla 2.2: Árboles de estado de Aztec.

Árbol	**Contenido**	**Altura**
Árbol de hashes de nota	Compromisos sobre las notas del estado privado. Solo admite inserciones al final.	42
Árbol de anuladores	Anuladores publicados, organizados como árbol de Merkle indexado.	42
Árbol de datos públicos	Valores del almacenamiento público de todos los contratos.	40
Árbol de mensajes de L1 a L2	Mensajes enviados desde Ethereum hacia Aztec.	36
Árbol de archivo	Hashes de los encabezados de todos los bloques anteriores.	30

El árbol de anuladores plantea un requisito particular. Para insertar un anulador, la red debe demostrar que este no existe previamente en el árbol, lo cual exige una *prueba de no pertenencia*. Un árbol de Merkle disperso, en el que cada valor posible ocupa la hoja de índice igual a dicho valor, permitiría esta prueba de forma directa, pero requeriría una profundidad de 254 niveles para cubrir los elementos del cuerpo de la curva BN254, con el consiguiente costo en restricciones. Aztec resuelve este problema con un **árbol de Merkle indexado** [31], en el que cada hoja almacena una terna $(v, i_{\text{sig}}, v_{\text{sig}})$ formada por el valor v, el índice i_{sig} de la hoja que contiene el siguiente valor mayor y dicho valor v_{sig}. Las hojas forman, así, una lista enlazada

ordenada. Para demostrar que un valor x no pertenece al árbol basta con exhibir la pertenencia de la hoja que lo precede, denominada *anulador inferior*, y comprobar que $v < x < v_{\text{sig}}$, o bien que $v < x$ y $v_{\text{sig}} = 0$ cuando x es mayor que todos los valores almacenados. Como la posición de una hoja ya no depende de su valor, una profundidad moderada es suficiente.

El **árbol de archivo** cumple una función semejante a la del encadenamiento por hash descrito en la Sección 2.1. Al almacenar el hash de cada encabezado, permite demostrar dentro de un circuito, con una prueba de inclusión logarítmica, que un bloque anterior forma parte de la historia de la cadena.

2.3.4. Ejecución de transacciones y estructura de bloques

La ejecución de una transacción en Aztec se divide en una fase privada y una fase pública [32]. La fase privada se ejecuta en el dispositivo del usuario, dentro de un componente denominado PXE (*Private eXecution Environment*), que almacena las claves y las notas del usuario, ejecuta las funciones privadas de los contratos y genera las pruebas correspondientes. Cada llamada a una función privada es procesada por el **circuito de núcleo privado** (*private kernel*), que verifica la prueba de la función, comprueba el contexto de la llamada y acumula sus efectos, es decir, los hashes de nota, los anuladores, los mensajes hacia Ethereum y las llamadas públicas pendientes [33]. Este circuito se aplica de forma recursiva, una vez por llamada, y concluye con una prueba única que certifica toda la ejecución privada de la transacción. Es el único circuito del protocolo que requiere efectivamente la propiedad de conocimiento cero, puesto que debe ocultar qué contrato y qué función se ejecutaron, quién es el usuario y cuáles fueron las entradas y salidas. Los restantes circuitos utilizan pruebas sucintas para comprimir la verificación, pero no necesitan ocultar sus testigos.

El usuario envía a la red la prueba de la fase privada junto con sus efectos. La fase pública, que comprende las funciones públicas encoladas por la fase privada, se ejecuta posteriormente en los nodos de la red mediante la AVM (*Aztec Virtual Machine*), una máquina virtual conceptualmente análoga a la EVM, cuya ejecución también se demuestra mediante una prueba. El costo de las transacciones se mide en *mana*, unidad equivalente al gas de Ethereum (Sección 2.2.2), que se desglosa en un componente de ejecución y otro de disponibilidad de datos, y cuyo precio se determina mediante un mecanismo inspirado en EIP-1559 [34].

El encabezado de un bloque de Aztec contiene la raíz del árbol de archivo antes del bloque, las raíces de los árboles de hashes de nota, de anuladores, de datos públicos y de mensajes de L1 a L2 tras su ejecución, un conjunto de variables globales que incluye el identificador de la cadena, el número de bloque, el número de *slot*, la marca de tiempo y los precios del mana, y los totales de tarifas y de mana consumidos [30]. Los bloques se agrupan en *checkpoints*, que son las unidades que se publican en Ethereum, y estos a su vez en *épocas* de hasta 32 checkpoints.

La validez de una época completa se demuestra mediante una única prueba, construida por los **circuitos de rollup** a partir de un árbol binario de pruebas [35]. En el nivel inferior, un circuito base verifica la prueba de cada transacción y actualiza los árboles de estado. Los niveles

superiores fusionan recursivamente pares de pruebas, primero en pruebas de bloque, luego de checkpoint y finalmente en la prueba raíz de la época, cuyas entradas públicas incluyen las raíces del árbol de archivo anterior y posterior. Esta topología permite distribuir la generación de las pruebas entre numerosos probadores que trabajan en paralelo. La prueba raíz se verifica en el contrato de rollup desplegado en Ethereum, de modo que el costo de verificación en la capa 1 es independiente del número de transacciones incluidas en la época. La Figura 2.3 resume el recorrido de una transacción desde el dispositivo del usuario hasta su asentamiento en Ethereum.

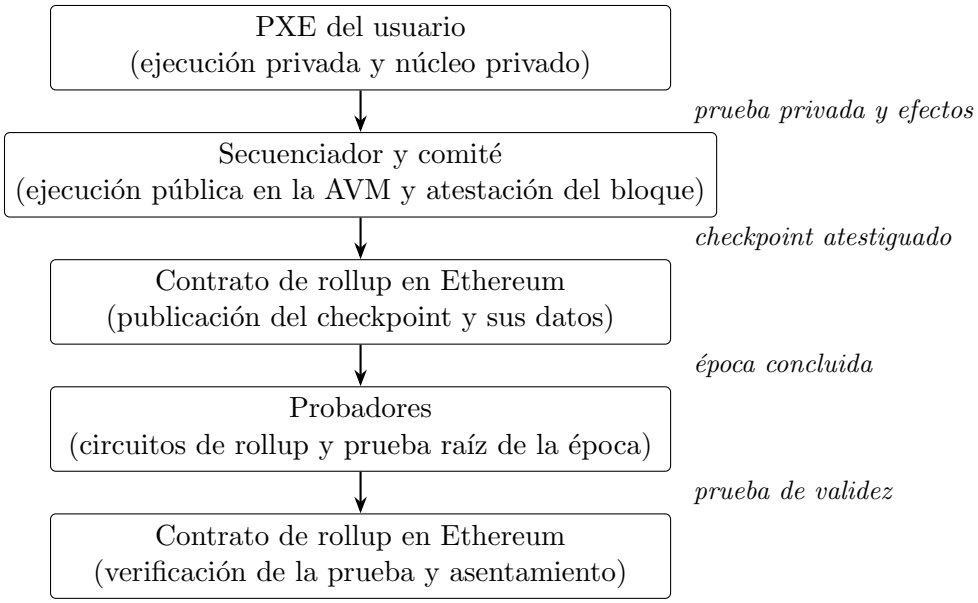


Figura 2.3: Recorrido de una transacción de Aztec desde su ejecución en el dispositivo del usuario hasta su asentamiento en Ethereum.

2.3.5. Secuenciadores, probadores y finalidad

La producción de bloques en Aztec está a cargo de **secuenciadores**, que son participantes que depositan una garantía de 200 000 tokens AZTEC para integrar el conjunto de validadores de la red [36]. En cada slot, se selecciona de forma pseudoaleatoria un proponente a partir de la aleatoriedad provista por Ethereum, y un comité de secuenciadores reejecuta las transacciones propuestas y firma atestaciones si las considera válidas [37]. Un bloque solo se acepta si reúne las atestaciones de al menos dos tercios más uno de los miembros del comité, y la publicación de un bloque sin ese respaldo constituye una falta sancionable con la pérdida de parte de la garantía.

Las atestaciones proporcionan una confirmación provisional, pero no constituyen la garantía definitiva. Una vez concluida una época, los **probadores** generan la prueba de validez de todos sus bloques y la remiten al contrato de rollup, que la verifica y actualiza el estado asentado. Si ninguna prueba válida llega dentro del plazo previsto, los bloques no demostrados se descartan y sus transacciones deben volver a proponerse [37]. La seguridad de las transiciones de estado

asentadas en Ethereum descansa, por lo tanto, en la validez de la prueba y no en la honestidad del comité, cuyo papel se limita a ordenar las transacciones y a garantizar la disponibilidad de los datos.

2.3.6. Mensajería entre Ethereum y Aztec

Las aplicaciones de Aztec se comunican con Ethereum mediante un protocolo de mensajes asíncronos. El extremo de una aplicación en Ethereum se denomina **portal**, y es un contrato inteligente vinculado a una dirección específica de Aztec [38]. Los mensajes desde Ethereum hacia Aztec se depositan en un contrato denominado *Inbox*, que almacena únicamente un compromiso de cada mensaje. Al procesar el checkpoint siguiente, los circuitos de rollup insertan esos mensajes en el árbol de mensajes de L1 a L2, y un contrato de Aztec puede consumirlos demostrando su pertenencia a dicho árbol y publicando un anulador que impide consumirlos dos veces. En sentido inverso, los mensajes que emite una transacción de Aztec quedan disponibles en un contrato denominado *Outbox* una vez que la prueba de la época que los contiene ha sido verificada en Ethereum, momento a partir del cual el portal correspondiente puede consumirlos.

Este protocolo permite construir puentes de activos del tipo *lock-and-mint* sin intermediarios de confianza. El propio protocolo incluye uno de ellos, el contrato `FeeJuicePortal`, que bloquea en Ethereum los tokens destinados al pago de tarifas y envía un mensaje al contrato correspondiente de Aztec, que acredita el saldo al destinatario cuando este presenta el secreto asociado al depósito [34]. La acuñación en Aztec no depende de un *relayer*, sino de que el mensaje figure en un árbol cuya raíz ha sido establecida por los circuitos de rollup y verificada en Ethereum, y la liberación de fondos en Ethereum depende, a su vez, de una prueba de validez aceptada por el contrato de rollup.

Esta garantía es posible porque Aztec y Ethereum comparten un contrato de asentamiento, ya que es Ethereum quien verifica las pruebas de Aztec y, por ende, conoce con certeza su estado. Dos cadenas independientes no disponen de un mecanismo equivalente, puesto que ninguna verifica de forma nativa el consenso de la otra. El puente desarrollado en este trabajo traslada el principio de los rollups de validez a este escenario, reemplazando la verificación de la prueba de un rollup por la verificación de pruebas de conocimiento cero sobre la firma del comité de sincronización de Ethereum y sobre la inclusión del evento de bloqueo, según la cadena de compromisos descrita en la Sección 2.2.8. Aunque la cadena de destino empleada en la validación, Optimism Sepolia, sí mantiene una relación nativa con Ethereum, el diseño no hace uso de ella, según se expone en la Sección 2.4.5.

2.4. Optimism

Optimism es una red de capa 2 de Ethereum del tipo *rollup* optimista, construida sobre el OP Stack, un conjunto de componentes de código abierto que permite desplegar cadenas con la misma arquitectura [39]. Su red de prueba, Optimism Sepolia, se apoya sobre la red de prueba

Ethereum Sepolia y actúa en este trabajo como cadena de destino del puente, es decir, como la cadena en la que se despliegan el contrato verificador de las pruebas de conocimiento cero y el contrato que acuña los activos. La presente sección describe la arquitectura de Optimism, el ciclo de confirmación de sus bloques, su modelo de tarifas, el mecanismo de pruebas de fraude que sostiene su seguridad y el puente nativo que lo comunica con Ethereum, con el fin de precisar las propiedades de la cadena de destino que condicionan el diseño del puente.

2.4.1. Arquitectura del OP Stack

De acuerdo con la clasificación presentada en la Sección 2.3.1, Optimism ejecuta las transacciones fuera de Ethereum, publica en Ethereum los datos necesarios para reconstruir su estado y presume la validez de los estados resultantes mientras nadie demuestre lo contrario. Su especificación la describe como una cadena *derivada*, puesto que su historia canónica queda definida exclusivamente por los datos publicados en la capa 1 [39]. Cualquier nodo que lea dichos datos desde Ethereum y aplique las reglas de derivación obtiene la misma cadena que construyó el operador de la red.

Al igual que un nodo de Ethereum posterior a *The Merge* (Sección 2.2.3), un nodo de Optimism se compone de un cliente de consenso, denominado nodo de *rollup*, y de un cliente de ejecución derivado de un cliente de Ethereum, ambos comunicados mediante la misma interfaz que utilizan los clientes de la capa 1. Sobre esta base, el OP Stack distingue cuatro funciones principales [39]:

- El **secuenciador** recibe las transacciones de los usuarios, las ordena y produce los bloques de la capa 2. En Optimism Sepolia se produce un bloque cada dos segundos [40].
- El **publicador de lotes** (*batcher*) comprime los bloques producidos y publica sus datos en Ethereum, habitualmente en forma de *blobs*, un tipo de dato introducido por la propuesta EIP-4844 que ofrece un almacenamiento temporal más económico que el *calldata* [41].
- El **proponente** (*proposer*) publica periódicamente en Ethereum *raíces de salida* (*output roots*), que son compromisos sobre el estado completo de la capa 2 en un bloque determinado.
- El **impugnador** (*challenger*) vigila las raíces de salida publicadas y las disputa cuando no coinciden con el estado que resulta de derivar la cadena.

En la capa 1, estas funciones se apoyan en un conjunto de contratos inteligentes. Entre ellos destaca el contrato `OptimismPortal`, que recibe los depósitos y procesa los retiros. También destaca el contrato `DisputeGameFactory`, que registra las raíces de salida y crea los juegos de disputa asociados a ellas [42]. La Figura 2.4 representa la relación entre los componentes de ambas capas.

El secuenciador no puede excluir de forma indefinida los depósitos realizados en la capa 1. Las transacciones enviadas a `OptimismPortal` deben incorporarse a la capa 2 dentro de una

ventana de secuenciación, que en Optimism Sepolia es de 3 600 bloques de Ethereum [40], y en caso contrario las reglas de derivación las incluyen de todas formas [42]. Esta propiedad otorga a los usuarios resistencia a la censura aun cuando el secuenciador sea un operador único.

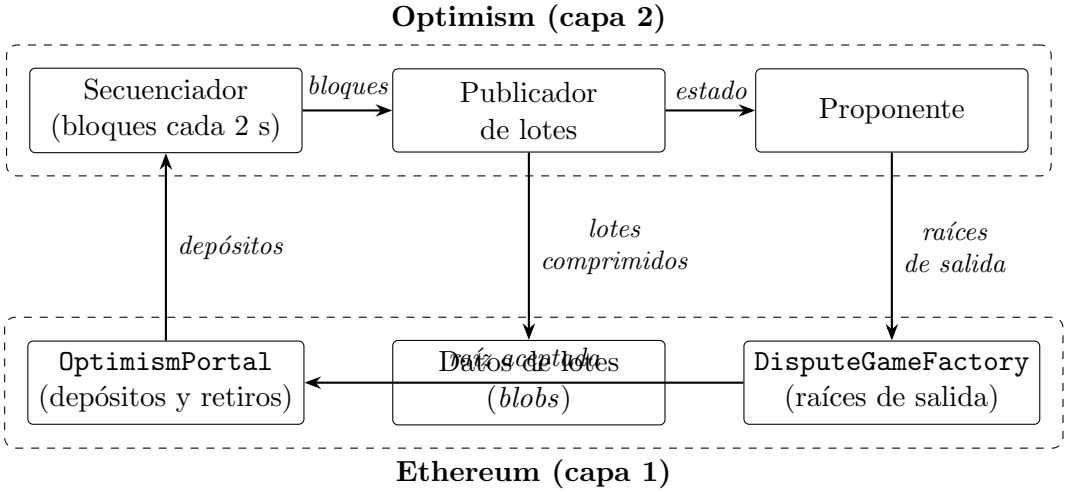


Figura 2.4: Componentes del OP Stack y su relación con los contratos desplegados en Ethereum.

2.4.2. Confirmación de bloques y finalidad

Dado que los bloques de Optimism se producen antes de que sus datos lleguen a Ethereum, un mismo bloque atraviesa distintos estados de confirmación a lo largo del tiempo. La Tabla 2.3 resume los tres estados que distingue el OP Stack [39].

Tabla 2.3: Estados de confirmación de un bloque de Optimism.

Estado	**Condición**	**Garantía**
No seguro (*unsafe*)	El secuenciador produjo el bloque y lo difundió por la red *peer-to-peer*.	Depende de la honestidad del secuenciador.
Seguro (*safe*)	Los datos del bloque fueron publicados en Ethereum y el bloque puede derivarse a partir de ellos.	Depende de que el bloque de Ethereum que contiene los datos no sea revertido.
Finalizado (*finalized*)	El bloque de Ethereum que contiene los datos alcanzó la finalidad.	Equivalente a la finalidad de Ethereum (Sección 2.2.6).

Conviene distinguir este concepto de finalidad del que se aplica a los retiros hacia Ethereum. Un bloque finalizado de Optimism no puede ser revertido, puesto que sus datos forman parte de la historia finalizada de Ethereum y su contenido queda determinado por las reglas de derivación. Sin embargo, Ethereum no ejecuta esas reglas por sí mismo, de modo que solo acepta una afirmación sobre el estado de Optimism, como la existencia de un retiro, una vez que la raíz de salida correspondiente ha superado el período de impugnación descrito en la Sección 2.4.4.

2.4.3. Compatibilidad con la EVM y modelo de tarifas

Optimism busca la **equivalencia con la EVM**, lo que significa que su cliente de ejecución reproduce el comportamiento de la máquina virtual de Ethereum con un número reducido de modificaciones, documentadas en su especificación [43]. Los contratos escritos para Ethereum pueden desplegarse en Optimism sin cambios, y los contratos precompilados descritos en la Sección 2.2.2, incluidos los de la curva BN254, se encuentran disponibles con los mismos costos de gas. La actualización Isthmus incorporó, además, los precompilados de BLS12-381 definidos en la propuesta EIP-2537 [44, 13]. Esta propiedad permite que el contrato verificador generado por las herramientas de Noir y de SP1 para Ethereum se despliegue en Optimism Sepolia sin modificaciones.

El modelo de tarifas, en cambio, presenta una diferencia relevante. Las transacciones de Optimism pagan la tarifa base y la propina del mecanismo EIP-1559 por su ejecución en la capa 2, y pagan además una **tarifa de datos de capa 1** (*L1 data fee*), que cubre el costo de publicar sus datos en Ethereum [43]. Esta segunda tarifa depende del tamaño de la transacción una vez comprimida y de los precios vigentes de *blobs* y de gas en Ethereum, que la capa 2 conoce a través del contrato predesplegado `L1Block`. En consecuencia, el costo total de verificar una prueba en Optimism no depende solo del gas de ejecución, sino también del tamaño de la prueba y de las entradas públicas que se transmiten en la transacción. Por esta razón, en este trabajo el costo de verificación en la cadena de destino se reporta en dos magnitudes separadas. La primera es el gas de ejecución consumido por las operaciones de verificación, que depende únicamente de las instrucciones ejecutadas por los contratos y no del precio del gas. La segunda es el costo monetario de cada transacción, que resulta del producto entre el gas consumido y el precio efectivo del gas, al que se suma la tarifa de datos de capa 1. Esta última incide de manera significativa en el puente, puesto que las pruebas se transmiten como datos de la transacción y su tamaño se refleja en lo que Optimism cobra por publicarlas en Ethereum. El costo monetario se expresa en ether y en dólares, con indicación de la fecha del precio empleado, y se complementa con el costo de generar las pruebas fuera de la cadena, según se detalla en el Capítulo 4.

2.4.4. Pruebas de fraude

La seguridad de las raíces de salida de Optimism descansa en un sistema de pruebas de fraude habilitado sin permisos en la red principal desde junio de 2024 [28]. Cuando un participante publica una raíz de salida, cualquier otro puede impugnarla depositando una garantía, lo que da inicio a un **juego de disputa**. En él, ambas partes acotan de forma iterativa, mediante bisección, el primer paso de la ejecución sobre el cual discrepan, hasta reducir el desacuerdo a una única instrucción. Esa instrucción se ejecuta entonces en un contrato de Ethereum que implementa la máquina virtual Cannon, la cual reproduce el programa de derivación y ejecución de Optimism compilado para una arquitectura MIPS [45]. El resultado de esta ejecución

determina qué parte tenía razón, y la parte equivocada pierde su garantía.

Si ninguna impugnación prospera durante el período previsto, de aproximadamente siete días, la raíz de salida se considera válida y puede utilizarse para procesar retiros [28]. El modelo de seguridad resultante se apoya en el supuesto de que existe al menos un participante honesto que vigila la cadena y que puede publicar una impugnación en Ethereum dentro de ese plazo. Este supuesto es más débil que el de confiar en un operador concreto, pero difiere de la garantía que ofrece una prueba de validez, en la que la corrección del estado se verifica de forma inmediata y no depende de la vigilancia de terceros.

2.4.5. Puente nativo con Ethereum

Optimism dispone de un puente nativo entre ambas capas, compuesto por los contratos `L1CrossDomainMessenger` y `L1StandardBridge` en Ethereum y sus equivalentes predesplegados en la capa 2 [42]. En un **depósito**, el contrato de la capa 1 bloquea los activos y emite un mensaje a través de `OptimismPortal`, que la capa 2 incorpora como una transacción especial capaz de acuñar los activos equivalentes. Como el contenido de la capa 2 se deriva de los datos de Ethereum, el depósito queda acreditado en pocos minutos y sin intervención de terceros.

Un **retiro** sigue el camino inverso y requiere dos pasos en Ethereum [46]. Primero, el usuario *prueba* el retiro, demostrando mediante una prueba de Merkle que el mensaje fue registrado en el contrato `L2ToL1MessagePasser` y que este registro forma parte de una raíz de salida publicada. Después, una vez transcurrido el período de impugnación y resuelto a favor el juego de disputa correspondiente, el usuario *finaliza* el retiro y el contrato libera los activos bloqueados. La asimetría entre ambos sentidos refleja la naturaleza del *rollup* optimista, puesto que la capa 2 conoce directamente el estado de Ethereum, mientras que Ethereum solo conoce el estado de Optimism a través de afirmaciones que deben superar el período de impugnación.

La capa 2 conoce el estado de Ethereum porque cada bloque de Optimism comienza con una transacción del sistema que registra en el contrato `L1Block` el número, la marca de tiempo y el hash del bloque de Ethereum que le sirve de origen [47]. El puente desarrollado en este trabajo no utiliza este mecanismo de forma deliberada, puesto que el contrato `L1Block` solo existe en las cadenas construidas con el OP Stack y el objetivo es que el puente pueda desplegarse en cadenas que no mantienen una relación nativa con Ethereum. La finalidad de Ethereum se ancla, en cambio, en un contrato de cliente ligero que solo acepta un bloque cuando una prueba de conocimiento cero demuestra tres afirmaciones. La primera es que al menos dos tercios del comité de sincronización firmaron el encabezado (Sección 2.2.7). La segunda es que las claves públicas empleadas corresponden al compromiso del comité registrado en el contrato. La tercera es que un conjunto de ramas de Merkle vincula dicho encabezado con el bloque de ejecución, conforme a la cadena de compromisos de la Sección 2.2.8. Los detalles de este contrato se exponen en la Sección 4.1.

Este mecanismo solo requiere que la cadena de destino verifique pruebas sucintas cuyos verificadores emplean los contratos precompilados de la curva BN254, disponibles en prácticamente

cualquier cadena compatible con la EVM. La única salvedad identificada se refiere a las instrucciones que emite el compilador de Solidity. Las versiones recientes del compilador generan por defecto la instrucción `PUSH0`, introducida por la propuesta EIP-3855 [48], de modo que en una cadena que no la admita los contratos deben compilarse para una versión anterior de la EVM, como la correspondiente a la actualización Paris. En este contexto, Optimism Sepolia cumple el papel de entorno de validación representativo de una cadena de destino compatible con la EVM y con costos de ejecución reducidos.

2.5. Puentes

CAPÍTULO 3

CONOCIMIENTO CERO

3.1. Fundamentos

El capítulo anterior describió los mecanismos mediante los cuales una red blockchain garantiza la integridad de su registro, desde el encadenamiento de bloques por hash hasta las pruebas de inclusión sobre árboles de Merkle y el modelo de confianza de los clientes ligeros (Sección 2.1). En todos esos casos, quien verifica una afirmación sobre la cadena necesita disponer de los datos pertinentes y ejecutar por sí mismo el cómputo correspondiente. Los sistemas de prueba estudiados en este capítulo, conocidos genéricamente como pruebas de conocimiento cero, ofrecen una alternativa a este modelo, puesto que permiten convencer a un verificador de que una afirmación es verdadera, o de que una computación se ejecutó correctamente, sin que este deba repetir dicha computación. Esta última capacidad no se deriva de la propiedad de conocimiento cero, sino de la sucintez de los sistemas empleados, distinción que se precisa en las Secciones 3.1.1 y 3.1.5.

La presente sección no pretende ofrecer un tratamiento exhaustivo de la teoría de las pruebas de conocimiento cero, sino establecer el vocabulario mínimo necesario para comprender las herramientas analizadas en el resto del capítulo. Para un desarrollo completo de los aspectos teóricos se remite al lector a la obra de Thaler [49], cuya terminología se adopta en lo que sigue.

3.1.1. Concepto de conocimiento cero

En su acepción habitual, demostrar una afirmación consiste en presentar evidencia que permita a otra parte comprobar su veracidad. Si alguien afirma conocer un valor w cuyo hash criptográfico coincide con un valor público y, es decir, un valor tal que $H(w) = y$, la forma más directa de demostrarlo es entregar w para que el receptor calcule $H(w)$ y lo compare con y. Esta demostración resulta convincente, pero el receptor termina conociendo el valor que

se pretendía mantener en secreto. Goldwasser, Micali y Rackoff demostraron que es posible convencer a otra parte de una afirmación de este tipo sin transferirle ningún conocimiento adicional a la validez de la afirmación misma, y denominaron a este tipo de demostraciones *pruebas de conocimiento cero* [50]. Posteriormente, Goldreich, Micali y Wigderson demostraron que, bajo supuestos criptográficos estándar, toda afirmación cuya veracidad puede comprobarse eficientemente a partir de un certificado admite una prueba de este tipo [51].

El ejemplo anterior permite introducir los cinco términos que se emplean en el resto del capítulo.

- El **probador** (*prover*), denotado $\mathcal{P}$, es la parte que afirma la validez de un enunciado y genera la evidencia que la respalda.
- El **verificador** (*verifier*), denotado $\mathcal{V}$, es la parte que examina dicha evidencia y decide si la acepta o la rechaza. Se asume que el verificador dispone de recursos de cómputo limitados, por lo que su trabajo debe ser eficiente.
- El **enunciado** (*statement*), denotado x, es la afirmación que se desea demostrar, expresada a partir de datos públicos conocidos por ambas partes. En el ejemplo, el enunciado es que existe un valor cuyo hash es y.
- El **testigo** (*witness*), denotado w, es la información que el probador utiliza para demostrar el enunciado y que no necesita entregar al verificador. En el ejemplo, el testigo es la preimagen w.
- La **prueba** (*proof*), denotada π, es el objeto que el probador genera a partir del enunciado y del testigo, y que el verificador examina para tomar su decisión.

Una **relación** $\mathcal{R}$ es un conjunto de pares (x, w) cuya pertenencia puede comprobarse eficientemente, y un enunciado x es verdadero cuando existe algún testigo w tal que $(x, w) \in \mathcal{R}$ [52]. En el ejemplo, la relación está formada por los pares (y, w) que satisfacen $H(w) = y$. Las herramientas que se analizan en las secciones siguientes reflejan esta distinción mediante las nociones de *entradas públicas*, que forman parte del enunciado, y *entradas privadas*, que forman parte del testigo.

Conviene aclarar que el conocimiento cero y la privacidad no son necesariamente lo mismo en las aplicaciones blockchain. Algunas aplicaciones emplean estas pruebas para ocultar información, como Zerocash, que valida pagos sin revelar su origen, su destino ni su monto [53]. Sin embargo, los sistemas de prueba modernos ofrecen además una propiedad independiente del conocimiento cero, que consiste en la posibilidad de verificar una computación en un tiempo muy inferior al que requiere ejecutarla. Walfish y Blumberg describen este objetivo como la verificación de computaciones sin reejecutarlas, de manera que una parte con recursos limitados puede delegar un cómputo en otra parte no confiable y comprobar después, a bajo costo, que el resultado recibido es correcto [54]. En muchas aplicaciones blockchain, los datos involucrados

son públicos y el valor de la prueba reside precisamente en esta segunda propiedad. La literatura y la industria emplean de todos modos la expresión *prueba de conocimiento cero* para referirse a estos sistemas, convención que se adopta también en el presente trabajo.

Un ejemplo relacionado con el Capítulo 2 permite apreciar esta diferencia. Una prueba de inclusión en un árbol de Merkle (Sección 2.1.2) puede formularse como un enunciado, cuyo contenido es que un elemento t pertenece al árbol de raíz r, y un testigo, formado por los hashes hermanos del camino que une a t con r. Ninguno de estos datos es secreto, ya que todos pueden obtenerse de la cadena. El interés de expresar esta verificación como un enunciado reside en la posibilidad de combinarla con otras condiciones de validez, como la comprobación de que la raíz r pertenece a un encabezado certificado por los validadores de la cadena, y de demostrar el conjunto mediante una única prueba cuya verificación resulte económica.

3.1.2. Propiedades fundamentales

Un sistema de prueba de conocimiento cero se caracteriza por tres propiedades, cada una de las cuales protege a una de las partes frente a un comportamiento incorrecto de la otra [50, 49].

La **completitud** (*completeness*) garantiza que, si el enunciado es verdadero y el probador sigue el protocolo utilizando un testigo válido, el verificador acepta la prueba. Esta propiedad protege al probador honesto, puesto que asegura que una demostración correcta no será rechazada.

La **solidez** (*soundness*) garantiza que, si el enunciado es falso, ningún probador logra que el verificador acepte, salvo con una probabilidad *despreciable*, es decir, tan pequeña que en la práctica puede considerarse nula. Esta propiedad protege al verificador frente a un probador deshonesto, denotado $\mathcal{P}^*$, que intenta hacer pasar por verdadera una afirmación falsa. Cuando esta garantía depende de la dificultad de algún problema computacional, como ocurre en todos los sistemas analizados en este capítulo, el sistema se denomina técnicamente *argumento* en lugar de *prueba* [55].

En ciertos casos la solidez resulta insuficiente. En el ejemplo de la preimagen, prácticamente todo valor y posee alguna preimagen, de manera que demostrar su existencia carece de interés. Lo que se pretende demostrar es que el probador *conoce* una. Esta exigencia se denomina **solidez de conocimiento** (*knowledge soundness*) y establece que de todo probador capaz de convencer al verificador podría obtenerse un testigo válido, lo que implica que dicho probador efectivamente lo conocía [56]. Los sistemas que cumplen esta propiedad se denominan *argumentos de conocimiento*.

El **conocimiento cero** (*zero-knowledge*) garantiza que el verificador no obtiene de la prueba ninguna información sobre el testigo más allá de la validez del enunciado. Esta propiedad protege al probador, y su formalización se basa en mostrar que todo lo que el verificador observa podría haberse generado a partir del enunciado sin conocer el testigo [50].

La importancia relativa de cada propiedad depende de la aplicación. En un puente entre

cadenas, el verificador es un contrato inteligente que acuña activos cuando acepta una prueba, y el adversario principal es un probador que intenta obtener esa acuñación a partir de un evento que nunca ocurrió en la cadena de origen. En este escenario la propiedad crítica es la solidez, puesto que su violación permitiría acuñar activos sin respaldo. El conocimiento cero, en cambio, resulta secundario cuando los datos que se prueban son públicos.

3.1.3. Pruebas interactivas y no interactivas

En el modelo original de Goldwasser, Micali y Rackoff, probador y verificador intercambian varios mensajes antes de que el verificador tome su decisión [50]. En estos protocolos, denominados **interactivos**, el verificador envía *desafíos* elegidos al azar, y la solidez se apoya en que el probador no puede anticiparlos. En una red blockchain, un contrato inteligente podría ejecutar un protocolo de varias rondas mediante una secuencia de transacciones, pero cada ronda añadiría costo en gas y latencia, y el contrato necesitaría obtener desafíos aleatorios que el probador no pudiera manipular.

Una prueba **no interactiva** consiste, en cambio, en un único mensaje π que cualquier parte puede verificar en cualquier momento, sin nuevos intercambios con el probador. Blum, Feldman y Micali demostraron que este tipo de pruebas es posible cuando ambas partes comparten parámetros públicos generados de antemano mediante una **configuración inicial** (*setup*) [57]. Si ese procedimiento emplea valores secretos que deben destruirse, porque su conocimiento permitiría generar pruebas falsas, se habla de una **configuración confiable** (*trusted setup*). Si solo emplea aleatoriedad pública, se denomina **transparente**. La configuración confiable no es, por lo tanto, un requisito de toda prueba no interactiva, y esta distinción es una de las diferencias principales entre las familias de sistemas presentadas en la Sección 3.1.5.

La técnica más difundida para eliminar la interacción es la **transformación de Fiat-Shamir** [58], aplicable a protocolos de *desafíos públicos*, en los que el verificador se limita a enviar valores aleatorios. La transformación sustituye cada desafío por el hash de la transcripción previa, cuyo cálculo debe vincular el enunciado público y los mensajes previos del probador, ya que omitir el enunciado ha dado lugar a ataques documentados [59]. Aunque suele analizarse en el modelo de oráculo aleatorio, que idealiza la función de hash [60], la transformación no es segura de forma automática, puesto que su seguridad depende del protocolo de partida, de sus parámetros y de la construcción de la transcripción [49, Sección 5.3]. Aplicada correctamente, permite generar la prueba sin intervención del verificador, de modo que esta puede incluirse en una transacción y ser verificada por un contrato inteligente.

3.1.4. De una computación a una prueba

Esta subsección describe en primer lugar el enfoque basado en circuitos aritméticos y restricciones, en el que el desarrollador describe la computación mediante un lenguaje, como Circom o Noir, y un compilador la transforma en la representación matemática sobre la que opera el sistema de prueba. La construcción de esa representación se denomina **aritmetización** [49]. Todos

sus valores son elementos de un **cuerpo finito** $\mathbb{F}_p$, formado por los enteros $\{0, 1, \ldots, p-1\}$ con la suma y el producto calculados módulo un número primo p, de manera que, a diferencia de lo que ocurre en un procesador convencional, todo resultado se reduce módulo p.

Circuitos y restricciones. Un **circuito aritmético** representa la computación como una red de compuertas de suma y de producto sobre $\mathbb{F}_p$. Un sistema de prueba, sin embargo, no necesita repetir la computación, sino comprobar que los valores propuestos por el probador son coherentes con ella. Por ese motivo, el circuito se expresa como un **sistema de restricciones**, es decir, un conjunto de ecuaciones sobre $\mathbb{F}_p$ que deben cumplirse simultáneamente y cuyas incógnitas incluyen las entradas, las salidas y todos los valores intermedios. Para demostrar que se conoce un valor w tal que $w^3+w+5 = 35$, por ejemplo, la computación puede descomponerse en tres restricciones con dos variables intermedias v_1 y v_2.

$$w \cdot w = v_1, \qquad v_1 \cdot w = v_2, \qquad v_2 + w + 5 = 35. \tag{3.1}$$

El testigo que satisface este sistema no es solamente el valor $w = 3$, sino la asignación completa $(w, v_1, v_2) = (3, 9, 27)$. El formato de restricciones más difundido es el sistema de restricciones de rango uno (*Rank-1 Constraint System*, R1CS) [61], que se retoma en la Sección 3.2.

Esta representación tiene dos consecuencias prácticas. La primera es que una restricción omitida no produce un error de ejecución, sino que amplía el conjunto de testigos aceptados y permite a un probador deshonesto satisfacer el sistema con valores ajenos a la computación prevista. Los circuitos insuficientemente restringidos constituyen, de hecho, una categoría recurrente entre las 141 vulnerabilidades reales analizadas por Chaliasos et al. [62]. La segunda es que las operaciones que no son nativas del cuerpo finito resultan costosas. Las funciones de hash SHA-256 y Keccak-256, empleadas en Ethereum, operan sobre bits, y representar cada bit como un elemento de $\mathbb{F}_p$ restringido a los valores 0 y 1 multiplica el número de restricciones. Los sistemas actuales mitigan este costo mediante tablas de búsqueda (*lookup tables*), que se retoman en la Sección 3.3.

Generación y verificación de la prueba. Una vez definida la representación, el proceso completo se desarrolla en cuatro etapas, que la Figura 3.1 representa de manera esquemática.

1. **Configuración inicial.** A partir del sistema de restricciones se generan los parámetros públicos que emplean el probador y el verificador. Esta etapa se realiza una única vez y, en los sistemas transparentes, no requiere valores secretos.

2. **Generación del testigo.** El probador ejecuta la computación sobre las entradas concretas y registra todos los valores que intervienen en ella.

3. **Generación de la prueba.** El probador transforma el testigo en una prueba π mediante operaciones criptográficas.

4. **Verificación.** El verificador comprueba la prueba π a partir del enunciado x y de los parámetros públicos, sin acceso al testigo ni a la computación original, y la acepta o la rechaza.

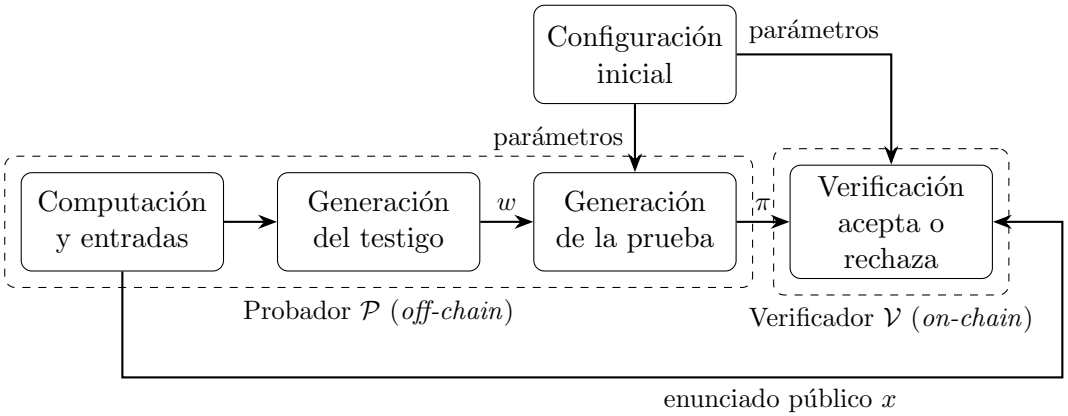


Figura 3.1: Etapas de generación y verificación de una prueba.

El reparto del trabajo entre ambas partes es marcadamente asimétrico. El costo de generar la prueba crece al menos de manera lineal con el tamaño de la computación y suele superar ampliamente el de ejecutarla, mientras que el de verificarla es, en los sistemas de la Sección 3.1.5, constante o muy pequeño en comparación. Por ese motivo, en las aplicaciones blockchain la prueba se genera fuera de la cadena (*off-chain*) y se verifica dentro de ella (*on-chain*), en un contrato inteligente. La garantía obtenida se limita, además, a que el probador conoce un testigo que satisface las restricciones para el enunciado dado. Su valor depende de que las restricciones expresen fielmente la afirmación que se pretende establecer, y todo dato que el verificador utilice para decidir, como el destinatario o el monto de una transferencia, debe ser una entrada pública o estar vinculado criptográficamente a una.

Del circuito a la ejecución de programas. En el enfoque descrito, cada computación da lugar a un sistema de restricciones específico, cuyo tamaño queda fijado de antemano. Los ciclos deben desplegarse hasta una cantidad máxima de iteraciones, las bifurcaciones que dependen de los datos obligan a representar ambas ramas y toda modificación de la lógica produce un circuito distinto. Circom y Noir, analizados en las Secciones 3.2 y 3.3, pertenecen a este enfoque.

Un enfoque alternativo consiste en demostrar que un programa arbitrario, ejecutado sobre un procesador determinado, produjo cierto resultado. En ese caso, las restricciones no describen el programa sino el procesador, ya que expresan que cada paso de la ejecución es una transición válida, y el testigo pasa a ser la **traza de ejecución**, es decir, el registro del estado del procesador en cada paso. Un único sistema de restricciones permite así probar la ejecución de cualquier programa, idea que Ben-Sasson et al. materializaron para programas escritos en C [63]. Una **máquina virtual de conocimiento cero** (*zero-knowledge virtual machine*, zkVM) aplica este principio a un conjunto de instrucciones concreto. SP1, analizado en la Sección 3.4, ejecuta de este modo programas escritos en Rust y compilados para RISC-V [4]. Ambos enfoques no

difieren en lo que pueden demostrar, sino en sus compromisos. Los circuitos exigen razonar en términos de restricciones, pero permiten ajustar la representación a la computación, mientras que una zkVM permite razonar en términos de un programa convencional, lo que facilita el desarrollo y la reutilización de bibliotecas, a cambio de un costo adicional por cada instrucción ejecutada. Este compromiso motiva la comparación entre Noir y SP1 planteada en la hipótesis del presente trabajo.

3.1.5. SNARKs y STARKs

Las propiedades descritas en la Sección 3.1.2 no se refieren al tamaño de la prueba ni al costo de verificarla, aspectos determinantes en las aplicaciones blockchain, ya que la prueba debe transmitirse en una transacción y su verificación se paga en gas. Un sistema de prueba es **sucinto** cuando tanto el tamaño de la prueba como el tiempo de verificación son muy pequeños en comparación con el tamaño de la computación probada [49]. Las dos familias de sistemas sucintos más utilizadas en la práctica son los SNARKs y los STARKs.

Ambas familias comparten una idea común, que consiste en codificar la computación como relaciones entre polinomios y comprobarlas en unos pocos puntos elegidos al azar, lo que se fundamenta en el principio de Schwartz–Zippel [64, 65] y reduce drásticamente el trabajo del verificador. Se diferencian principalmente en el mecanismo con el que el probador se compromete con esos polinomios, basado en curvas elípticas en los SNARKs más difundidos, como ocurre con el esquema KZG [66], y en funciones de hash en los STARKs, mediante árboles de Merkle y el protocolo FRI [67].

SNARKs. La sigla SNARK (*Succinct Non-interactive ARgument of Knowledge*) describe un argumento sucinto, no interactivo y con solidez de conocimiento [68]. Un SNARK que además es de conocimiento cero se denomina zk-SNARK. Las construcciones más difundidas se basan en curvas elípticas y emplean una operación criptográfica denominada *emparejamiento bilineal*, definida entre grupos asociados a dichas curvas. El sistema de Groth, conocido como Groth16, produce pruebas del orden de pocos cientos de bytes, cuya verificación requiere un número constante de emparejamientos más un trabajo proporcional al número de entradas públicas, ya que cada una de ellas debe combinarse con la clave de verificación mediante una operación sobre la curva [69, 70]. Para un número fijo de entradas públicas y un parámetro de seguridad dado, el costo de verificación no depende, por lo tanto, del tamaño del circuito. PLONK, otro sistema de esta familia, admite una configuración universal, válida para cualquier circuito hasta un tamaño máximo, en lugar de la configuración específica para cada circuito que requiere Groth16 [71]. Noir, el lenguaje analizado en la Sección 3.3, delega la generación de pruebas en un backend independiente. Su backend de referencia, Barretenberg, genera pruebas mediante UltraHonk, un sistema de la familia de PLONK [3, 72].

La principal limitación de construcciones como Groth16, o de PLONK instanciado con compromisos KZG, es que requieren una configuración confiable, cuyos valores secretos permitirían

generar pruebas falsas si no se destruyen. Este requisito depende de la construcción y no es propio de todos los SNARKs, ya que existen esquemas basados en compromisos transparentes que no necesitan una ceremonia de este tipo [49]. Para reducir esta confianza se realizan ceremonias con numerosos participantes, cuyo resultado es seguro siempre que al menos uno de ellos haya destruido su contribución [73]. En contrapartida, estos sistemas ofrecen la verificación más económica disponible en Ethereum, que desde 2017 incorpora contratos precompilados para las operaciones sobre la curva elíptica `alt_bn128`, también denominada BN254 [10, 11].

STARKs. La sigla STARK (*Scalable Transparent ARgument of Knowledge*) designa una familia de sistemas propuesta por Ben-Sasson et al. que prescinde de la configuración confiable [74]. En lugar de curvas elípticas, los STARKs se basan únicamente en funciones de hash resistentes a colisiones, lo que los hace además plausiblemente resistentes a ataques mediante computación cuántica. Son *transparentes*, en el sentido descrito en la Sección 3.1.3, y *escalables*, ya que el tiempo de generación de la prueba crece de manera casi lineal con el tamaño de la computación, lo que resulta conveniente para computaciones extensas como la ejecución de un programa completo. Su principal desventaja es el tamaño de la prueba, varios órdenes de magnitud mayor que el de una prueba Groth16, lo que encarece considerablemente su verificación directa en Ethereum. La Tabla 3.1 resume las diferencias entre ambas familias.

Tabla 3.1: Diferencias conceptuales entre SNARKs y STARKs.

Característica	**SNARK**	**STARK**
Tamaño de la prueba	Muy pequeño	Mayor
Verificación	Muy eficiente	Eficiente, pero generalmente más costosa
Configuración inicial	Algunas construcciones requieren una configuración confiable	Transparente
Base criptográfica	Habitualmente curvas elípticas y emparejamientos	Funciones de hash y FRI
Contexto de uso	Circuitos y verificación compacta	Computaciones extensas y sistemas de tipo zkVM

Ambas familias no son excluyentes. Es posible verificar una prueba STARK dentro de un circuito y generar sobre esa verificación una prueba SNARK, de manera que el resultado combine la escalabilidad del probador de los STARKs con la verificación económica de los SNARKs. Esta composición es la que emplean las zkVM modernas para que sus pruebas puedan verificarse en Ethereum, y su aplicación en SP1 se analiza en la Sección 3.4.

Relación con los puentes entre cadenas. Los conceptos presentados permiten reformular el problema de los puentes descrito en la Sección 2.5. Un contrato inteligente desplegado en la

cadena de destino necesita establecer la validez de una afirmación sobre otra cadena, por ejemplo, que una transacción de bloqueo fue incluida en un bloque certificado por los validadores de la cadena de origen. Un cliente ligero (Sección 2.1.4) establecería esta afirmación verificando la firma que certifica el encabezado y la prueba de inclusión de la transacción, pero reproducir esas verificaciones dentro de un contrato inteligente resulta demasiado costoso. Si ambas verificaciones se expresan como un único enunciado, el trabajo costoso queda a cargo de un probador fuera de la cadena, y el contrato solo necesita verificar una prueba sucinta. La solidez del sistema garantiza entonces que ningún probador puede obtener una prueba válida de un enunciado falso respecto de la relación compilada, de modo que la aceptación del contrato deja de depender de la honestidad de un intermediario. Esa garantía no agota, sin embargo, la seguridad del puente, que depende también de que las restricciones expresen fielmente la afirmación prevista, de los supuestos de seguridad y de la finalidad del consenso de la cadena de origen, de la vinculación criptográfica entre la prueba y los datos públicos que utiliza el contrato receptor, de la corrección de la lógica de ese contrato y de su protección contra la repetición de una misma operación. Xie et al. demostraron la viabilidad práctica de este enfoque [75]. El diseño concreto de esta verificación para el puente desarrollado en el presente trabajo se expone en el Capítulo 4.

3.2. Circom

La Sección 3.1.4 presentó el enfoque basado en circuitos aritméticos y restricciones. Circom es una de las herramientas más difundidas dentro de ese enfoque y permite observar en concreto cómo una descripción de la computación se transforma en un sistema de restricciones y, finalmente, en una prueba verificable en Ethereum. Aunque el puente desarrollado en este trabajo no emplea Circom, su análisis sirve de referencia para las implementaciones en Noir y SP1 descritas en las secciones siguientes.

3.2.1. Origen y propósito

Circom es un lenguaje para la descripción de circuitos aritméticos, acompañado de un compilador que traduce dichas descripciones a un sistema de restricciones de rango uno (R1CS). Fue desarrollado en el ecosistema del proyecto iden3, y su diseño y semántica fueron descritos académicamente por Bellés-Muñoz et al. [76]. La versión actual del compilador, conocida como Circom 2, está implementada en Rust [77]. El ecosistema se completa con circomlib, una biblioteca de plantillas que implementan primitivas criptográficas [78], y con snarkjs, que implementa los sistemas de prueba que consumen los artefactos producidos por el compilador [70]. Circom no constituye, por lo tanto, un sistema de prueba por sí mismo.

Circom es un **lenguaje de dominio específico** (*domain-specific language*, DSL), cuyas construcciones corresponden directamente a las señales, que representan valores de $\mathbb{F}_p$, y a las restricciones que dichas señales deben satisfacer. La descripción de un circuito cumple dos

funciones distintas. Por un lado, indica cómo calcular los valores de las señales a partir de las entradas, cálculo que solo realiza el probador. Por otro, establece las ecuaciones que esos valores deben satisfacer, y únicamente estas ecuaciones forman parte de lo que el verificador comprueba. Esta separación, que Circom expone de forma explícita al desarrollador, es el aspecto central del lenguaje y la fuente de sus principales riesgos, analizados en la Sección 3.2.4.

3.2.2. Construcción de circuitos

La unidad básica de descripción en Circom es la **plantilla** (*template*), que define un subcircuito parametrizable y puede instanciarse como **componente** dentro de otras plantillas. Todo circuito tiene un componente principal, denominado `main`, que define su interfaz [77]. Las **señales** se clasifican en señales de entrada (`signal input`), de salida (`signal output`) e intermedias (`signal`). Las salidas del componente principal son siempre públicas, las entradas son privadas salvo que se declaren explícitamente como públicas mediante la sintaxis `component main {public [...]}`, y las señales intermedias son siempre privadas [77]. Las señales públicas forman, por lo tanto, el enunciado (Sección 3.1.1), mientras que los valores de todas las señales conforman el testigo. Las variables declaradas con `var`, en cambio, no forman parte del testigo ni generan restricciones por sí mismas.

La relación entre señales se establece mediante tres operadores [77]. El operador `===` impone una restricción sin asignar ningún valor, `<--` asigna un valor sin generar ninguna restricción, de modo que ese valor solo interviene en el cálculo del testigo, y `<==` combina ambas acciones. El compilador exige que toda restricción sea de grado a lo sumo dos y pueda llevarse a la forma $A \cdot B + C = 0$, donde A, B y C son combinaciones lineales de señales, que coincide con la de una fila de un sistema R1CS [77]. La restricción $e = a \cdot b + c \cdot d$, por ejemplo, es de grado dos pero no admite esa forma, por lo que el compilador la rechaza y debe descomponerse mediante una señal intermedia.

La Figura 3.2 retoma la computación de la Sección 3.1.4. El circuito permite demostrar el conocimiento de un valor privado w tal que $w^3 + w + 5 = y$, siendo y una salida pública.

Cada operador `<==` del ejemplo expresa una restricción, de modo que el código describe el sistema

$$w \cdot w - v_1 = 0, \qquad v_1 \cdot w - v_2 = 0, \qquad (v_2 + w + 5) \cdot 1 - y = 0, \tag{3.2}$$

donde las operaciones se realizan en $\mathbb{F}_p$ y p es, por defecto, el orden primo de 254 bits del grupo asociado a la curva BN254, para la que Ethereum dispone de contratos precompilados [77, 11]. Para $w = 3$ se obtienen los valores $(y, w, v_1, v_2) = (35, 3, 9, 27)$. Solo y es pública, aunque que la prueba no revele información adicional sobre w depende de que el sistema de prueba empleado sea de conocimiento cero (Sección 3.1.2).

El sistema compilado no coincide necesariamente con el expresado en el código, ya que la opción `--O2` elimina restricciones lineales sustituyendo una señal privada por la expresión que la define, a diferencia de `--O1`, nivel predeterminado desde la versión 2.2.0 [77, 79]. En una compilación del ejemplo realizada para el presente trabajo con la versión 2.2.3, `--O2` sustituyó

$v_2 = y - w - 5$ en la segunda restricción y redujo el sistema a dos restricciones equivalentes.

```
pragma circom 2.1.6;

template Cubic() {
    signal input w;      // entrada privada
    signal output y;     // salida publica
    signal v1;           // senales intermedias
    signal v2;

    v1 <== w * w;
    v2 <== v1 * w;
    y  <== v2 + w + 5;
}

component main = Cubic();
```

Figura 3.2: Circuito Circom que expresa la relación $w^3 + w + 5 = y$.

Un circuito está **correctamente restringido** cuando las asignaciones que satisfacen su sistema corresponden exactamente a los casos en que se cumple la relación que se pretende demostrar, aunque no todas las señales auxiliares queden determinadas unívocamente. En el ejemplo, una vez fijado w, las restricciones determinan v_1, v_2 e y, de modo que ningún probador puede presentar un valor de y distinto de $w^3 + w + 5$. Un circuito *insuficientemente restringido* admite, en cambio, asignaciones que conducen a aceptar afirmaciones que la especificación rechaza, y uno *sobrerrestringido* rechaza asignaciones legítimas. Estos defectos no afectan a la solidez del sistema de prueba respecto de la relación compilada, sino a la correspondencia entre esa relación y la especificación prevista.

3.2.3. Compilación, cálculo del testigo y generación de la prueba

El proceso que conduce desde un circuito hasta una prueba verificable concreta las etapas descritas en la Sección 3.1.4 y se distribuye entre el compilador de Circom y un sistema de prueba externo. La Figura 3.3 lo representa para el caso de Groth16 implementado en snarkjs.

En la primera etapa, el compilador genera dos artefactos independientes, el sistema de restricciones, almacenado en un archivo `.r1cs`, y un programa que calcula el testigo, en WebAssembly o en C++ [77]. Durante la compilación comprueba además la forma de las restricciones y aplica el nivel de simplificación elegido. Con la opción `--inspect` realiza también comprobaciones destinadas a identificar señales potencialmente insuficientemente restringidas, como las que no aparecen en ninguna restricción o las asignadas con `<--` a partir de una expresión cuadrática [77, 79].

En la segunda etapa, el probador ejecuta el calculador del testigo sobre entradas concretas, expresadas en formato JSON, y obtiene un archivo `.wtns` con la asignación correspondiente al sistema compilado [77]. Este cálculo no constituye una garantía criptográfica, ya que un

probador puede producir el testigo por cualquier otro medio. Lo que determina si una prueba será aceptada es únicamente que el testigo satisfaga el sistema de restricciones.

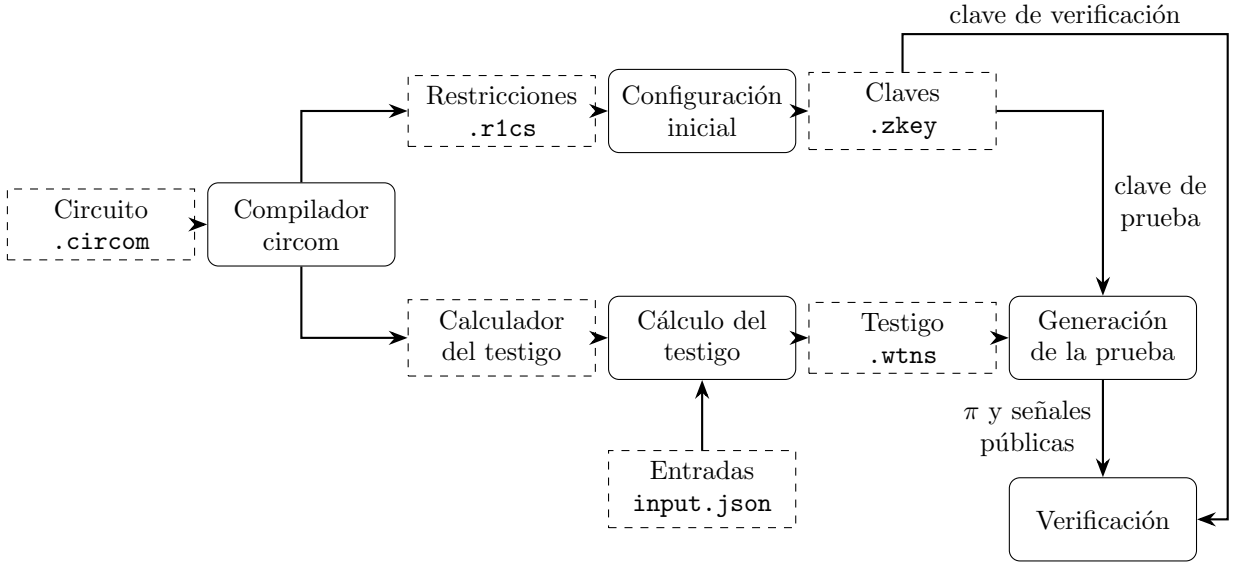


Figura 3.3: Proceso de compilación, cálculo del testigo, generación y verificación de una prueba a partir de un circuito Circom.

La tercera etapa corresponde a la configuración inicial. En el caso de Groth16 (Sección 3.1.5), la configuración se divide en dos fases. La primera, denominada *powers of tau*, es independiente del circuito y puede reutilizarse para cualquier circuito hasta un tamaño máximo. La segunda es específica del circuito y produce un archivo `.zkey` con la clave de prueba, del que se exporta la clave de verificación [77]. Ambas fases admiten contribuciones de múltiples participantes [73].

En la cuarta etapa, el probador combina la clave de prueba y el testigo para generar la prueba π, junto con un archivo que contiene los valores de las señales públicas. La verificación puede realizarse con snarkjs o mediante un contrato en Solidity que la propia herramienta genera a partir de la clave de verificación y que se apoya en los contratos precompilados de Ethereum para la curva BN254 [70, 10, 11]. El verificador no recibe el archivo `.r1cs` ni el programa que calculó el testigo, pero la clave de verificación se deriva del sistema compilado, por lo que la prueba certifica exactamente la relación que dicho sistema expresa y ninguna propiedad que no haya quedado expresada como restricción.

3.2.4. Limitaciones y consideraciones de seguridad

La separación entre cálculo y restricción es la principal fuente de errores en el desarrollo con Circom. El operador `<--` resulta necesario cuando el valor de una señal no puede calcularse mediante una restricción cuadrática, pero ese valor solo queda respaldado por la prueba si el desarrollador añade restricciones que lo vinculen con las demás señales. La plantilla `IsZero` de circomlib, reproducida en la Figura 3.4, ilustra este patrón [78].

La plantilla devuelve 1 si la entrada es nula y 0 en caso contrario. El valor de `inv` se obtiene mediante una expresión condicional con una división, que no puede asignarse con `<==`, por lo

que el operador `<--` solo aporta un valor candidato, y son las restricciones las que garantizan el resultado. Si la entrada es distinta de cero, la segunda obliga a que la salida sea nula, y la primera obliga entonces a que `inv` sea el inverso de la entrada. Si la entrada es nula, la primera obliga a que la salida sea 1 sin determinar `inv`, libertad que no permite aceptar ninguna afirmación incorrecta. Si, en cambio, un circuito asignara su salida mediante `y <-- w * w;` sin ninguna restricción adicional, ninguna ecuación vincularía y con w, y un probador deshonesto podría generar una prueba válida para cualquier valor de y. Las comprobaciones descritas en la Sección 3.2.3 advierten sobre este caso concreto, pero solo identifican patrones conocidos y no sustituyen una revisión de seguridad del circuito [77].

```
template IsZero() {
    signal input in;
    signal output out;
    signal inv;

    inv <-- in != 0 ? 1 / in : 0; // solo calculo
    out <== -in * inv + 1;         // asigna y restringe
    in * out === 0;                // solo restringe
}
```

Figura 3.4: Plantilla `IsZero` de circomlib, que combina un cálculo auxiliar con restricciones explícitas.

Los circuitos insuficientemente restringidos constituyen una de las categorías de vulnerabilidades más estudiadas en sistemas de prueba sucintos [62] y han motivado herramientas de análisis automático específicas [80, 81]. Este riesgo no es exclusivo de Circom, sino propio de todo enfoque en el que el desarrollador define explícitamente las restricciones.

Una segunda limitación, común al enfoque basado en circuitos, es el costo de las operaciones que no son nativas del cuerpo finito (Sección 3.1.4). La plantilla `Num2Bits` de circomlib descompone un valor en n bits mediante una restricción de la forma $b_i(b_i - 1) = 0$ para cada bit y otra que recompone el valor original [78]. Las funciones de hash que operan sobre palabras binarias, como SHA-256 y Keccak-256, deben construirse sobre descomposiciones de este tipo, y el formato R1CS no incorpora de forma nativa los argumentos de búsqueda que se retoman en la Sección 3.3.

Estas limitaciones adquieren especial relevancia en un puente entre cadenas, en el que conviene distinguir tres afirmaciones diferentes. La primera es que una transacción fue incluida en un bloque, lo que se demuestra mediante una prueba de inclusión respecto de la raíz de transacciones del encabezado y exige recalcular hashes Keccak-256. La segunda es que dicho bloque es auténtico y está finalizado según el consenso de la cadena de origen. La tercera es que la operación de bloqueo se ejecutó con éxito, ya que una transacción incluida puede haber revertido su ejecución. La prueba de inclusión no establece esta última propiedad, cuya demostración puede requerir el recibo de la transacción, que registra su estado de ejecución y los eventos emitidos [8, 19]. La segunda afirmación, por su parte, no se reduce a la verificación de una firma, ya que

el protocolo de cliente ligero de Ethereum exige además comprobar, entre otros aspectos, la participación del comité de sincronización, el dominio de firma y la pertenencia del encabezado finalizado al estado certificado [24, 25]. La verificación de la firma es, aun así, la comprobación más costosa de expresar como circuito, ya que emplea la curva BLS12-381, cuyo cuerpo base de 381 bits excede al de 254 bits del circuito y obliga a emular cada operación aritmética. La implementación de referencia circom-pairing documenta que la verificación de una única firma BLS requiere aproximadamente 19,2 millones de restricciones no lineales y una clave de prueba de 12 GB [82]. Estas cifras no constituyen una medición del presente trabajo, pero ilustran el orden de magnitud del problema. La forma en que el puente desarrollado aborda cada una de las tres afirmaciones se expone en el Capítulo 4.

Circom ofrece, en definitiva, un control preciso sobre las restricciones generadas, lo que permite optimizar manualmente los circuitos y conocer con exactitud la relación que se probará. Ese control tiene como contrapartida una mayor dificultad de desarrollo, ya que el programador debe razonar simultáneamente sobre el cálculo del testigo y sobre las ecuaciones que lo restringen. Noir, analizado en la Sección 3.3, conserva el enfoque basado en circuitos, pero eleva el nivel de abstracción y asume la generación de las restricciones correspondientes.

3.3. Noir

La Sección 3.2 concluyó que Circom ofrece un control preciso sobre las restricciones a costa de exigir al desarrollador que razone simultáneamente sobre el cálculo del testigo y sobre las ecuaciones que lo restringen. Noir conserva el enfoque basado en circuitos descrito en la Sección 3.1.4, pero eleva el nivel de abstracción, ya que el desarrollador describe la computación con construcciones próximas a las de un lenguaje de programación convencional y el compilador asume la generación de las restricciones. Este cambio desplaza el trabajo de traducción hacia el compilador, pero no la responsabilidad de especificar correctamente la relación que se pretende demostrar. La presente sección describe el origen y los componentes de Noir, la forma de describir computaciones en el lenguaje, el proceso que conduce desde el código fuente hasta una prueba verificable en Ethereum mediante el backend Barretenberg y las limitaciones relevantes para un puente entre cadenas. Dado que varios de estos aspectos cambian entre versiones, la descripción corresponde a las utilizadas en el proyecto, Nargo 1.0.0-beta.22 y Barretenberg 5.0.0-nightly.20260522.

3.3.1. Origen y propósito

Noir es un lenguaje de dominio específico, de código abierto, para escribir programas cuya ejecución correcta puede demostrarse mediante pruebas de conocimiento cero, desarrollado en el ecosistema de Aztec [3, 83]. Su documentación lo describe como un lenguaje influido por Rust [84], y su sintaxis adopta efectivamente construcciones de ese lenguaje, como la declaración de funciones con `fn`, las ligaduras con `let` o el sistema de módulos. Noir no es, sin embargo, un

subconjunto de Rust. Posee un sistema de tipos propio, organizado alrededor del tipo `Field`, impone restricciones que no existen en Rust, como el tamaño fijo de las entradas del programa principal, y no es compatible con las bibliotecas de ese ecosistema.

En la cadena de herramientas de Noir conviene distinguir cinco elementos con funciones diferentes. El primero es el lenguaje y su compilador, que traduce el programa a una representación intermedia denominada ACIR, descrita en la Sección 3.3.3. El segundo es Nargo, la herramienta de línea de órdenes que gestiona los proyectos y sus dependencias e invoca al compilador, al ejecutor y a las pruebas unitarias. El tercero corresponde a los componentes que calculan el testigo ejecutando la representación intermedia, disponibles tanto a través de Nargo como de la biblioteca `noir_js`, que permite hacerlo desde JavaScript [85]. El cuarto es el backend de pruebas, en este caso Barretenberg, una biblioteca en C++ que se utiliza mediante la interfaz de línea de órdenes `bb` o mediante `bb.js`, su envoltorio para JavaScript y WebAssembly [86, 87]. El quinto es el sistema criptográfico que implementa ese backend, UltraHonk, cuyas características se describen en la Sección 3.3.3.

Esta separación sustenta la independencia de Noir respecto del backend, ya que el repositorio oficial declara que el lenguaje fue diseñado para utilizar cualquier sistema de prueba compatible con ACIR [83]. Dicha independencia tiene, no obstante, condiciones precisas. El backend debe interpretar todos los tipos de instrucciones de ACIR, incluidas las funciones de caja negra que la especificación delega en él [84]. El tamaño del tipo `Field` depende del cuerpo nativo del backend, por lo que un programa cuya corrección dependa del módulo no es trasladable sin revisión [84]. Además, los formatos de los artefactos y de las pruebas están ligados a versiones concretas de las herramientas, motivo por el cual el proyecto exige utilizar pares de versiones de Nargo y Barretenberg que coincidan entre sí. La independencia se sitúa, por lo tanto, en el nivel del lenguaje y de la representación intermedia, y no implica que cualquier versión de cualquier herramienta pueda intercambiarse.

El interés de Noir para el presente trabajo reside en la forma de representar la computación. La verificación de una prueba de inclusión sobre un árbol de Merkle-Patricia exige decodificar estructuras RLP, recorrer nodos y recalcular hashes Keccak-256, operaciones que en Noir pueden expresarse con funciones, ciclos acotados y arreglos, sin escribir manualmente cada restricción. Al mismo tiempo, Noir permanece dentro del enfoque basado en circuitos, de modo que el sistema resultante describe específicamente esa verificación, y Barretenberg permite generar verificadores en Solidity para comprobar las pruebas en Ethereum. Las consecuencias de estas características sobre el rendimiento no se anticipan en este capítulo, sino que forman parte de la comparación experimental del Capítulo 4.

3.3.2. Descripción de computaciones y generación de restricciones

Un programa Noir se compone de funciones cuyos parámetros tienen tipos conocidos durante la compilación. Dentro de ellas, las ligaduras declaradas con `let` son inmutables salvo que se indique `mut`, las operaciones aritméticas se escriben con la notación habitual y la instrucción

`assert` establece una condición que toda ejecución válida debe satisfacer. La función `main` es el punto de entrada del programa y sus parámetros deben tener un tamaño fijo, conocido durante la compilación [84].

El tipo `Field` representa un elemento del cuerpo nativo del backend, que en el caso de Barretenberg es un cuerpo primo de 254 bits [84]. Su aritmética es la de $\mathbb{F}_p$ descrita en la Sección 3.1.4, por lo que el resultado se reduce módulo p sin ninguna comprobación de desbordamiento. Los tipos enteros, como `u8`, `u32` o `u64`, son en cambio valores del cuerpo restringidos a un rango [84]. En una compilación realizada para el presente trabajo con Nargo 1.0.0-beta.22, una función que suma dos parámetros `u8` dio lugar a restricciones de rango de 8 bits sobre cada parámetro y sobre el resultado de la suma, de modo que la ejecución con los valores 200 y 100 falló por desbordamiento, mientras que la suma de $p - 1$ y 1 con el tipo `Field` produjo 0 sin error. En esta operación concreta, la suma sobre `Field` evita las restricciones de rango y la comprobación de desbordamiento que requiere la suma de enteros, aunque exige al desarrollador garantizar que el comportamiento modular sea el previsto. El ejemplo no permite, sin embargo, concluir que `Field` sea más económico en cualquier programa, ya que el costo final depende también de las operaciones posteriores, de las comprobaciones que la aplicación necesite y de las optimizaciones del backend. Los enteros, por su parte, incorporan automáticamente las restricciones necesarias para reproducir la aritmética de ancho fijo.

La visibilidad se declara de forma explícita en la firma de la función principal. Sus parámetros son privados salvo que se marquen con `pub`, en cuyo caso forman parte de las entradas públicas, y su valor de retorno, si existe, debe declararse obligatoriamente como público, ya que el compilador rechaza un tipo de retorno de `main` sin `pub`. A diferencia de Circom, donde las salidas del componente principal son siempre públicas y las entradas se hacen públicas mediante una lista en la declaración de `main` (Sección 3.2.2), en Noir la visibilidad se indica individualmente para cada parámetro.

La Figura 3.5 muestra un programa que expresa la misma relación que el circuito de la Figura 3.2, $y = w^3 + w + 5$, con w privado e y público. Se eligió una interfaz con una entrada pública comprobada mediante una aserción, en lugar de un valor de retorno público, porque reproduce el modo en que un contrato verificador recibe el valor público y lo contrasta con sus propios datos. Ambas formulaciones son válidas y en las dos el valor de y forma parte del enunciado.

```
1 // w: privado; y: publico
2 fn main(w: Field, y: pub Field) {
3     assert(w * w * w + w + 5 == y);
4 }
```

Figura 3.5: Programa Noir que expresa la relación $w^3 + w + 5 = y$.

En este programa, el probador aporta el valor privado w y el valor público y, mientras que el enunciado contiene únicamente y. La prueba respalda que el probador conoce un w tal que $w^3 + w + 5 = y$ en $\mathbb{F}_p$, relación que, al tratarse del tipo `Field`, se evalúa módulo

p. El desarrollador no declaró valores intermedios, ya que es el compilador quien traduce la expresión a la representación intermedia. En la compilación realizada para el presente trabajo, el programa dio lugar a dos aserciones aritméticas, $w_2 = w_0 \cdot w_0$ y $w_1 = w_0 \cdot w_2 + w_0 + 5$, donde w_0 corresponde a w, w_1 a y y w_2 es un valor intermedio introducido por el compilador, análogo a la señal v_1 del ejemplo de Circom. La ejecución con $w = 3$ e $y = 35$ produjo un testigo válido, mientras que con $y = 36$ falló porque la aserción no puede satisfacerse. La relación demostrada es matemáticamente equivalente a la del ejemplo de Circom, pero su representación no lo es. Noir no produce un sistema R1CS, sino la representación intermedia descrita a continuación, que el backend transforma a su vez en las restricciones de su propio sistema de prueba, por lo que el número de restricciones y el costo de ambos ejemplos no son comparables directamente.

3.3.3. Compilación, cálculo del testigo y generación de la prueba

El compilador de Noir produce un programa en **ACIR** (*Abstract Circuit Intermediate Representation*), una representación intermedia diseñada para servir de enlace entre un lenguaje de alto nivel y un sistema de prueba, independiente de cualquier sistema concreto [83]. Un programa ACIR es una secuencia de instrucciones, denominadas *opcodes*, que restringen un conjunto de valores llamados testigos. La instrucción más básica es una aserción sobre una expresión, entendida como la suma de términos lineales en los testigos, de términos formados por el producto de dos testigos y de un término constante. Una fila de R1CS relaciona el producto de dos combinaciones lineales con una tercera (Sección 3.2.2), mientras que una expresión de ACIR puede contener varios términos de producto independientes. La restricción $e = a \cdot b + c \cdot d$, que no puede escribirse como el producto de dos combinaciones lineales y que el compilador de Circom rechaza, se compiló en Noir como una única aserción. ACIR incluye además llamadas a funciones de caja negra, cuya implementación se delega en el backend, operaciones sobre memoria y llamadas a código no restringido [83, 84]. Los testigos de ACIR se denominan *parciales*, porque el backend puede introducir testigos y restricciones adicionales al construir su propio circuito, en particular al expandir las funciones de caja negra.

La representación intermedia debe distinguirse de su ejecución. La **ACVM** (*Abstract Circuit Virtual Machine*) es la máquina virtual que ejecuta un programa ACIR sobre las entradas proporcionadas por el usuario y obtiene el testigo parcial que el backend necesita para generar la prueba [83]. **Brillig**, por su parte, es un formato de código y una máquina virtual de propósito general destinados a ejecutar las funciones no restringidas de Noir. Cuando el programa ACIR contiene una llamada a código no restringido, la ACVM delega su ejecución en la máquina virtual de Brillig e incorpora los resultados al testigo [83]. Ni ACIR, ni la ACVM, ni Brillig constituyen un sistema criptográfico, ya que la generación y verificación de la prueba corresponde exclusivamente al backend.

La Figura 3.6 representa el recorrido completo desde el código fuente hasta una prueba verificable, tal como se organiza con las herramientas utilizadas en el proyecto.

La primera etapa es la compilación. Nargo compila el programa y produce un artefacto que

contiene el código ACIR, el código Brillig de las funciones no restringidas y una descripción de la interfaz del programa principal, que indica el tipo y la visibilidad de cada entrada. La segunda etapa consiste en preparar las entradas concretas, ya sea en el archivo de configuración que utiliza Nargo o como un objeto de JavaScript cuando se emplea `noir_js`, codificadas según esa interfaz. En la tercera etapa, la ACVM ejecuta el programa sobre esas entradas y obtiene el testigo, ya sea mediante la orden `nargo execute` o mediante la función de ejecución de `noir_js` [84, 85]. Si alguna aserción no puede satisfacerse, la ejecución falla y no se obtiene ningún testigo.

La cuarta etapa corresponde a la preparación del backend. Barretenberg construye su propio circuito a partir del código ACIR, expandiendo las funciones de caja negra, y necesita una cadena de referencia común cuyo tamaño depende del tamaño del circuito. La clave de verificación se deriva del código ACIR y de esa cadena sin necesitar el testigo de una ejecución válida del programa. En la versión utilizada, `bb` construye el circuito para ese fin con valores auxiliares generados rápidamente y no válidos, que solo sirven para expandir las funciones de caja negra, de modo que la clave puede generarse una única vez por circuito, antes de cualquier operación concreta, y utilizarse para producir el contrato verificador en Solidity [86]. Esta etapa no depende, por lo tanto, de la tercera, y ambas pueden realizarse de forma independiente. En la quinta etapa, el backend combina el código ACIR, el testigo y la cadena de referencia para generar la prueba π junto con los valores de las entradas públicas, mediante la orden `bb prove` o mediante la clase `UltraHonkBackend` de `bb.js` [86, 87]. Finalmente, la verificación puede realizarse con `bb`, con `bb.js` o mediante el contrato generado, que requiere los contratos precompilados de Ethereum para la suma, la multiplicación escalar y el emparejamiento sobre la curva BN254, así como el de exponenciación modular [86].

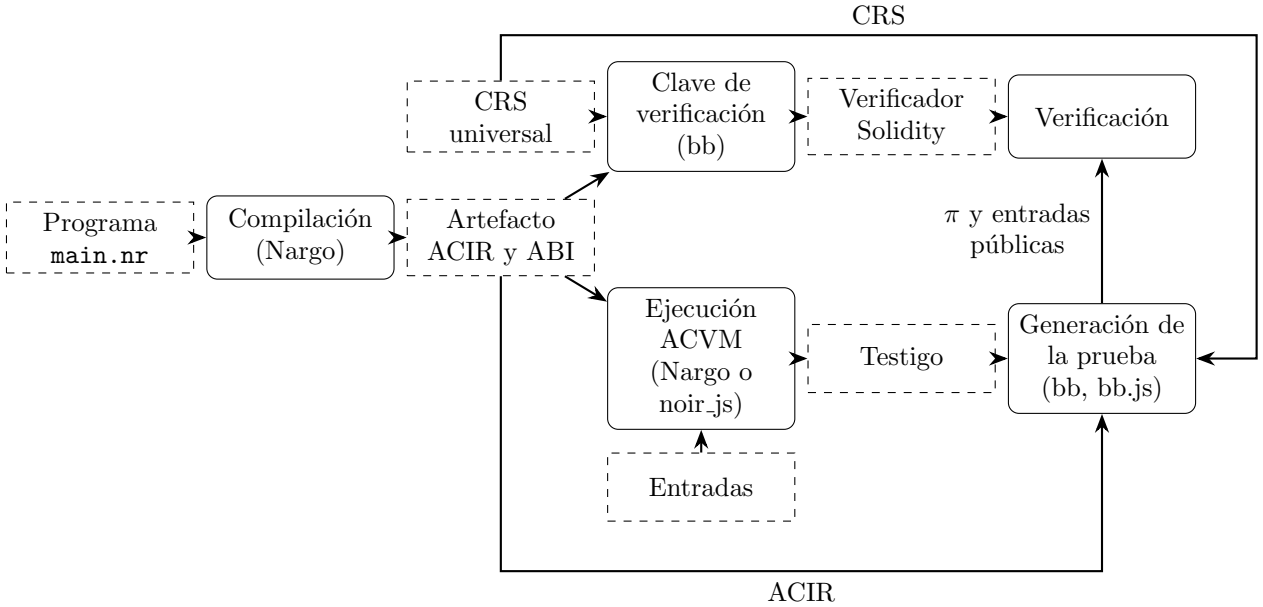


Figura 3.6: Proceso de compilación, cálculo del testigo, generación y verificación de una prueba a partir de un programa Noir con el backend Barretenberg.

Según su documentación y su código fuente, el prototipo del puente distribuye estas he-

rramientas en dos momentos. Nargo y la interfaz `bb` se emplean durante la preparación, para compilar el circuito y generar la clave de verificación y el contrato verificador. Durante cada operación del puente, en cambio, el servicio que genera las pruebas utiliza `noir_js` para calcular el testigo y `bb.js` para generar la prueba, mientras que la verificación se realiza en la cadena de destino mediante el contrato generado. Los detalles de esta integración se describen en el Capítulo 4.

El sistema de prueba que implementa Barretenberg en esta modalidad es **UltraHonk**, al que su documentación describe como un sistema de tipo PLONK más eficiente que su predecesor UltraPlonk [72, 86]. Su aritmetización organiza el circuito como una tabla de ejecución con un número fijo de columnas, en la que cada fila representa una compuerta cuyo tipo se determina mediante polinomios selectores. Además de las compuertas aritméticas, la clave de verificación generada en la versión utilizada declara selectores para compuertas especializadas, entre ellas las de comprobación de rango, operaciones sobre curvas elípticas, accesos a memoria, aritmética sobre cuerpos no nativos y la función de hash Poseidon2. La igualdad entre valores de distintas filas se impone mediante restricciones de copia, como en PLONK [71], y la pertenencia de ciertos valores a tablas precomputadas se comprueba mediante un argumento de búsqueda. Esta técnica fue introducida en este contexto por Plookup [88], aunque la versión de Barretenberg utilizada implementa una variante basada en derivadas logarítmicas [89, 86]. Estas **tablas de búsqueda** permiten verificar operaciones como las comprobaciones de rango o las operaciones bit a bit consultando una tabla, en lugar de descomponer cada valor en bits mediante restricciones aritméticas, como se describió para Circom en la Sección 3.2.4. Las relaciones del circuito se verifican mediante el protocolo *sumcheck*, que la documentación de Barretenberg incluye entre sus componentes con y sin conocimiento cero [86]. Para el ejemplo de la Figura 3.5, la herramienta `bb gates` informó un circuito de 20 filas, cifra que incluye filas fijas del sistema y que ilustra que el número de instrucciones ACIR no coincide con el tamaño del circuito que finalmente se prueba.

Tres propiedades de esta modalidad deben precisarse, porque no se aplican automáticamente a cualquier ejecución de Noir. En primer lugar, la variante destinada a Ethereum emplea compromisos KZG sobre la curva BN254 [86], por lo que requiere una configuración confiable de tipo universal, en el sentido de la Sección 3.1.5. Barretenberg obtiene la cadena de referencia de la ceremonia Ignition de Aztec, en la que participaron 176 personas e instituciones entre 2019 y 2020 y cuya seguridad requiere que al menos un participante haya destruido su aleatoriedad [90]. Los puntos necesarios se descargan cuando no están disponibles localmente y su integridad se comprueba mediante hashes [86]. A diferencia del flujo de Groth16 descrito para Circom, no se requiere una fase específica para cada circuito. En segundo lugar, la propiedad de conocimiento cero depende de la variante elegida. El destino `evm` de `bb` genera pruebas con conocimiento cero y utiliza Keccak como función de hash en la transformación de Fiat-Shamir, mientras que el destino `evm-no-zk` prescinde de dicha propiedad [86]. En tercer lugar, la verificación en Ethereum exige que la prueba, la clave de verificación y el contrato verificador se generen con el mismo destino. En una ejecución realizada para el presente trabajo con Barre-

tenberg 5.0.0-nightly.20260522, la prueba del ejemplo con el destino `evm` ocupó 4544 bytes, un tamaño considerablemente mayor que el de una prueba Groth16, y fue verificada correctamente con `bb` y con `bb.js`.

Esta descripción permite identificar intervalos de tiempo de naturaleza distinta. La compilación, la generación de la clave de verificación y del contrato verificador y el despliegue de este último constituyen una preparación que se realiza una vez por circuito. La descarga de la cadena de referencia y la inicialización del backend son costos de puesta en marcha que pueden presentarse solo en la primera operación. El cálculo del testigo, la generación de la prueba y su eventual verificación local se repiten en cada operación, mientras que la verificación en la cadena de destino forma parte de una transacción, cuyo costo se expresa en gas y cuya duración incluye la espera hasta su inclusión en un bloque. Estas etapas no tienen necesariamente un equivalente directo en SP1, por lo que su correspondencia se establece en la metodología del Capítulo 4.

3.3.4. Limitaciones y consideraciones de seguridad

La primera limitación se refiere a la diferencia entre elevar el nivel de abstracción y garantizar la corrección de la especificación. En el código restringido, el compilador genera las restricciones correspondientes a cada instrucción, de modo que el desarrollador no necesita escribir por separado el cálculo y la restricción de cada valor, como ocurre en Circom. Esta automatización no elimina, sin embargo, todos los riesgos de restricciones insuficientes. Los resultados del cálculo no restringido, descrito más adelante, requieren comprobaciones explícitas cuando intervienen en la relación demostrada, y la prueba sigue certificando únicamente lo que el programa expresa. Un programa que verifica la inclusión de una transacción respecto de una raíz suministrada como entrada establece solo esa inclusión, y no la autenticidad de la raíz, la finalidad del bloque ni el éxito de la operación, distinciones ya presentadas en la Sección 3.2.4. Del mismo modo, una condición omitida en el programa no produce un error de compilación, sino una relación más débil que la prevista. Los errores de especificación de este tipo forman parte de las vulnerabilidades documentadas en sistemas basados en pruebas sucintas [62], y el propio repositorio de Noir advierte que la implementación no ha sido auditada ni revisada para su uso en producción [83].

Noir conserva, además, una forma de cálculo no restringido. Las funciones declaradas como `unconstrained` se compilan a código Brillig y se ejecutan fuera del sistema de restricciones, de manera que sus resultados se incorporan al testigo sin que ninguna restricción los respalde [84]. La documentación justifica este mecanismo porque ciertas operaciones resultan mucho más baratas de comprobar que de calcular dentro del circuito, y exige por ello que el programa añada las restricciones necesarias sobre sus resultados. La Figura 3.7 muestra este patrón para el cálculo de un inverso.

El ejemplo presenta dos particularidades que conviene precisar. Cuando $x = 0$, la función auxiliar devuelve cero, pero la aserción $x \cdot inv = 1$ no puede satisfacerse, de modo que el programa

completo solo acepta entradas no nulas. Además, el inverso se devuelve como valor público, y para una entrada no nula basta con invertirlo para recuperar x, por lo que la declaración de x como privada no mantiene su valor en secreto. Esto no contradice la propiedad de conocimiento cero (Sección 3.1.2), que garantiza que la prueba no revela información sobre el testigo más allá de la validez del enunciado, pero no oculta lo que puede deducirse de los propios valores públicos.

La semejanza con el operador `<--` de Circom se limita a que, en ambos casos, un valor entra en el testigo sin quedar restringido por la operación que lo calcula. Las diferencias son relevantes. Una función no restringida de Noir puede contener una computación completa, con ciclos y bifurcaciones, ejecutada por la máquina virtual de Brillig, y no una única asignación. Además, el lenguaje exige que su invocación desde código restringido se realice dentro de un bloque `unsafe`, cuya ausencia produce un error de compilación, y la versión utilizada advierte cuando ese bloque carece de un comentario que justifique su seguridad. En una compilación realizada para el presente trabajo, la eliminación de la aserción de la Figura 3.7 produjo un diagnóstico que indicaba que los resultados de la llamada no estaban suficientemente restringidos, aunque el programa se compiló y ejecutó de todos modos. Este diagnóstico, como las comprobaciones de Circom, identifica patrones conocidos, pero no sustituye una revisión del programa.

```
1  unconstrained fn inverse(x: Field) -> Field {
2      if x == 0 { 0 } else { 1 / x }
3  }
4
5  fn main(x: Field) -> pub Field {
6      // Safety: el resultado se restringe a continuacion
7      let inv = unsafe { inverse(x) };
8      assert(x * inv == 1);
9      inv
10 }
```

Figura 3.7: Cálculo no restringido de un inverso en Noir, seguido de la restricción que lo valida.

La vinculación entre los datos públicos y la relación demostrada constituye una segunda consideración. Como se señaló en la Sección 3.1.4, la garantía de la prueba alcanza únicamente a los valores que forman parte del enunciado, de modo que, si un contrato acuña activos a favor de un destinatario y por un monto determinados, ambos valores deben formar parte de las entradas públicas o estar vinculados criptográficamente a un valor público que el contrato compruebe. En el primer caso, el contrato compara directamente las entradas públicas con los parámetros de su propia llamada. En el segundo, el enunciado puede contener, por ejemplo, el hash de una codificación inequívoca de esos datos, de modo que el circuito restringe la relación entre el hash y los valores, y el contrato calcula el mismo hash a partir de los parámetros de su llamada y lo compara con la entrada pública. La interfaz concreta del prototipo se describe en el Capítulo 4. En cualquier caso, la codificación de las entradas públicas debe conocerse con precisión. En la versión utilizada, por ejemplo, un arreglo público de n elementos `u8` se expone

como n entradas públicas, cada una restringida a 8 bits, por lo que el contrato debe reconstruir los valores a partir de esa representación.

Las restricciones sobre el control de flujo y el tamaño de los datos derivan del enfoque basado en circuitos. En el código restringido, el número de iteraciones de cada ciclo debe conocerse durante la compilación, y las instrucciones para interrumpir un ciclo solo se admiten en código no restringido [84]. Los parámetros del programa principal deben tener un tamaño fijo, por lo que datos de longitud variable, como una transacción codificada en RLP o los nodos de una prueba de Merkle-Patricia, deben representarse mediante arreglos de un tamaño máximo acompañados de su longitud efectiva. El circuito incluye, en consecuencia, las restricciones correspondientes al tamaño máximo con independencia de la longitud real de los datos.

El costo de los hashes binarios y de la aritmética sobre cuerpos no nativos tampoco desaparece con el cambio de lenguaje. Noir ofrece algunas de estas operaciones como funciones de caja negra, entre ellas la permutación de Keccak, la compresión de SHA-256 y las operaciones bit a bit, cuya implementación se delega en el backend para que este utilice restricciones optimizadas cuando dispone de ellas [84]. La función Keccak-256 completa, en cambio, no forma parte de la biblioteca estándar de la versión utilizada y debe construirse sobre esa permutación o tomarse de bibliotecas externas [84]. Las tablas de búsqueda reducen el costo de las comprobaciones de rango y de las operaciones bit a bit, pero no eliminan el costo de emular la aritmética de un cuerpo de mayor tamaño, como el de 381 bits de la curva BLS12-381 que emplea la firma del comité de sincronización de Ethereum, cuyos elementos deben seguir representándose mediante varios valores del cuerpo nativo.

Por último, los costos de un programa Noir dependen del backend, de sus parámetros y de las versiones utilizadas. El número de instrucciones ACIR no determina por sí solo el tamaño del circuito, que depende de la forma en que el backend expande cada instrucción, y ese tamaño condiciona a su vez la cantidad de puntos de la cadena de referencia y la memoria necesarios para generar la prueba. Un cambio de versión de Nargo o de Barretenberg puede modificar la representación intermedia, el circuito resultante o el formato de las pruebas, por lo que cualquier medición debe asociarse a las versiones con que se obtuvo.

Noir permite describir la verificación como un programa, pero cada programa sigue dando lugar a un circuito específico. La Sección 3.4 estudia el enfoque alternativo presentado en la Sección 3.1.4, en el que se demuestra la ejecución de un programa arbitrario sobre una máquina virtual de conocimiento cero.

3.4. SP1

La Sección 3.3 concluyó que Noir permite describir la verificación como un programa, pero que cada programa sigue dando lugar a un circuito específico. SP1 adopta el enfoque alternativo presentado en la Sección 3.1.4, en el que el sistema de restricciones describe un procesador y la prueba certifica que un programa arbitrario se ejecutó correctamente sobre él. Para el desarrollador, el cambio consiste en escribir un programa convencional en Rust y delegar en

la máquina virtual la representación de su ejecución. Este cambio no elimina la necesidad de especificar correctamente la relación que se pretende demostrar, sino que la traslada al programa que se ejecuta. La presente sección describe el origen y los componentes de SP1, la forma de escribir programas y de definir sus valores públicos, el proceso que conduce desde el código fuente hasta una prueba verificable en Ethereum y las limitaciones relevantes para un puente entre cadenas. La descripción corresponde a la serie 6 de SP1, en particular a las versiones 6.1.0 y 6.2.2 que resuelven las dependencias del proyecto, ya que esta serie introdujo cambios de arquitectura respecto de las anteriores.

3.4.1. Origen y propósito

SP1 es una máquina virtual de conocimiento cero de código abierto, desarrollada por Succinct Labs, que permite demostrar la ejecución correcta de programas compilados para la arquitectura RISC-V [91, 92]. Su repositorio reconoce como antecedentes directos el conjunto de herramientas criptográficas Plonky3, el diseño de tablas y de búsquedas cruzadas de la máquina virtual Valida, la cadena de compilación de RISC Zero y la idea de los *builtins* de Cairo, que SP1 generaliza mediante precompilados implementados como tablas adicionales [91].

La serie 6, publicada a partir de febrero de 2026, incorporó un sistema de pruebas denominado Hypercube, basado en polinomios multilineales [92]. En esta serie, la máquina virtual implementa el conjunto de instrucciones RV64IM, es decir, la base entera de 64 bits de RISC-V junto con la extensión M de multiplicación y división [93, 92], y los programas se compilan para el destino `riscv64im-succinct-zkvm-elf` [91]. El conjunto no incluye instrucciones de punto flotante, de modo que esas operaciones deben emularse mediante software. Las versiones anteriores empleaban un conjunto de instrucciones de 32 bits, y la documentación advierte que las cadenas de compilación de 32 y 64 bits son incompatibles entre sí [92]. Los valores con los que opera el sistema de pruebas pertenecen al cuerpo primo KoalaBear, de módulo $p = 2^{31} - 2^{24} + 1$, y a una extensión de grado 4 de dicho cuerpo [91].

Al igual que en el caso de Noir, conviene distinguir los componentes que intervienen. El programa se escribe en Rust, aunque el repositorio indica que puede emplearse cualquier lenguaje compilado mediante LLVM [91]. Su compilación al conjunto de instrucciones admitido se realiza con una cadena de compilación de Rust distribuida por Succinct, que produce un ejecutable en formato ELF. La máquina virtual ejecuta ese archivo y registra los datos necesarios para demostrar su ejecución. El sistema de pruebas, formado por el protocolo Hypercube, un mecanismo de recursión y una etapa final que traduce la prueba a Groth16 o PLONK, genera la prueba a partir de esos datos. Por último, la verificación puede realizarse con el kit de desarrollo de SP1, escrito en Rust, o mediante contratos verificadores en Solidity publicados por Succinct [91, 94].

La reutilización de código constituye el principal atractivo de este enfoque. El repositorio indica que los programas pueden utilizar la biblioteca estándar de Rust y que la mayoría de las bibliotecas de su ecosistema deberían ser compatibles [91]. Los programas del puente utilizan,

por ejemplo, bibliotecas existentes para SHA-256, Keccak-256, aritmética sobre la curva BLS12-381 y codificación según el ABI de Solidity. Esta compatibilidad tiene, sin embargo, límites. El programa se ejecuta sin sistema operativo, por lo que solo dispone de las llamadas al sistema que define la propia máquina virtual, como la lectura de entradas, la publicación de valores, la finalización de la ejecución y los precompilados descritos en la Sección 3.4.3 [91]. Las bibliotecas que dependen del acceso a archivos, a la red o a otros servicios del sistema operativo no pueden utilizarse sin adaptación, y toda dependencia debe poder compilarse para el destino de 64 bits mencionado. Que una biblioteca compile, además, no implica que su costo de prueba resulte razonable, cuestión que se retoma en la Sección 3.4.4.

SP1 debe distinguirse, asimismo, de la infraestructura de generación de pruebas que ofrece Succinct. El kit de desarrollo permite generar las pruebas localmente, en procesador o en GPU, y también delegarlas en una red de probadores remota, funcionalidad que requiere habilitar una característica específica del kit y credenciales del servicio [91]. Los manifiestos del proyecto no habilitan esa característica, por lo que su implementación genera las pruebas de forma local.

El interés de SP1 para el presente trabajo reside en que materializa el enfoque basado en la ejecución de programas que la hipótesis contrapone al de Noir. La lógica de verificación de un puente, que incluye la decodificación de estructuras RLP, el cálculo de hashes y la verificación de firmas, puede escribirse como un programa Rust convencional que reutiliza bibliotecas existentes, mientras que la máquina virtual asume su representación como sistema de restricciones. Las consecuencias de este enfoque sobre los tiempos de prueba no se anticipan en este capítulo, sino que forman parte de la comparación del Capítulo 4.

3.4.2. Programas, entradas y valores públicos

Una aplicación de SP1 se compone de dos programas. El **programa invitado** (*guest*) es el que se compila para la máquina virtual y cuya ejecución se demuestra. El **programa anfitrión** (*host*) es un programa Rust convencional que se ejecuta fuera de la máquina virtual y utiliza el kit de desarrollo para preparar las entradas, solicitar la ejecución o la generación de la prueba y procesar el resultado [91, 92]. La prueba respalda únicamente la ejecución del programa invitado. Las operaciones del anfitrión, como la obtención de datos de un nodo de la cadena de origen, su decodificación o la construcción de las entradas, permanecen fuera de la ejecución demostrada, por lo que, si la corrección de esos datos es relevante, el programa invitado debe comprobarla por sí mismo.

El anfitrión escribe las entradas en un objeto `SP1Stdin`, que las serializa, y el programa invitado las lee mediante la función `sp1_zkvm::io::read`, que las deserializa, o mediante `read_vec`, que devuelve los bytes sin procesar [91]. La documentación del código advierte que este flujo de entrada está controlado por el probador [91]. Las entradas cumplen, por lo tanto, el papel del testigo de la Sección 3.1.1, ya que nada garantiza que correspondan a datos reales salvo las comprobaciones que el propio programa realice sobre ellas.

Las condiciones se comprueban con las construcciones ordinarias de Rust, como la macro

`assert!` o el tratamiento de errores que interrumpe la ejecución. Una comprobación fallida provoca la terminación del programa con un código de salida distinto de cero, y una ejecución que termina de ese modo no puede dar lugar a una prueba que los verificadores acepten. La función de verificación del kit comprueba por defecto que la ejecución haya terminado con éxito, y el contrato verificador de la versión utilizada rechaza cualquier código de salida distinto de cero [91, 94].

Los valores públicos se publican mediante `sp1_zkvm::io::commit`, que serializa un valor, o mediante `commit_slice`, que añade bytes sin procesar [91]. El conjunto de bytes publicados forma parte del enunciado, se entrega al verificador junto con la prueba y, en el contrato verificador, interviene a través de su hash SHA-256 [94]. Su codificación debe acordarse con quien lo consume, de modo que un contrato en Solidity, por ejemplo, necesita una codificación que pueda decodificar sin ambigüedad, como la del ABI de Ethereum.

El programa cuya ejecución se demuestra se identifica mediante su clave de verificación, que el kit deriva de forma determinista del ejecutable ELF durante la etapa de preparación [91]. En el contrato verificador, esa clave se representa mediante un valor de 32 bytes que se proporciona junto con los valores públicos y la prueba [94]. Quien verifica debe fijar el valor esperado, ya que, si no lo comprobara, aceptaría también una prueba válida de otro programa que publicara valores con el mismo formato.

```
#![no_main]
sp1_zkvm::entrypoint!(main);

pub fn main() {
    // Entrada privada: la suministra el probador.
    let w = sp1_zkvm::io::read::<u64>();
    // Aritmetica comprobada: un desbordamiento detiene la ejecucion.
    let y = w
        .checked_mul(w)
        .and_then(|w2| w2.checked_mul(w))
        .and_then(|w3| w3.checked_add(w))
        .and_then(|s| s.checked_add(5))
        .expect("w fuera del dominio admitido");
    // Valor publico: pasa a formar parte del enunciado.
    sp1_zkvm::io::commit(&y);
}
```

Figura 3.8: Programa invitado de SP1 que calcula $y = w^3 + w + 5$ con aritmética entera comprobada.

Los datos que el programa lee y no publica no se entregan al verificador, pero esto no basta para afirmar que permanezcan secretos. Por una parte, los valores públicos pueden permitir deducirlos. Por otra, la propiedad de conocimiento cero depende del modo de prueba. Según el modelo de seguridad de SP1, las pruebas Groth16 y PLONK, generadas con la biblioteca gnark, son de conocimiento cero, mientras que las pruebas de núcleo y las comprimidas no satisfacen actualmente esa propiedad [95]. En concordancia con ello, el repositorio de la versión

utilizada incluye un componente experimental y no auditado destinado a añadir conocimiento cero a los protocolos multilineales en los que se basan estas últimas [91]. En el diseño del puente, la prueba que se verifica en la cadena de destino es una prueba Groth16, aunque la propiedad de conocimiento cero no resulta necesaria en una aplicación cuyos datos son públicos (Sección 3.1.1).

La Figura 3.8 muestra un programa invitado que retoma la relación $y = w^3 + w + 5$ de los ejemplos de Circom y Noir. A diferencia de estos, que evalúan la relación en $\mathbb{F}_p$, el programa emplea enteros de 64 bits sin signo, cuya aritmética no es la de un cuerpo finito. Para evitar cualquier ambigüedad, cada operación se realiza de forma comprobada, de modo que un resultado que exceda el rango de esos enteros interrumpe la ejecución en lugar de reducirse módulo 2^{64}. El programa acepta, por lo tanto, únicamente los valores de w para los que $w^3+w+5 < 2^{64}$, es decir, $0 \leq w \leq 2\,642\,245$. En ese dominio, el resultado coincide con el de los ejemplos anteriores, ya que el valor entero es menor que el módulo de 254 bits que estos emplean, pero fuera de él la relación no es equivalente, puesto que aquellos aceptan cualquier elemento del cuerpo.

El programa lee w como entrada privada, calcula y con aritmética comprobada y publica y. La Figura 3.9 muestra el código anfitrión imprescindible para este flujo, que escribe w en la entrada, deriva las claves a partir del ejecutable, genera y verifica una prueba y lee el valor público. La prueba respalda que el programa identificado por la clave de verificación, ejecutado con alguna entrada, terminó correctamente y publicó y. El verificador recibe la clave de verificación, los valores públicos y la prueba, pero no w. El ejemplo no ilustra, sin embargo, ninguna forma de confidencialidad, ya que la función es estrictamente creciente en el dominio admitido y el valor público y determina, por lo tanto, un único w que cualquiera puede calcular.

```
use sp1_sdk::{
    blocking::{ProveRequest, Prover, ProverClient},
    include_elf, Elf, ProvingKey, SP1Stdin,
};

const ELF: Elf = include_elf!("cubic-program");

fn main() {
    let client = ProverClient::from_env();
    let mut stdin = SP1Stdin::new();
    stdin.write(&3u64); // w: entrada privada

    let pk = client.setup(ELF).unwrap(); // claves derivadas del ELF
    let mut proof = client.prove(&pk, stdin).compressed().run().unwrap();
    client.verify(&proof, pk.verifying_key(), None).unwrap();

    let y: u64 = proof.public_values.read(); // valor publico
    println!("y = {y}");
}
```

Figura 3.9: Programa anfitrión que prepara la entrada, genera y verifica una prueba comprimida y lee el valor público.

En una compilación y ejecución realizadas para el presente trabajo con SP1 6.2.2, el ejecutable del programa invitado resultó ser un archivo ELF para RISC-V de 64 bits. La ejecución con $w = 3$ publicó $y = 35$ tras 4802 instrucciones, cifra que incluye la lectura y deserialización de la entrada, mientras que con $w = 3\,000\,000$ la comprobación de desbordamiento interrumpió el programa, que terminó con código de salida 1. El programa anfitrión de la Figura 3.9 generó y verificó correctamente una prueba comprimida para $w = 3$.

3.4.3. Compilación, ejecución, generación y verificación de la prueba

La Figura 3.10 representa el recorrido desde el código fuente hasta una prueba verificable. La primera etapa es la compilación del programa invitado, que el kit realiza con la cadena de compilación de Succinct, ya sea desde el script de compilación del anfitrión o mediante una orden específica, y que produce un ejecutable ELF para RISC-V de 64 bits [91]. El anfitrión incorpora ese ejecutable en su propio binario. A partir de él, la etapa de preparación deriva de forma determinista la clave de prueba y la clave de verificación, sin emplear ningún valor secreto [91]. Esta preparación no debe confundirse con la configuración confiable de algunos SNARKs descrita en la Sección 3.1.5, ya que solo depende del programa. Las entradas, por su parte, se preparan en el anfitrión y se codifican al escribirlas en el objeto de entrada, con independencia de la preparación de las claves.

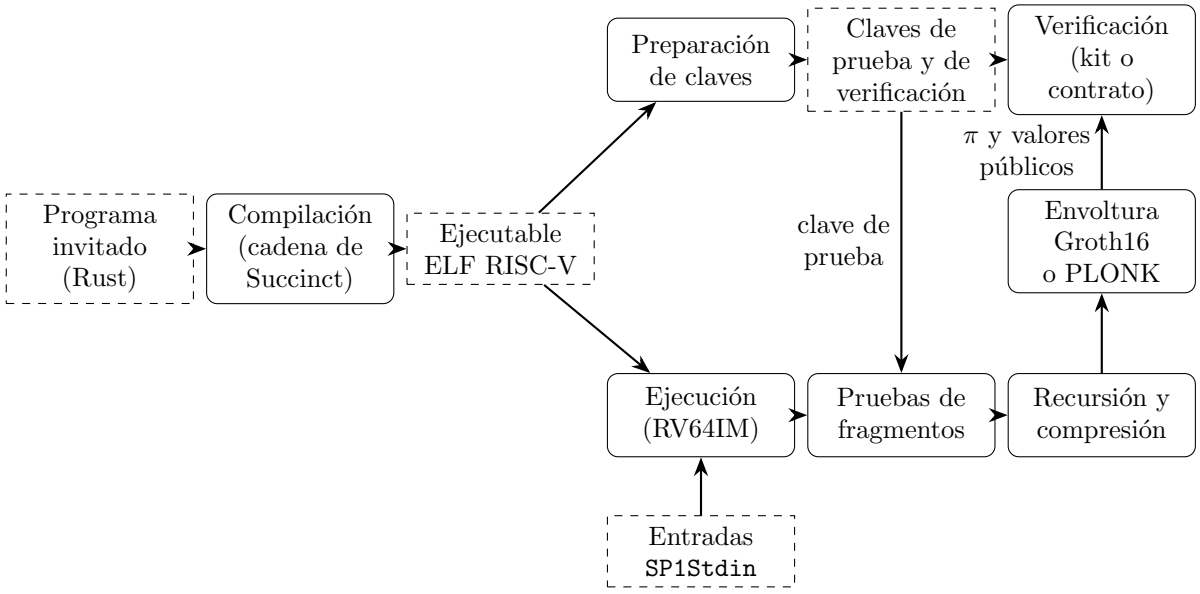


Figura 3.10: Proceso de compilación, ejecución, generación y verificación de una prueba con SP1.

En la etapa de ejecución, la máquina virtual ejecuta el programa sobre las entradas. El kit permite ejecutar sin generar ninguna prueba, en cuyo caso devuelve los valores públicos y un informe con el número de instrucciones ejecutadas y de llamadas al sistema [91]. Cuando se genera una prueba, la ejecución se divide además en **fragmentos** (*shards*), de aproximadamente 2^{22} instrucciones cada uno según la documentación [92], y los eventos registrados en cada

fragmento dan lugar a la traza que se demuestra. La división acota los recursos necesarios para probar cada fragmento y permite procesarlos de forma independiente.

La traza de cada fragmento se organiza en tablas, en las que cada tabla corresponde a un tipo de instrucción o a un componente especializado y sus columnas contienen los operandos necesarios para comprobar la operación [92]. Dado que una palabra de 64 bits no cabe en un elemento del cuerpo KoalaBear, cada palabra se representa mediante cuatro valores de 16 bits [91]. Las restricciones que relacionan las columnas de una misma tabla se comprueban mediante un protocolo de tipo *sumcheck*, denominado *zerocheck*, y las **interacciones** entre tablas, por ejemplo entre la tabla que registra una instrucción y las que verifican sus operandos, se comprueban mediante LogUp-GKR, un argumento de derivadas logarítmicas que verifica que los valores enviados y recibidos coinciden con sus multiplicidades [92, 96]. La coherencia de la memoria entre fragmentos se comprueba mediante un hash de multiconjuntos, que transforma cada lectura y escritura en un punto de una curva elíptica mediante la función Poseidon2, de modo que la igualdad entre los conjuntos de escrituras y de lecturas se reduce a la igualdad de sus hashes [92]. Por último, el probador se compromete con toda la traza del fragmento como un único polinomio multilineal, mediante un esquema de compromiso denominado *jagged*, que admite tablas de alturas distintas [97], construido sobre el esquema BaseFold [98, 91]. Este esquema se basa en funciones de hash, por lo que esta parte del sistema no requiere configuración confiable, y la interacción se elimina mediante la transformación de Fiat-Shamir (Sección 3.1.3).

Las pruebas de los fragmentos constituyen la prueba de núcleo, cuyo tamaño total es proporcional a la extensión de la ejecución [92]. Para obtener una prueba de tamaño constante, el mecanismo de recursión verifica las pruebas de los fragmentos dentro de la propia máquina de pruebas y las combina en forma de árbol hasta obtener una única prueba *comprimida* [92]. Esta prueba puede verificarse con el kit, pero no está pensada para su verificación en Ethereum. Para ello, una etapa adicional cambia la función de hash empleada por la recursión y, finalmente, una etapa de envoltura produce una prueba Groth16 o PLONK sobre la curva BN254 [92, 91]. Esta última etapa utiliza la biblioteca gnark [99] y puede ejecutarse mediante una imagen de Docker o, habilitando una característica del kit, de forma nativa [91]. Según la documentación, las claves del circuito de Groth16 proceden de la ceremonia Ignition, complementada con contribuciones del equipo de Succinct, mientras que la variante PLONK reutiliza la cadena de referencia universal de dicha ceremonia, y las pruebas resultantes ocupan unos 260 y 868 bytes, respectivamente [92].

La verificación puede realizarse con el kit, que admite las pruebas de cualquiera de los modos y comprueba por defecto que la ejecución terminó con éxito, o en Ethereum, con las pruebas Groth16 o PLONK [91, 92]. En este último caso, el contrato recibe la clave de verificación del programa, los valores públicos y la prueba. Los primeros cuatro bytes de la prueba identifican la versión del verificador, lo que permite a un contrato de enrutamiento dirigirla al verificador correspondiente. El contrato de Groth16 de la versión de circuito 6.1.0, que corresponde a las versiones utilizadas en el proyecto, comprueba la prueba con cinco entradas públicas, que son la clave del programa, el hash SHA-256 de los valores públicos truncado a 253 bits para que

quepa en el cuerpo de BN254, el código de salida, que debe ser cero, una raíz que identifica las claves del mecanismo de recursión y un valor de uso único [94].

La máquina virtual ofrece, además, **precompilados**, es decir, tablas especializadas que demuestran operaciones costosas sin descomponerlas en instrucciones RISC-V. El programa los invoca mediante llamadas al sistema y, en la versión utilizada, abarcan la permutación de Keccak, la expansión y la compresión de SHA-256, operaciones sobre las curvas secp256k1, secp256r1, ed25519, BN254 y BLS12-381, aritmética sobre los cuerpos de estas curvas y sobre enteros grandes, la función Poseidon2 y la verificación de pruebas de SP1 [91]. Su uso habitual no requiere escribir estas llamadas, sino sustituir bibliotecas criptográficas comunes por versiones modificadas que dirigen automáticamente sus operaciones hacia los precompilados [92]. Conviene distinguir, por lo tanto, el precompilado, que forma parte del sistema de pruebas, de la biblioteca modificada que lo invoca, y de las operaciones del anfitrión, que quedan fuera de la prueba. Si una dependencia no utiliza la versión modificada, la operación se ejecuta como instrucciones ordinarias y su costo aumenta considerablemente [92].

Esta descripción permite identificar intervalos de tiempo de naturaleza distinta. La compilación del programa invitado, la preparación de las claves y el despliegue del contrato verificador constituyen una preparación que se realiza una vez por programa. La construcción del cliente del kit y la obtención de los artefactos que necesita la etapa de envoltura son costos de puesta en marcha. En cada operación se repiten, en cambio, la preparación de las entradas en el anfitrión, que no forma parte de la prueba, la ejecución, la generación de las pruebas de los fragmentos, su compresión y, cuando se requiere la verificación en Ethereum, la envoltura final, seguidas de la verificación local y de la verificación en la cadena de destino, cuyo costo se expresa en gas y cuya duración incluye la espera hasta la inclusión de la transacción. El modo de ejecución simulado del kit, que produce valores públicos reales pero una prueba vacía, no genera ninguna prueba y no debe confundirse con estas etapas. Ninguna de ellas tiene necesariamente un equivalente directo en Noir, por lo que su correspondencia se establece en la metodología del Capítulo 4.

3.4.4. Limitaciones y consideraciones de seguridad

La primera limitación se refiere, como en Noir, a la diferencia entre la expresividad del lenguaje y la corrección de la especificación. La máquina virtual garantiza que el programa se ejecutó correctamente, pero no que el programa compruebe lo que se pretende. Dado que las entradas están controladas por el probador, un valor recibido como entrada solo merece confianza en la medida en que algo lo autentique. Recibir como entrada la raíz contra la que se verifica la inclusión de una transacción no implica, por sí mismo, que esa raíz sea arbitraria. El problema surge cuando el programa no la autentica, por ejemplo derivándola de un encabezado cuya autenticidad también comprueba, y cuando la aplicación verificadora tampoco la contrasta con un valor de confianza mediante comprobaciones vinculadas a la prueba, como la comparación de un valor público de la prueba con un encabezado previamente aceptado. En ese caso, la prueba establece únicamente la inclusión respecto de una raíz elegida por el probador.

Las distinciones de la Sección 3.2.4 entre inclusión, autenticidad y finalidad del bloque y éxito de la operación siguen siendo responsabilidad del programa y de la aplicación que lo utiliza.

La segunda consideración es la identificación del programa. Las modificaciones del código, de sus dependencias o de la cadena de compilación pueden alterar el ejecutable y, en consecuencia, la clave de verificación que se deriva de él. Por ese motivo, el verificador debe fijar la clave correspondiente al programa autorizado y comprobarla en cada verificación, y toda nueva compilación exige confirmar que la clave resultante coincide con la fijada o actualizar esta última de manera deliberada. Para que terceros puedan reproducir esa clave, la herramienta de compilación ofrece la opción de compilar dentro de un contenedor de Docker con el fin de obtener ejecutables reproducibles [91]. La verificación debe exigir, además, que la ejecución haya terminado con éxito, condición que el kit y el contrato verificador comprueban por defecto.

SP1 admite también cálculo no restringido. El repositorio de la versión utilizada define una macro que ejecuta un bloque de código fuera de las restricciones, descarta al finalizar cualquier cambio en el estado de la máquina virtual y solo permite transmitir datos al programa a través de un flujo de sugerencias que este lee después como una entrada más [91]. Su papel es análogo al de las funciones no restringidas de Noir y al operador `<--` de Circom, ya que los datos así obtenidos solo quedan respaldados por las comprobaciones que el programa realice sobre ellos.

Los precompilados introducen supuestos propios. La documentación advierte que los precompilados de curvas elípticas asumen que sus entradas son puntos válidos de la curva, sin comprobarlo, que los valores de los cuerpos deben estar en su representación canónica y que ciertos casos, como el uso del precompilado de suma para duplicar un punto o la presencia del punto en el infinito, deben tratarse por separado para no producir pruebas imposibles de verificar [92]. Las bibliotecas modificadas deben corresponder, además, a la versión de SP1 utilizada, y una sustitución que no se aplique hace que la operación vuelva a ejecutarse como instrucciones ordinarias sin que se produzca ningún error [92].

El costo de la generalidad constituye la limitación central del enfoque. Cada instrucción ejecutada ocupa filas en las tablas de la traza, cada palabra de 64 bits se representa mediante varios elementos del cuerpo y las operaciones criptográficas que no utilizan precompilados se ejecutan como secuencias de instrucciones ordinarias. Los hashes binarios y la aritmética sobre la curva BLS12-381, presentes en la verificación de un puente, pueden dominar, en consecuencia, el número de instrucciones de un programa. El costo final depende, además, de la versión, de la configuración de los fragmentos, del hardware disponible, que incluye la posibilidad de emplear GPU, y del modo de prueba solicitado, ya que la envoltura en Groth16 o PLONK añade una etapa a la generación. Por ese motivo, cualquier medición debe asociarse a la versión, a la configuración y al hardware con que se obtuvo.

Por último, la privacidad de las entradas depende del modo de prueba y de quién la genera. Como se indicó en la Sección 3.4.2, solo las pruebas Groth16 y PLONK son de conocimiento cero, y esa propiedad protege las entradas frente al verificador, pero no frente a quien genera la prueba, que necesariamente las conoce. Por ese motivo, la delegación de la prueba en un servicio remoto implica, salvo en modalidades específicas que la documentación describe para ese fin,

que el servicio recibe las entradas del programa [92]. En un puente cuyos datos son públicos esta limitación carece de consecuencias, pero debe tenerse en cuenta en otras aplicaciones.

Con SP1 concluye la exposición de los dos enfoques que el presente trabajo compara, el basado en circuitos específicos, representado por Noir, y el basado en la ejecución de programas sobre una máquina virtual, representado por SP1. La Sección 3.5 reseña brevemente otras herramientas del ecosistema, y el Capítulo 4 describe la implementación del puente con ambas tecnologías y la metodología con que se comparan.

3.5. Otros entornos y herramientas del ecosistema

Noir y SP1 no son las únicas herramientas para expresar y demostrar computaciones. La presente sección reseña otras que ocupan posiciones distintas en el ecosistema, con el fin de situar la elección del presente trabajo, y la Tabla 3.2 resume su relación con el caso de estudio.

RISC Zero y Cairo pertenecen, como SP1, al enfoque basado en la ejecución de programas descrito en la Sección 3.1.4. RISC Zero es una máquina virtual de conocimiento cero que emula una computadora RISC-V y admite programas escritos en Rust, C y C++. La ejecución se demuestra mediante pruebas STARK por segmentos, que un circuito de recursión agrega en una única prueba STARK sucinta, y esta puede envolverse opcionalmente en una prueba Groth16 [100, 101]. Cairo es, a la vez, una arquitectura de procesador diseñada para que su ejecución pueda demostrarse de forma eficiente mediante STARKs y un lenguaje de programación cuya principal aplicación actual es Starknet, una solución de escalabilidad de capa 2 de Ethereum [102, 103].

Halo2 y Plonky3 se sitúan en otro nivel, ya que no son lenguajes ni máquinas virtuales, sino bibliotecas para construir sistemas de prueba. Halo2 emplea una aritmetización derivada de PLONK [104], por lo que se encuadra en el enfoque basado en circuitos de Noir. Sus garantías dependen de la variante. La implementación de Zcash utiliza un compromiso basado en un argumento de producto interno y desciende de Halo, presentado como el primer esquema práctico de composición recursiva de pruebas sin configuración confiable [104, 105], mientras que la variante de Privacy Scaling Explorations sustituyó ese compromiso por KZG para abaratar la verificación en Ethereum [106], lo que reintroduce la configuración confiable descrita en la Sección 3.1.5. Plonky3, mantenido por el equipo Polygon Zero, ofrece primitivas, como compromisos basados en FRI y cuerpos finitos pequeños, que se emplean principalmente para construir máquinas virtuales basadas en STARKs [107], y el repositorio de SP1 lo reconoce como antecedente (Sección 3.4.1).

El servicio vigente de Aligned recibe pruebas ya generadas por distintos usuarios y las combina en una única prueba recursiva que se verifica en Ethereum, de modo que el costo de la verificación se reparte entre todas ellas [108]. Su capa de verificación anterior, hoy obsoleta, delegaba en cambio la verificación en operadores de EigenLayer que la reejecutaban fuera de la cadena y cuyas firmas se agregaban [109], por lo que el resultado dependía de esos operadores y no únicamente de la prueba.

Succinct, por último, no constituye una herramienta adicional, sino la organización que desarrolla SP1. Impulsa, además, una red de probadores en la que las aplicaciones solicitan pruebas de programas de SP1 y los probadores compiten por generarlas [110]. Esa red es una forma de obtener las pruebas, no un sistema de pruebas distinto, y el prototipo no la utiliza (Sección 3.4.1).

Tabla 3.2: Herramientas y entornos del ecosistema de pruebas de conocimiento cero y su relación con el caso de estudio.

Herramienta o entorno	**Categoría**	**Función principal**	**Relación con el caso de estudio**
Noir	Lenguaje de dominio específico	Describir circuitos que un backend demuestra	Implementación analizada (Sección 3.3)
SP1	Máquina virtual de conocimiento cero	Demostrar la ejecución de programas en Rust	Implementación analizada (Sección 3.4)
RISC Zero	Máquina virtual de conocimiento cero	Demostrar programas en Rust, C y C++	Alternativa dentro del enfoque de SP1
Cairo	Lenguaje y arquitectura de procesador	Programas demostrados con STARKs	Alternativa dentro del enfoque de ejecución, con instrucciones propias
Halo2	Biblioteca de sistemas de prueba	Circuitos de tipo PLONK con IPA o KZG, según la variante	Alternativa dentro del enfoque de circuitos de Noir
Plonky3	Biblioteca de primitivas criptográficas	Construir probadores basados en STARKs	Antecedente reconocido de SP1
Aligned	Servicio de agregación de pruebas	Combinar pruebas ya generadas en una prueba agregada verificable en Ethereum	Alternativa para la verificación en cadena de pruebas generadas con otras herramientas
Red de probadores de Succinct	Red de generación de pruebas	Generar pruebas de programas de SP1 a solicitud	Modalidad de SP1 no utilizada por el prototipo

Las herramientas reseñadas en esta sección, con excepción de Noir y SP1, que figuran en la tabla como referencia, no se incorporaron a la evaluación experimental del presente trabajo, que se concentra en estas dos. El Capítulo 4 describe la implementación del puente con Noir y SP1 y la metodología con que se comparan ambos enfoques.

CAPÍTULO 4

ZKBRIDGE

El Capítulo 3 presentó los sistemas de prueba y las dos herramientas con que el presente trabajo aborda el problema de los puentes descrito en la Sección 2.5. Este capítulo traslada esos fundamentos a un puente concreto, en el que la cadena de destino acuña un activo solo cuando verifica pruebas de que una transacción que invoca la función de bloqueo fue incluida y se ejecutó con éxito en un bloque de ejecución de la cadena de origen cuyo hash está vinculado al encabezado atestiguado por su comité de sincronización. Como se precisa en la Sección 4.1.3, estas pruebas no acreditan que el bloque haya alcanzado la finalidad.

La Sección 4.1 describe la arquitectura común del puente, el flujo de una transferencia y las condiciones bajo las que se autoriza la acuñación. Las secciones siguientes reseñan zkBridge, de Berkeley RDI, como referencia de diseño, presentan el diseño y la implementación de las pruebas en Noir y en SP1 y exponen la metodología con que ambas se comparan y los resultados obtenidos.

4.1. Implementación de un puente lock-and-mint

El puente transfiere el token `GuaraniToken` desde una cadena de origen hacia una cadena de destino mediante el esquema *lock-and-mint*. El presente trabajo implementa únicamente el flujo de ida, de modo que la operación inversa, de quema y liberación, queda fuera de su alcance. La descripción que sigue corresponde a la documentación y al código del repositorio del puente [111].

4.1.1. Arquitectura y componentes

Conviene distinguir dos variantes del esquema. En el puente de referencia descrito en un trabajo previo de la misma institución, un *relayer* autorizado observa el bloqueo en la cadena

de origen e invoca la acuñación en la de destino, cuyo contrato receptor comprueba únicamente que la llamada proviene de ese *relayer* y que el identificador del bloqueo no se procesó antes, por lo que la corrección de cada acuñación depende por completo de su honestidad [112, 113]. En la variante que desarrolla el presente trabajo, el contrato de destino acuña solo tras verificar dos pruebas, y el *relayer* deja de contar con un permiso exclusivo, ya que cualquier parte puede presentar esas pruebas y solicitar la acuñación. Las garantías de una variante no se trasladan a la otra, y lo que sigue se refiere exclusivamente a la segunda.

La arquitectura se compone de los siguientes elementos.

- `GuaraniToken`, un token ERC-20 con símbolo GUA desplegado en ambas cadenas. En la de origen se emite con un suministro inicial, y en la de destino se despliega sin suministro y solo el contrato receptor recibe el rol que autoriza la acuñación.

- `Sender`, el contrato de bloqueo en la cadena de origen. Su función `lock(recipientL2, amount)` exige un destinatario distinto de la dirección cero, un monto positivo y una autorización de gasto suficiente, transfiere los tokens a su custodia, registra el bloqueo y emite el evento `Locked`. Este contrato y el token se conservan sin cambios respecto del prototipo de ese trabajo previo, del que deriva el repositorio según su historial de versiones [112, 111].

- `BeaconLightClient`, en la cadena de destino, que verifica la prueba de consenso y registra como anclado el hash del bloque de ejecución que esta certifica. Su propietario fija los parámetros de política que la prueba no puede validar por sí sola, que son el dominio de firma, el compromiso del comité de sincronización vigente y la posición del hash del bloque en el encabezado de la cadena de baliza.

- `ZKReceiver`, en la cadena de destino, que verifica la prueba de inclusión, que abarca la transacción de bloqueo y su recibo, y acuña los tokens. Sus funciones de acuñación son públicas, de modo que cualquier parte que disponga de pruebas válidas puede invocarlas.

- Un contrato verificador, en el que ambos contratos de destino delegan la comprobación criptográfica de las pruebas.

- El *relayer*, un proceso en Node.js que se ejecuta fuera de las cadenas, detecta los bloqueos, obtiene los datos de la cadena de origen, invoca los programas que generan las pruebas y envía la operación a la cadena de destino. Los contratos no le reconocen ningún permiso exclusivo ni aceptan como parámetro de la acuñación ningún dato suyo que no esté comprometido por una prueba, aunque su disponibilidad condiciona que las transferencias se completen.

- Una interfaz web que permite al usuario conectar su billetera, autorizar el gasto, invocar el bloqueo y seguir los eventos de bloqueo y de acuñación.

Las pruebas establecen dos enunciados enlazados por el hash del bloque de ejecución. La prueba de consenso establece que el comité de sincronización firmó un encabezado de la cadena de baliza del que se deriva ese hash, y la prueba de inclusión, que la transacción que invoca `lock` pertenece al bloque cuyo encabezado produce el mismo hash y que su recibo, comprometido en ese mismo encabezado, registra una ejecución exitosa. Los contratos, el token, el *relayer*, la interfaz y estos dos enunciados son comunes a las implementaciones en Noir y en SP1. La forma en que cada herramienta expresa y demuestra los enunciados, la codificación de sus valores públicos y el contrato verificador que corresponde a cada sistema de prueba se describen en las secciones dedicadas a cada una de ellas.

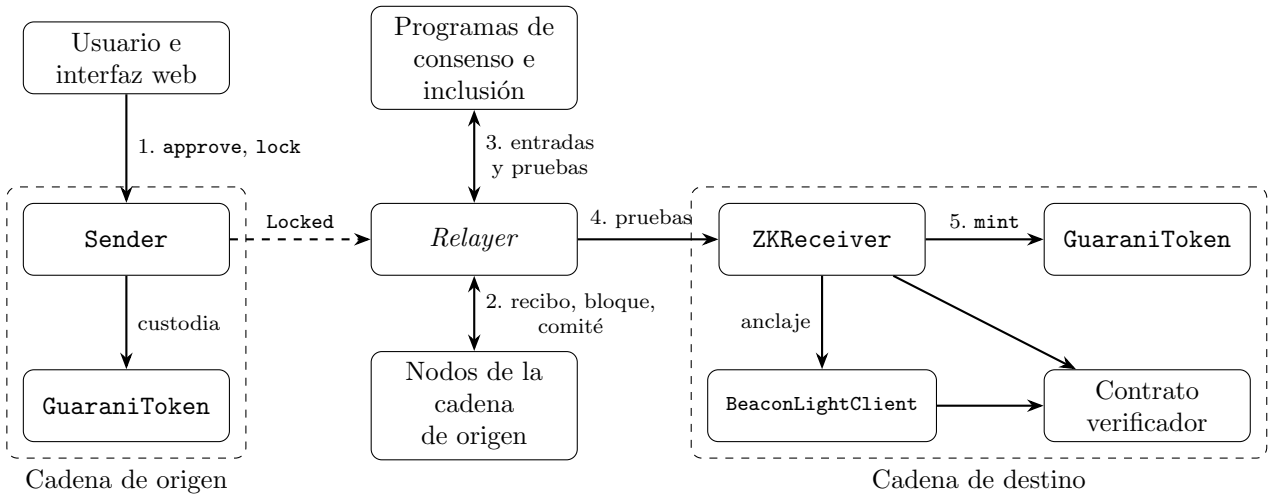


Figura 4.1: Arquitectura del puente y recorrido de una transferencia de ida. Las líneas discontinuas indican eventos y los números, las etapas descritas en el texto.

4.1.2. Flujo operativo de la transferencia

La Figura 4.1 resume la arquitectura y el recorrido de una transferencia de ida, que se desarrolla en cinco etapas.

1. **Solicitud y bloqueo.** El usuario autoriza al contrato `Sender` a transferir sus tokens e invoca `lock(recipientL2, amount)`. Si las comprobaciones del contrato se cumplen, los tokens quedan en custodia y se emite el evento `Locked`. Los datos de la llamada, es decir, el selector de la función, el destinatario y el monto, son los que el destino utilizará más adelante.

2. **Detección y obtención de datos.** El *relayer* detecta el evento, ya sea por suscripción o por consulta periódica, y obtiene el recibo de la transacción para identificar el bloque que la contiene. Con un nodo de la cadena de baliza obtiene los datos del comité de sincronización que firmó ese bloque, y con un nodo de ejecución reconstruye el encabezado del bloque y obtiene todas sus transacciones y sus recibos. El *relayer* comprueba que el hash del encabezado reconstruido y el de cada transacción coincidan con los que informa

el nodo y que cada recibo corresponda a la transacción de su posición, y detiene el proceso si alguna comprobación falla o si el recibo del bloqueo registra una ejecución revertida. Los datos del evento solo sirven para iniciar el proceso y no se utilizan para acuñar.

3. **Generación de las pruebas.** El *relayer* invoca el programa de consenso con el encabezado de la cadena de baliza, las claves públicas del comité, el vector de participación, la firma agregada y la rama de Merkle que conduce al hash del bloque de ejecución, y el programa de inclusión con el encabezado de ejecución, las transacciones y los recibos del bloque y la posición de la transacción de bloqueo. Cada programa produce una prueba y sus valores públicos.

4. **Envío a destino.** Si el bloque todavía no está anclado, el *relayer* envía ambas pruebas en una única transacción a `mintRemoteWithHeader` , junto con los datos de la llamada de bloqueo, que no forman parte de los valores públicos. Si el bloque ya fue anclado por otra operación, envía solo la prueba de inclusión a `mintRemote`. En ambos casos espera después el evento `Minted` para confirmar la operación.

5. **Comprobaciones y acuñación.** Las comprobaciones dependen de la ruta utilizada en la etapa anterior. Con `mintRemoteWithHeader` , el contrato `ZKReceiver` solicita primero a `BeaconLightClient` que verifique la prueba de consenso y ancle el bloque que esta certifica. Con `mintRemote` omite ese paso y se apoya en el anclaje registrado por una operación anterior, sin volver a verificar la prueba que lo produjo. En ambos casos verifica después la prueba de inclusión, exige que el bloque de la transacción probada figure como anclado y, si se cumplen las demás condiciones de la Sección 4.1.3, acuña el monto a favor del destinatario y emite `Minted`. Cualquier comprobación fallida revierte los cambios realizados dentro de la misma transacción, de modo que con `mintRemoteWithHeader` el anclaje no persiste si la acuñación falla. Un anclaje registrado en una transacción anterior, en cambio, se conserva aunque falle una solicitud de acuñación posterior.

4.1.3. Condiciones de aceptación y modelo de confianza

`BeaconLightClient` solo ancla un bloque cuando el contrato verificador acepta la prueba de consenso para la clave del programa correspondiente, la prueba declara válida la firma, condición que el programa solo establece si la firma agregada y la rama de Merkle son correctas y participaron en ella más de dos tercios de los miembros del comité, y el dominio, la posición del hash y el compromiso del comité coinciden con los fijados por el propietario. `ZKReceiver`, por su parte, solo acuña cuando el verificador acepta la prueba de inclusión para la clave de su programa, la prueba declara verificada la inclusión, la raíz de las transacciones proviene del encabezado y la transacción pudo interpretarse, el recibo de la misma posición pertenece al árbol de recibos de ese encabezado y registra una ejecución exitosa, el hash de ese encabezado está anclado, la transacción se dirigió al contrato `Sender`, los datos de la llamada coinciden en longitud y en hash con los comprometidos por la prueba y corresponden a una llamada a `lock`

con una dirección bien formada y un monto positivo, y el par formado por el hash del bloque y la posición de la transacción no se procesó antes.

Estas condiciones respaldan algunas de las afirmaciones que distingue la Sección 3.2.4, pero no todas. La inclusión de la transacción y la vinculación de la raíz con un encabezado concreto quedan establecidas por la prueba de inclusión, que establece además, mediante el código de estado del recibo, que la llamada de nivel superior de la transacción terminó sin revertir [19]. Como la transacción se dirige directamente a `Sender`, cuya función `lock` revierte si alguna de sus comprobaciones falla, y el token, basado en la implementación ERC-20 de OpenZeppelin, revierte ante una autorización o un saldo insuficientes en lugar de devolver un valor de error, una ejecución exitosa implica que los tokens quedaron en custodia y que se emitió el evento `Locked` [111, 114]. El destinatario y el monto se extraen de los datos de la transacción probada, por lo que el *relayer* no puede alterarlos. La reutilización de una operación se impide mediante el registro de los pares ya procesados. La autenticidad del encabezado, en cambio, depende de la honestidad de la mayoría del comité de sincronización y de que el propietario de `BeaconLightClient` mantenga actualizado el compromiso del comité, que se renueva cada 256 épocas [22]. Según la documentación de la revisión examinada, la rotación del comité se actualiza mediante una operación del propietario y no mediante una prueba, y el contrato ancla cualquier bloque con una firma válida aunque no esté finalizado [111]. A ello se suman la corrección del contrato verificador y los supuestos del sistema de prueba que lo sustenta, incluida la configuración confiable cuando existe (Sección 3.1.5).

El papel del *relayer* admite, en consecuencia, varias distinciones. Los contratos de destino no le otorgan un permiso exclusivo, ya que `updateHeader`, `mintRemote` y `mintRemoteWithHeader` pueden ser invocadas por cualquier parte. Las comprobaciones de `ZKReceiver` y de los programas probados impiden, por su parte, que quien presenta las pruebas determine el bloque, la transacción, el destinatario o el monto al margen de lo que estas comprometen, o que reutilice una operación ya procesada. Exigen, además, que el bloqueo se haya ejecutado con éxito, de modo que una llamada a `lock` incluida pero revertida no permite acuñar. La detección del evento `Locked` por el *relayer* habitual es una condición de su propio funcionamiento y no una exigencia del receptor, que se apoya en el recibo probado y no en el evento. Esas comprobaciones no verifican, sin embargo, la finalidad del bloque, de modo que no distinguen un bloque firmado que no llegue a finalizarse de uno final. Finalmente, el *relayer* sigue siendo relevante para la disponibilidad del servicio, puesto que si deja de operar ninguna transferencia se completa hasta que otra parte genere y envíe las pruebas. El tratamiento de las limitaciones señaladas en la versión evaluada se discute junto con los resultados del capítulo.

4.2. zkBridge de Berkeley RDI

Una cadena de destino no puede consultar directamente el estado de otra cadena, de modo que, antes de actuar sobre un dato proveniente de ella, necesita establecer que ese dato pertenece a un bloque aceptado por el consenso de la cadena de origen. Ejecutar en un contrato las reglas

de un cliente ligero evitaría recurrir a un comité externo, pero verificar en destino las firmas y transiciones que exigen puede resultar prohibitivo. La propuesta zkBridge, de Xie et al., traslada esa ejecución fuera de la cadena y envía al contrato de destino el nuevo encabezado junto con una prueba sucinta de que satisface la regla de validación del cliente ligero [75].

Los nodos de la red de retransmisión consultan varios nodos completos de la cadena de origen, obtienen el encabezado que sucede al último registrado y demuestran que la regla de validación, aplicada al estado vigente del cliente ligero y al encabezado anterior, acepta el nuevo. El contrato actualizador (*updater contract*) comprueba que el encabezado anterior ya figure en su registro y que la prueba sea válida respecto del estado que mantiene, y solo entonces actualiza ese estado e incorpora el encabezado, que las aplicaciones consultan mediante una interfaz independiente de su lógica. La participación en la retransmisión es abierta: el contrato verifica la prueba sin necesidad de confiar en quien la presenta. Para que el puente siga incorporando encabezados, debe haber al menos un retransmisor honesto que los envíe. El análisis de los autores supone además que la cadena de origen mantiene la consistencia y sigue progresando, que su protocolo de cliente ligero es seguro y que el sistema de pruebas es sólido [75].

Aceptar un encabezado no autoriza por sí solo ninguna operación, porque el encabezado únicamente compromete el contenido del bloque mediante raíces de Merkle. En zkBridge, el contrato receptor de cada aplicación obtiene el encabezado del actualizador y verifica contra la raíz correspondiente una prueba adicional de que un dato concreto, como la inclusión de una transacción, pertenece al bloque. Que un encabezado aceptado no quede luego fuera de la cadena canónica depende, a su vez, de que el verificador del cliente ligero rechace encabezados que puedan quedar huérfanos y del criterio de confirmación con que se extrae la cadena principal del registro [75].

En su tesis de grado en la FP-UNA, Figueredo Camé y Riquelme Godoy reseñaron la propuesta de Berkeley y construyeron su prototipo a partir de una implementación abierta de zkBridge [112]. El prototipo actual presenta una separación semejante, en la que `BeaconLightClient` registra el hash del bloque de ejecución certificado por la prueba de consenso y `ZKReceiver` comprueba la inclusión y la ejecución exitosa de la transacción antes de acuñar (Sección 4.1.1). La ruta `mintRemote` ilustra esa división, ya que el receptor aprovecha un anclaje previo sin que se vuelva a presentar la prueba que lo estableció. Se trata de una semejanza de diseño y no de una equivalencia de garantías, puesto que el prototipo no encadena cada encabezado con su predecesor, recibe el compromiso del comité de su propietario y puede anclar bloques no finalizados (Sección 4.1.3). La sección siguiente aborda la expresión en Noir de los enunciados de consenso y de inclusión.

4.3. Diseño e implementación en Noir

La implementación en Noir expresa los dos enunciados presentados en la Sección 4.1.1 mediante dos programas, uno de consenso y otro de inclusión, cada uno de los cuales se compila en un circuito cuyas pruebas genera Barretenberg con el sistema UltraHonk. Los contratos de

destino, el *relayer* y el flujo de una transferencia son los de la Sección 4.1. Esta sección describe las relaciones que establece cada programa, las consecuencias de expresarlas como circuitos, la codificación de sus valores públicos, los contratos verificadores y la integración con el resto del puente. Como las relaciones son las mismas que demuestran los programas de SP1, su descripción sirve también de referencia para la sección siguiente. Las herramientas corresponden a las versiones indicadas en la Sección 3.3, Nargo 1.0.0-beta.22 y Barretenberg 5.0.0-nightly.20260522.

4.3.1. Programa de consenso

El programa de consenso recibe los cinco campos del encabezado de la cadena de baliza que firmó el comité, es decir, el *slot*, el índice del validador proponente y las raíces `parent_root`, `state_root` y `body_root` (Sección 2.2.6). Recibe, además, el dominio de firma, las claves públicas comprimidas de los 512 miembros del comité, el vector de participación, la firma agregada, el hash del bloque de ejecución que se pretende anclar y la rama SSZ que lo vincula con `body_root`, junto con su índice generalizado. Con estos datos, el programa deriva el mensaje firmado y realiza tres comprobaciones. Para derivar el mensaje, calcula la raíz del encabezado mediante la función `hash_tree_root`, que organiza sus cinco campos en un árbol binario de ocho hojas [16], y obtiene el hash SHA-256 de la concatenación de esa raíz con el dominio, que es la forma que adopta la raíz de firma de la especificación de consenso para un contenedor de dos raíces [15]. La primera comprobación suma las claves de los miembros cuyo bit de participación vale uno y verifica la Ecuación (2.4) sobre ese mensaje. La segunda verifica que la rama SSZ conduce desde el hash del bloque de ejecución hasta `body_root` y que el índice generalizado corresponde exactamente a la profundidad de la rama, condición sin la cual un nodo interno del árbol podría hacerse pasar por la hoja buscada. La tercera cuenta los firmantes y comprueba que superen los dos tercios del comité, lo que con 512 miembros equivale a un mínimo de 342 firmas. El resultado conjunto de las tres comprobaciones se expone como un único valor booleano, que `BeaconLightClient` exige verdadero (Sección 4.1.3).

Dos decisiones de diseño determinan la utilidad de esta prueba para el puente [111]. La primera consiste en derivar el mensaje firmado dentro del programa, en lugar de recibirlo como un valor de 32 bytes ya calculado. Un mensaje recibido de ese modo sería opaco, ya que la prueba no permitiría saber de qué encabezado proviene ni qué bloque de ejecución representa. La segunda consiste en sumar las claves dentro del programa y exponer el compromiso del comité completo, calculado como el hash SHA-256 de sus 512 claves comprimidas concatenadas en orden, en lugar de la clave agregada. La clave agregada depende de qué miembros firmaron y cambia, por lo tanto, de un *slot* a otro, mientras que el compromiso permanece estable durante el período del comité, de unas 27 horas (Sección 2.2.7), lo que permite que el propietario de `BeaconLightClient` lo registre una vez por período.

La expresión de este enunciado en Noir se ve favorecida porque casi todos sus datos tienen un tamaño fijado por el protocolo. El comité tiene siempre 512 miembros, cada clave comprimida ocupa 48 bytes y la firma 96, y la rama tiene nueve niveles, ya que el índice generalizado 812,

con el que el puente localiza el hash del bloque de ejecución, resulta de componer la posición del campo `execution_payload` en el cuerpo del bloque, organizado en dieciséis hojas, con la del campo `block_hash` dentro del *execution payload*, organizado en treinta y dos [111]. Los arreglos de tamaño fijo que exige el programa principal (Sección 3.3.2) no imponen, por lo tanto, máximos arbitrarios. El costo se concentra, en cambio, en la aritmética de la curva BLS12-381. Noir no ofrece funciones de caja negra para esa curva [84], de modo que la descompresión de las claves, que exige calcular raíces cuadradas en el cuerpo base de 381 bits, su suma, la transformación del mensaje en un punto de $\mathbb{G}_2$ y la comprobación de los emparejamientos deben expresarse mediante aritmética no nativa, representando cada elemento de ese cuerpo con varios elementos del cuerpo nativo (Sección 3.3.4). La función $H_{\mathbb{G}_2}$ incorpora, además, sus propios hashes, puesto que sigue el procedimiento que define la RFC 9380 para la curva BLS12-381, en el que el mensaje se expande mediante SHA-256 antes de transformarse en un punto de la curva [115, 15]. Por otra parte, en un circuito una instrucción condicional no evita el costo de la rama que no se ejecuta, ya que el compilador genera restricciones para ambas [84]. La suma condicionada por el vector de participación tiene, en consecuencia, el costo correspondiente al comité completo, con independencia del número de firmantes. Los hashes SHA-256 del enunciado, entre ellos los de la raíz del encabezado, la rama y el compromiso del comité, que abarca 24 576 bytes, se apoyan en la función de compresión que Noir ofrece como caja negra. Si alguna operación costosa, como una raíz cuadrada, se calcula mediante una función no restringida, su resultado solo queda respaldado por la prueba cuando el programa lo restringe explícitamente, conforme al patrón de la Figura 3.7.

4.3.2. Programa de inclusión

El programa de inclusión recibe el encabezado del bloque de ejecución codificado en RLP, las transacciones y los recibos del bloque y la posición de la transacción de bloqueo, que son los datos que el *relayer* obtiene y contrasta en la segunda etapa del flujo descrito en la Sección 4.1.2. El programa calcula el hash Keccak-256 del encabezado, que por definición es el hash del bloque (Sección 2.2.4), y extrae del propio encabezado la raíz del árbol de transacciones, de manera que esa raíz no es un dato aportado por el probador, sino un campo del bloque identificado por ese hash. Recalcula después la raíz del árbol de Merkle-Patricia a partir de las transacciones, cuyas claves son las codificaciones RLP de sus posiciones (Sección 2.2.5), y comprueba que coincide con la del encabezado y que la transacción de bloqueo ocupa la posición declarada. Por último, calcula el hash de esa transacción, decodifica su sobre, cuyo formato depende del tipo de transacción [116], y extrae el destinatario, el valor transferido y los datos de la llamada, de los cuales expone el hash Keccak-256 y la longitud [111].

La inclusión no basta, sin embargo, para el puente, porque una transacción que revierte permanece en el bloque sin haber bloqueado ningún token (Sección 2.2.5). Por ello, el programa extrae también del encabezado la raíz del árbol de recibos, recalcula esa raíz a partir de los recibos del bloque, que se organizan con las mismas claves que las transacciones, y lee el código

de estado del recibo situado en la posición de la transacción de bloqueo [19]. Expone como valores públicos si ese recibo pertenece al árbol del encabezado y si registra una ejecución exitosa, y `ZKReceiver` exige ambas condiciones (Sección 4.1.3).

El programa no interpreta la llamada a `lock`. Los datos de la llamada tienen longitud variable y su incorporación íntegra a los valores públicos aumentaría su tamaño, por lo que el enunciado expone solo su hash y su longitud. `ZKReceiver` recibe esos datos como argumento, comprueba que su hash y su longitud coinciden con los comprometidos por la prueba y obtiene de ellos el destinatario y el monto (Sección 4.1.3). La vinculación con la transacción probada se conserva, ya que presentar otros datos con el mismo hash exigiría encontrar una segunda preimagen de Keccak-256.

La expresión de este enunciado como circuito presenta una dificultad que no aparece en el de consenso, porque ninguno de sus datos tiene un tamaño fijado por el protocolo. La longitud del encabezado depende de la cantidad de campos que añadieron las sucesivas actualizaciones (Sección 2.2.4), el número de transacciones varía de un bloque a otro y cada transacción y cada recibo, que incluye los eventos emitidos, tienen su propia longitud. Como los parámetros del programa principal deben tener un tamaño fijo, el programa recibe arreglos dimensionados para un encabezado de longitud máxima, un número máximo de transacciones y una longitud máxima por transacción y por recibo, acompañados de las longitudes efectivas (Sección 3.3.4). Las operaciones cuyo recorrido depende de los datos, como la decodificación RLP, cuyas posiciones solo se conocen durante la ejecución, o el cálculo de Keccak-256 sobre entradas de longitud variable a partir de la permutación que Noir ofrece como caja negra, se expresan mediante ciclos acotados por esos máximos, que el compilador desenrolla, y mediante accesos a posiciones de los arreglos que solo se conocen durante la ejecución, más costosos que los accesos a posiciones fijas [84]. El circuito incorpora, en consecuencia, las restricciones correspondientes a los tamaños máximos con independencia del bloque concreto, y un bloque que los supere no puede demostrarse con él. Aumentar los máximos modifica el circuito y, con él, su clave de verificación y su contrato verificador (Sección 4.3.3). Estos máximos constituyen, por lo tanto, un parámetro del diseño que condiciona tanto los bloques admisibles como el costo de cada prueba, y su efecto se considera en la metodología de comparación.

4.3.3. Valores públicos y contratos verificadores

Los valores públicos de ambos programas son los que consumen los contratos de destino, por lo que su contenido coincide con el de la implementación en SP1. La Tabla 4.1 los enumera con los nombres de los campos que utilizan los contratos e indica el uso que cada contrato hace de ellos. Algunos no intervienen en ninguna comprobación y se exponen con fines de trazabilidad, como las raíces que `BeaconLightClient` registra en el evento que emite al anclar un bloque o el hash de la transacción que `ZKReceiver` registra en el evento `Minted`.

En Noir, estos valores se expresan como entradas públicas del circuito, cada una de las cuales es un elemento del cuerpo de 254 bits de Barretenberg (Sección 3.3.2). Los valores booleanos

y los enteros de hasta 64 bits ocupan un elemento cada uno. Un valor de 32 bytes, como un hash, el dominio o el compromiso del comité, puede superar en cambio el módulo de ese cuerpo, por lo que no puede representarse en un único elemento sin pérdida de información y debe descomponerse en varios, por ejemplo en sus bytes, como ocurre con los arreglos de tipo `u8` (Sección 3.3.4), o en dos mitades de 128 bits. Mientras que en SP1 los valores públicos forman una secuencia de bytes que el contrato decodifica según el ABI de Ethereum (Sección 3.4.2), aquí el contrato recibe un arreglo de elementos de 32 bytes y debe reconstruir cada valor a partir de los elementos que lo representan antes de aplicar las comprobaciones de la Sección 4.1.3, que no cambian. La descomposición y el orden de las entradas forman parte, por lo tanto, de la interfaz entre el programa y el contrato, y una discrepancia entre ambos haría que el contrato comparara valores distintos de los demostrados. La descomposición elegida incide, además, en el costo, ya que el verificador generado incorpora cada entrada pública en la transcripción de Fiat-Shamir y en el cálculo del término que la vincula con la prueba, de modo que su costo en gas crece con el número de entradas [86].

El contrato verificador lo genera Barretenberg a partir de la clave de verificación de cada circuito, con el destino `evm` descrito en la Sección 3.3.3. En la versión utilizada, el contrato generado incorpora como constantes la clave de verificación, que incluye el tamaño del circuito y el número de entradas públicas, y expone una función `verify` que recibe la prueba y el arreglo de entradas públicas. Esta función comprueba que la longitud de la prueba, que depende del logaritmo del tamaño del circuito, y el número de entradas sean los esperados, recalcula los desafíos de Fiat-Shamir con Keccak, verifica el protocolo *sumcheck* y la apertura de los compromisos KZG mediante los contratos precompilados de la curva BN254, y revierte la transacción si alguna de estas comprobaciones falla [86]. Como la clave de verificación forma parte del contrato, cada circuito requiere su propio verificador, de modo que el papel del contrato verificador de la Sección 4.1.1 lo cumplen dos contratos generados, uno para el programa de consenso y otro para el de inclusión. Fijar en los contratos de destino la dirección del verificador de cada circuito cumple la misma función que fijar la clave del programa en SP1, ya que un verificador distinto aceptaría pruebas de otro circuito. Ambos verificadores emplean la cadena de referencia universal de la ceremonia Ignition, por lo que no requieren una ceremonia específica para cada circuito (Sección 3.3.3), y pueden desplegarse en Optimism Sepolia sin modificaciones (Sección 2.4.3).

4.3.4. Generación de las pruebas e integración

La preparación se realiza una vez por circuito. Nargo compila los dos programas, Barretenberg obtiene la clave de verificación de cada circuito y genera su contrato verificador, y ambos verificadores se despliegan en la cadena de destino antes que `BeaconLightClient` y `ZKReceiver`, en los que sus direcciones se configuran junto con los parámetros de política descritos en la Sección 4.1.1. La cadena de referencia debe contener al menos tantos puntos como requiere el mayor de los dos circuitos, y sus puntos se descargan cuando no están disponibles

localmente (Sección 3.3.3). Cualquier modificación de un programa, incluida la de sus tamaños máximos, exige repetir estos pasos para el circuito afectado.

Tabla 4.1: Valores públicos de los programas de consenso y de inclusión y uso que hacen de ellos los contratos de destino.

Valor	**Contenido**	**Uso en el contrato**
Programa de consenso, consumido por `BeaconLightClient`		
`valid`	Conjunción de la verificación de la firma, de la rama SSZ y del umbral de participación.	Debe ser verdadero.
`signingRoot`	Mensaje firmado por el comité.	Se registra en el evento de anclaje.
`headerRoot`	Raíz del encabezado de la cadena de baliza.	Se registra en el evento de anclaje.
`executionBlockHash`	Hash del bloque de ejecución.	Se ancla y no puede ser cero.
`domain`	Dominio de firma.	Debe coincidir con el fijado por el propietario.
`slot`	*Slot* del encabezado.	Se registra con el anclaje y no puede ser cero.
`blockHashGindex`	Índice generalizado de la rama SSZ.	Debe coincidir con el fijado por el propietario.
`committeeCommitment`	Hash SHA-256 de las claves del comité.	Debe coincidir con el fijado por el propietario.
`committeeSize`, `participants`	Tamaño del comité y número de firmantes.	No se comprueban, porque el umbral forma parte de `valid`.
Programa de inclusión, consumido por `ZKReceiver`		
`verified`	Resultado de la comprobación de inclusión.	Debe ser verdadero.
`headerBound`	La raíz de transacciones proviene del encabezado.	Debe ser verdadero.
`txParsed`	El sobre de la transacción pudo decodificarse.	Debe ser verdadero.
`txType`	Tipo de la transacción.	No se comprueba.
`blockHash`	Hash Keccak-256 del encabezado de ejecución.	Debe estar anclado y forma parte de la clave contra la reutilización.
`transactionsRoot`	Raíz del árbol de transacciones.	El programa la contrasta con la raíz reconstruida, y el contrato no la compara por separado.
`txIndex`	Posición de la transacción en el bloque.	Forma parte de la clave contra la reutilización.
`txHash`	Hash Keccak-256 de la transacción.	Se registra en el evento `Minted`.
`txTo`	Destinatario de la transacción.	Debe ser el contrato `Sender`.
`txValue`	Valor transferido en la moneda nativa.	No se comprueba.
`txDataHash`	Hash Keccak-256 de los datos de la llamada.	Debe coincidir con el de los datos recibidos.
`txDataLen`	Longitud de los datos de la llamada.	Debe coincidir con la de los datos recibidos.
`receiptVerified`	El recibo de la misma posición pertenece al árbol de recibos del encabezado.	Debe ser verdadero.
`txSucceeded`	Ese recibo registra una ejecución exitosa.	Debe ser verdadero.
`receiptsRoot`	Raíz del árbol de recibos.	El programa la contrasta con la raíz reconstruida, y el contrato no la compara por separado.

Durante cada transferencia, el *relayer* sigue las etapas de la Sección 4.1.2. Una vez obtenidos y contrastados los datos de la cadena de origen, los organiza según la interfaz de cada programa, lo que incluye completar los arreglos hasta sus tamaños máximos e indicar las longitudes efectivas, ejecuta los programas con `noir_js` para obtener los testigos y genera las pruebas con `bb.js` y el destino `evm` (Sección 3.3.3). Cada generación produce la prueba y el arreglo de entradas públicas que espera el contrato verificador correspondiente. El *relayer*, común a ambas implementaciones, genera las dos pruebas y consulta después si el bloque ya está anclado. Si no lo está, las envía a `mintRemoteWithHeader` , y si ya lo está, envía solo la de inclusión a `mintRemote`, en ambos casos junto con los datos de la llamada a `lock`. Una ejecución que no satisface las aserciones de un programa no produce testigo, de modo que esos fallos se detectan antes de generar la prueba.

Las etapas de preparación, de puesta en marcha y de cada operación distinguidas en la Sección 3.3.3 se aplican del mismo modo a ambos circuitos, y su medición se define en la metodología de comparación. La sección siguiente describe la expresión de estos mismos enunciados en SP1.

4.4. Diseño e implementación en SP1

La implementación en SP1 expresa los dos enunciados presentados en la Sección 4.1.1 mediante dos programas invitados escritos en Rust, cuya ejecución se demuestra en la máquina virtual descrita en la Sección 3.4, y dos programas anfitriones que preparan sus entradas y solicitan las pruebas. La arquitectura, los contratos y el flujo de una transferencia son los de la Sección 4.1, por lo que esta sección sigue la estructura de la Sección 4.3 y se ocupa de lo que demuestra cada programa, de la codificación y la verificación de sus valores públicos y de la generación de las pruebas. La descripción corresponde al código del repositorio del puente y de los repositorios de sus dos programas [111, 117, 118]. En ninguno de los dos programas las dependencias criptográficas, que son las bibliotecas `sha2`, `sha3` y `bls12_381_plus`, se sustituyen por las versiones modificadas que dirigen sus operaciones hacia los precompilados de la máquina virtual (Sección 3.4.3), de modo que el cálculo de SHA-256, de Keccak-256 y de las operaciones sobre BLS12-381 se ejecuta como instrucciones RISC-V ordinarias.

4.4.1. Programa de consenso

El programa invitado de consenso corresponde al paquete `sync-committee-program`, y su anfitrión es el ejecutable `sync-committee`, que lee un archivo con los datos de la cadena de baliza y escribe en la entrada del invitado doce valores. Esos valores son los cinco campos del encabezado atestiguado, el dominio de firma, la firma agregada, las claves públicas comprimidas de los miembros del comité, el vector de participación, el hash del bloque de ejecución que se pretende anclar, la rama SSZ que lo vincula con `body_root` y su índice generalizado [117]. Cuando la cadena de origen dispone de un nodo de baliza, el puente obtiene esos datos tomando

como encabezado atestiguado el del bloque de baliza cuyo *execution payload* contiene el hash del bloque del bloqueo, y tomando la firma y el vector de participación del bloque siguiente, ya que el agregado incluido en un *slot* firma la raíz del bloque anterior [111, 22]. El anfitrión resuelve el dominio, a partir de un valor explícito o de la versión de la bifurcación y de la raíz de los validadores del bloque génesis, y verifica de forma nativa la rama y la firma antes de solicitar la ejecución, con el fin de rechazar entradas inconsistentes sin esperar a la máquina virtual. Estas comprobaciones del anfitrión quedan fuera de la ejecución demostrada (Sección 3.4.2), por lo que el invitado repite todas las que sustentan el enunciado, con la única excepción de la procedencia del dominio, que se contrasta en el contrato.

El invitado calcula la raíz del encabezado mediante `hash_tree_root` y obtiene la raíz de firma como el hash SHA-256 de la concatenación de esa raíz con el dominio [15]. Suma después las claves de los miembros cuyo bit de participación vale uno, descomprimiendo únicamente las de esos miembros, y verifica la Ecuación (2.4) sobre la raíz de firma, con la función $H_{\mathbb{G}_2}$ implementada como hash a la curva con la etiqueta de separación de dominio que emplea Ethereum [115]. A diferencia de un circuito, la ejecución solo recorre las claves de los firmantes, de manera que el trabajo de esta etapa depende de la participación efectiva. Verifica también, mediante la función `verify_merkle_branch`, que la rama conduce desde el hash del bloque de ejecución hasta `body_root` y que el índice generalizado corresponde exactamente a la profundidad de la rama, y comprueba que el número de firmantes multiplicado por tres supere el doble del tamaño del comité. Si las claves o el vector de participación están mal formados, el invitado no interrumpe su ejecución, sino que registra cero firmantes, de modo que la prueba se genera igualmente con un resultado negativo. El invitado compromete diez valores públicos, entre ellos el indicador `valid`, que solo es verdadero si la firma, la rama y el umbral se verifican, y `committeeCommitment`, el hash SHA-256 de las claves comprimidas del comité concatenadas en orden [117].

`BeaconLightClient` verifica la prueba con la clave del programa de consenso, exige que `valid` sea verdadero, compara el dominio, el índice generalizado y el compromiso del comité con los valores fijados por su propietario, rechaza un hash de bloque o un *slot* nulos y, si todas las comprobaciones se cumplen, registra el *slot* asociado al hash del bloque de ejecución [111]. El encabezado que firma el comité es el atestiguado. Ni el invitado ni el contrato verifican la rama que vincularía un encabezado finalizado con el `state_root` del atestiguado (Sección 2.2.8), de modo que el anclaje acredita la firma del comité sobre el bloque, pero no su finalidad.

4.4.2. Programa de inclusión y ejecución exitosa

El programa invitado de inclusión corresponde al paquete `tx-inclusion-verifier-program`, cuya función principal lee una estructura de entrada, delega la verificación en la función `process_input` de la biblioteca `tx-inclusion-verifier-lib` y compromete el resultado [118]. Su anfitrión, el ejecutable `tx-inclusion-verifier`, recibe esa entrada en un archivo que prepara el *relayer*. Este reconstruye el encabezado del bloque

y comprueba que su hash Keccak-256 coincida con el que informa el nodo, reserializa cada transacción y comprueba su hash, obtiene los recibos del bloque, comprueba que cada uno corresponda a la transacción de su posición y los serializa en la codificación que almacena el árbol de recibos, con el byte de tipo antepuesto en los recibos tipados [111, 116].

El invitado calcula el hash Keccak-256 del encabezado, que identifica el bloque, y extrae de ese mismo encabezado `transactionsRoot` y `receiptsRoot` (Tabla 2.1). Con la lista completa de transacciones reconstruye el árbol de transacciones, cuyas claves son las codificaciones RLP de las posiciones, comprueba que su raíz coincida con la del encabezado y que la transacción declarada ocupe la posición indicada, y repite el procedimiento con los recibos y su raíz. El recibo que examina es el de esa misma posición, por lo que presentar el recibo exitoso de otra transacción alteraría la raíz reconstruida o correspondería a otra posición. La biblioteca admite también la verificación de una rama del árbol, pero el puente utiliza la reconstrucción completa. El invitado decodifica después el sobre de la transacción, del que obtiene el destinatario, el valor y los datos de la llamada, cuyo hash y longitud compromete, y lee el código de estado del recibo, que desde Byzantium indica si la ejecución terminó sin revertir [19]. Un recibo anterior a esa actualización, que contiene una raíz de estado en lugar del código, se considera no válido [118].

El estado exitoso no se exige mediante una aserción del invitado. La función `process_input` no interrumpe la ejecución ante un error, sino que lo refleja en los indicadores `receiptVerified` y `txSucceeded`, y este último solo es verdadero si el recibo se verificó y su código indica éxito. Una prueba de un bloqueo revertido es, por lo tanto, una prueba válida, y es `ZKReceiver` quien la rechaza con el error `TransactionFailed` [111]. El *relayer*, por su parte, descarta un bloqueo revertido antes de generar las pruebas y vuelve a comprobar ambos indicadores antes de enviarlas, pero esa conducta solo evita trabajo innecesario. La condición que impide acuñar es la del receptor, que se aplica a cualquier parte que presente la prueba. Los datos de la llamada se vinculan con el destinatario y el monto del mismo modo descrito en la Sección 4.1.3, ya que el receptor recibe esos datos, comprueba su longitud y su hash frente a los comprometidos y decodifica de ellos la llamada a `lock`. El evento `Locked` sirve al *relayer* para detectar la operación, mientras que el receptor se apoya en el recibo probado.

4.4.3. Valores públicos y verificación en Solidity

Los valores públicos de cada programa son una secuencia de bytes con la codificación del ABI de Ethereum. El invitado de consenso construye una estructura declarada con la biblioteca `alloy-sol-types`, cuyo cuarto campo es `executionBlockHash`, la codifica y la compromete mediante `commit_slice` (Sección 3.4.2). Como todos los campos son de tamaño fijo, la codificación ocupa exactamente diez palabras de 32 bytes y coincide byte a byte con la que decodifica `abi.decode` en Solidity, de modo que el hash del bloque ocupa la cuarta palabra. El anfitrión escribe esos bytes junto con la prueba en un archivo, y el *relayer* los entrega como argumento de `mintRemoteWithHeader`. `ZKReceiver` los reenvía a `updateHeader`, donde `BeaconLightClient` verifica la prueba, decodifica la estructura y registra el anclaje con ese hash como clave. Los

valores del programa de inclusión ocupan quince palabras y llevan el hash del bloque en la quinta. `ZKReceiver` los decodifica y consulta `isAnchored` con ese valor, de modo que el vínculo entre ambas pruebas es la igualdad de esos 32 bytes, comprobada a través del registro de anclajes [111, 117, 118].

Ambos contratos delegan la verificación en la función `verifyProof` del contrato verificador, que recibe la clave del programa, los valores públicos y la prueba, y revierte si la prueba no es válida para esa combinación [94]. Las claves de los dos programas se fijan como valores inmutables al desplegar los contratos, por lo que una prueba válida de otro programa con valores de la misma forma se rechaza. El verificador incorpora los valores públicos a través de su hash SHA-256 (Sección 3.4.3), de manera que alterar un solo byte de la secuencia invalida la prueba. Los campos y el uso que hacen de ellos los contratos coinciden con los de la Tabla 4.1, y no se encontraron diferencias. Conviene precisar dos aspectos de esta implementación. El indicador `txSucceeded` implica `receiptVerified`, porque el programa solo lo activa sobre un recibo verificado, y `ZKReceiver` comprueba ambos, de modo que un recibo no verificado se rechaza con `ReceiptNotVerified` antes de examinar el éxito. Las raíces `transactionsRoot` y `receiptsRoot` se contrastan dentro del programa con las reconstruidas, y ningún contrato las compara por separado.

4.4.4. Generación de pruebas e integración

La preparación se realiza una vez por programa. La compilación de cada anfitrión compila también el invitado para la arquitectura RISC-V de la máquina virtual, y la clave de verificación se deriva de forma determinista del ejecutable resultante (Sección 3.4.2). El script de despliegue de la cadena de destino lee esas claves de los ejecutables y las fija en `BeaconLightClient` y `ZKReceiver`, junto con la dirección del contrato verificador de SP1 de la red y los parámetros de política. Los scripts del puente recompilan los programas cuando sus fuentes son más recientes que los ejecutables, y cualquier modificación de un invitado, como la incorporación de la verificación del recibo, produce una clave distinta y exige desplegar de nuevo los contratos que la fijan [111].

Durante cada transferencia, el *relayer* prepara las dos entradas y ejecuta ambos anfitriones en el modo destinado a Ethereum, que genera la prueba de la ejecución y la envuelve en una prueba Groth16 sobre la curva BN254 (Sección 3.4.3). Los anfitriones habilitan la característica `native-gnark` del kit, que permite realizar la envoltura sin un contenedor, y verifican localmente cada prueba antes de escribirla. El *relayer* genera siempre ambas pruebas y consulta el anclaje del bloque solo después de obtenerlas. Si el bloque no está anclado, envía las dos a `mintRemoteWithHeader` , que ancla el bloque y acuña en una sola transacción, mientras que, si ya lo está, envía solo la prueba de inclusión a `mintRemote` , y la de consenso, aunque se generó, queda sin utilizar. En ambos casos acompaña los datos de la llamada a `lock`. Los anfitriones ofrecen, además, un modo que ejecuta el invitado sin generar ninguna prueba y otro que genera la prueba comprimida, que no está destinada a la verificación en Ethereum.

Estos modos, junto con las transacciones de la cadena de destino, permiten distinguir la ejecución sin prueba, que informa el número de ciclos, la generación de la prueba comprimida, el tiempo adicional de la envoltura Groth16, que se estima por diferencia entre los tiempos de los modos destinado a Ethereum y comprimido, $T_{\text{Ethereum}} - T_{\text{comprimido}}$, la verificación local y la verificación en la cadena de destino. Los anfitriones no cronometran internamente sus etapas, y en los dos modos que generan pruebas ejecutan en secuencia la preparación de las claves, la generación de la prueba, su verificación local y la escritura de los archivos de salida [117, 118]. Si se cronometra cada modo completo, la diferencia es, por lo tanto, una estimación, que incluye además la diferencia entre el costo de verificar localmente una prueba Groth16 y el de verificar una prueba comprimida. Del mismo modo, el tiempo total del *relayer* incluye todo el trabajo que efectivamente realiza, aun cuando una de las pruebas no se envíe, mientras que los tiempos de generación de las pruebas de consenso y de inclusión se comparan por separado. La sección siguiente define cómo se miden estas etapas y su correspondencia con las de la implementación en Noir.

CAPÍTULO 5

CONCLUSIÓN

REFERENCIAS

[1] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. 2014. URL: https://ethereum.org/en/whitepaper/.

[2] Mengya Zhang, Xiaokuan Zhang, Josh Barbee, Yinqian Zhang y Zhiqiang Lin. *SoK: Security of Cross-chain Bridges: Attack Surfaces, Defenses, and Open Problems*. arXiv:2312.12573 [cs.CR]. 2023. URL: https://arxiv.org/abs/2312.12573.

[3] Aztec Labs. *Noir: A Domain Specific Language for Zero-Knowledge Proofs*. 2024. URL: https://noir-lang.org/docs/.

[4] Succinct Labs. *SP1: A Zero-Knowledge Virtual Machine for Rust*. 2024. URL: https://docs.succinct.xyz/docs/sp1/introduction.

[5] Ralph C. Merkle. «A Digital Signature Based on a Conventional Encryption Function». En: *Advances in Cryptology — CRYPTO '87* (1988), págs. 369-378. DOI: 10.1007/3-540-48184-2_32.

[6] Leslie Lamport, Robert Shostak y Marshall Pease. «The Byzantine Generals Problem». En: *ACM Transactions on Programming Languages and Systems* 4.3 (1982), págs. 382-401. URL: https://lamport.azurewebsites.net/pubs/byz.pdf.

[7] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. URL: https://bitcoin.org/bitcoin.pdf.

[8] Gavin Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Inf. téc. Yellow Paper. Ethereum Foundation, 2014. URL: https://ethereum.github.io/yellowpaper/paper.pdf.

[9] Vitalik Buterin, Eric Conner, Rick Dudley, Matthew Slipper, Ian Norden y Abdelhamid Bakhta. *EIP-1559: Fee Market Change for ETH 1.0 Chain*. Ethereum Improvement Proposals. 2019. URL: https://eips.ethereum.org/EIPS/eip-1559.

[10] Christian Reitwiessner. *EIP-196: Precompiled Contracts for Addition and Scalar Multiplication on the Elliptic Curve alt_bn128*. Ethereum Improvement Proposals. 2017. URL: https://eips.ethereum.org/EIPS/eip-196.

[11] Vitalik Buterin y Christian Reitwiessner. *EIP-197: Precompiled Contracts for Optimal Ate Pairing Check on the Elliptic Curve alt_bn128*. Ethereum Improvement Proposals. 2017. URL: https://eips.ethereum.org/EIPS/eip-197.

[12] Antonio Salazar Cardozo y Zachary Williamson. *EIP-1108: Reduce alt_bn128 Precompile Gas Costs*. Ethereum Improvement Proposals. 2018. URL: https://eips.ethereum.org/EIPS/eip-1108.

[13] Alex Vlasov, Kelly Olson, Alex Stokes y Antonio Sanso. *EIP-2537: Precompile for BLS12-381 Curve Operations*. Ethereum Improvement Proposals. 2020. URL: https://eips.ethereum.org/EIPS/eip-2537.

[14] Mikhail Kalinin, Danny Ryan y Vitalik Buterin. *EIP-3675: Upgrade Consensus to Proof-of-Stake*. Ethereum Improvement Proposals. 2021. URL: https://eips.ethereum.org/EIPS/eip-3675.

[15] Ethereum Foundation. *Phase 0 – The Beacon Chain*. Especificaciones de consenso de Ethereum (consensus-specs). 2020. URL: https://github.com/ethereum/consensus-specs/blob/master/specs/phase0/beacon-chain.md.

[16] Ethereum Foundation. *SimpleSerialize (SSZ)*. Especificaciones de consenso de Ethereum (consensus-specs). 2020. URL: https://github.com/ethereum/consensus-specs/blob/master/ssz/simple-serialize.md.

[17] Mikhail Kalinin y Danny Ryan. *EIP-4399: Supplant DIFFICULTY Opcode with PREVRANDAO*. Ethereum Improvement Proposals. 2021. URL: https://eips.ethereum.org/EIPS/eip-4399.

[18] Alex Stokes, Lightclient, Danny Ryan y Mikhail Kalinin. *EIP-4788: Beacon Block Root in the EVM*. Ethereum Improvement Proposals. 2022. URL: https://eips.ethereum.org/EIPS/eip-4788.

[19] Nick Johnson. *EIP-658: Embedding Transaction Status Code in Receipts*. Ethereum Improvement Proposals. 2017. URL: https://eips.ethereum.org/EIPS/eip-658.

[20] Vitalik Buterin, Diego Hernandez, Thor Kamphefner, Khiem Pham, Zhi Qiao, Danny Ryan, Juhyeok Sin, Ying Wang y Yan X. Zhang. *Combining GHOST and Casper*. arXiv:2003.03052 [cs.CR]. 2020. URL: https://arxiv.org/abs/2003.03052.

[21] Vitalik Buterin y Virgil Griffith. *Casper the Friendly Finality Gadget*. arXiv:1710.09437 [cs.CR]. 2017. URL: https://arxiv.org/abs/1710.09437.

[22] Ethereum Foundation. *Altair – The Beacon Chain*. Especificaciones de consenso de Ethereum (consensus-specs). 2021. URL: https://github.com/ethereum/consensus-specs/blob/master/specs/altair/beacon-chain.md.

[23] Dan Boneh, Ben Lynn y Hovav Shacham. «Short Signatures from the Weil Pairing». En: *Advances in Cryptology — ASIACRYPT 2001*. Vol. 2248. Lecture Notes in Computer Science. Springer, 2001, págs. 514-532. DOI: 10.1007/3-540-45682-1_30. URL: https://doi.org/10.1007/3-540-45682-1_30.

[24] Ethereum Foundation. *Altair Light Client – Sync Protocol*. Especificaciones de consenso de Ethereum (consensus-specs). 2021. URL: https://github.com/ethereum/consensus-specs/blob/master/specs/altair/light-client/sync-protocol.md.

[25] Ethereum Foundation. *Capella Light Client – Sync Protocol*. Especificaciones de consenso de Ethereum (consensus-specs). 2023. URL: https://github.com/ethereum/consensus-specs/blob/master/specs/capella/light-client/sync-protocol.md.

[26] Aztec Labs. *Aztec Documentation*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/.

[27] The Block. *Aztec Launches Privacy-Focused L2 Ignition Chain on Mainnet Following Token Sale*. 2025. URL: https://www.theblock.co/post/379592/aztec-launches-ignition-chain-mainnet.

[28] Optimism Foundation. *Fault Proofs Explainer*. Documentación oficial del OP Stack. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.optimism.io/stack/fault-proofs/explainer.

[29] Aztec Labs. *State Management*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/developers/docs/foundational-topics/state_management.

[30] Aztec Labs. *Protocol Circuit Types: constants.nr y block_header.nr, versión 5.2.0*. Código fuente en el repositorio aztec-packages. Consultado el 15 de octubre de 2026. 2026. URL: https://github.com/AztecProtocol/aztec-packages/tree/v5.2.0/noir-projects/noir-protocol-circuits/crates/types/src.

[31] Aztec Labs. *Indexed Merkle Tree (Nullifier Tree)*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/developers/docs/foundational-topics/advanced/storage/indexed_merkle_tree.

[32] Aztec Labs. *Transactions*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/developers/docs/foundational-topics/transactions.

[33] Aztec Labs. *Private Kernel Circuit*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/developers/docs/foundational-topics/advanced/circuits/private_kernel.

[34] Aztec Labs. *Fees*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/developers/docs/foundational-topics/fees.

[35] Aztec Labs. *Rollup Circuits*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/developers/docs/foundational-topics/advanced/circuits/rollup_circuits.

[36] Aztec Labs. *Run a Node*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/operate/operators.

[37] Aztec Labs. *Blocks*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/participate/basics/blocks.

[38] Aztec Labs. *L1-L2 Communication (Portals)*. Documentación oficial de la red Aztec. Consultada el 15 de octubre de 2026. 2026. URL: https://docs.aztec.network/developers/docs/foundational-topics/ethereum-aztec-messaging.

[39] Optimism Foundation. *Rollup Overview*. Documentación oficial del OP Stack. Consultada el 24 de octubre de 2026. 2026. URL: https://docs.optimism.io/stack/rollup/overview.

[40] Optimism Foundation. *Superchain Registry: configuración de OP Sepolia Testnet*. Archivo de configuración op.toml. Consultado el 24 de octubre de 2026. 2026. URL: https://github.com/ethereum-optimism/superchain-registry/blob/main/superchain/configs/sepolia/op.toml.

[41] Vitalik Buterin, Dankrad Feist, Diederik Loerzel, George Kadianakis, Matt Garnett, Mofi Taiwo y Ansgar Dudley. *EIP-4844: Shard Blob Transactions*. Ethereum Improvement Proposals. 2022. URL: https://eips.ethereum.org/EIPS/eip-4844.

[42] Optimism Foundation. *OP Stack Specification: Overview*. Especificación del protocolo. Consultada el 24 de octubre de 2026. 2026. URL: https://specs.optimism.io/protocol/overview.html.

[43] Optimism Foundation. *OP Stack Specification: L2 Execution Engine*. Especificación del protocolo. Consultada el 24 de octubre de 2026. 2026. URL: https://specs.optimism.io/protocol/exec-engine.html.

[44] Optimism Foundation. *Isthmus Hardfork*. Documentación oficial del OP Stack. Consultada el 24 de octubre de 2026. 2025. URL: https://docs.optimism.io/op-stack/protocol/hardforks/isthmus.

[45] Optimism Foundation. *OP Stack Specification: Cannon Fault Proof Virtual Machine*. Especificación del protocolo. Consultada el 24 de octubre de 2026. 2026. URL: https://specs.optimism.io/fault-proof/cannon-fault-proof-vm.html.

[46] Optimism Foundation. *OP Stack Specification: Withdrawals*. Especificación del protocolo. Consultada el 24 de octubre de 2026. 2026. URL: https://specs.optimism.io/protocol/withdrawals.html.

[47] Optimism Foundation. *OP Stack Specification: Predeploys*. Especificación del protocolo. Consultada el 24 de octubre de 2026. 2026. URL: https://specs.optimism.io/protocol/predeploys.html.

[48] Alex Beregszaszi, Hugo Gluzman y Ramana Sarma. *EIP-3855: PUSH0 Instruction*. Ethereum Improvement Proposals. 2021. URL: https://eips.ethereum.org/EIPS/eip-3855.

[49] Justin Thaler. «Proofs, Arguments, and Zero-Knowledge». En: *Foundations and Trends in Privacy and Security* 4.2–4 (2022), págs. 117-660. DOI: 10.1561/3300000030. URL: https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.pdf.

[50] Shafi Goldwasser, Silvio Micali y Charles Rackoff. «The Knowledge Complexity of Interactive Proof Systems». En: *SIAM Journal on Computing* 18.1 (1989), págs. 186-208. DOI: 10.1137/0218012. URL: https://doi.org/10.1137/0218012.

[51] Oded Goldreich, Silvio Micali y Avi Wigderson. «Proofs that Yield Nothing but Their Validity or All Languages in NP Have Zero-Knowledge Proof Systems». En: *Journal of the ACM* 38.3 (1991), págs. 690-728. DOI: 10.1145/116825.116852. URL: https://doi.org/10.1145/116825.116852.

[52] Oded Goldreich. *Foundations of Cryptography. Volume 1: Basic Tools*. Cambridge University Press, 2001. DOI: 10.1017/CBO9780511546891. URL: https://doi.org/10.1017/CBO9780511546891.

[53] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer y Madars Virza. «Zerocash: Decentralized Anonymous Payments from Bitcoin». En: *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, págs. 459-474. DOI: 10.1109/SP.2014.36. URL: https://doi.org/10.1109/SP.2014.36.

[54] Michael Walfish y Andrew J. Blumberg. «Verifying Computations without Reexecuting Them». En: *Communications of the ACM* 58.2 (2015), págs. 74-84. DOI: 10.1145/2641562. URL: https://doi.org/10.1145/2641562.

[55] Gilles Brassard, David Chaum y Claude Crépeau. «Minimum Disclosure Proofs of Knowledge». En: *Journal of Computer and System Sciences* 37.2 (1988), págs. 156-189. DOI: 10.1016/0022-0000(88)90005-0. URL: https://doi.org/10.1016/0022-0000(88)90005-0.

[56] Mihir Bellare y Oded Goldreich. «On Defining Proofs of Knowledge». En: *Advances in Cryptology — CRYPTO '92*. Vol. 740. Lecture Notes in Computer Science. Springer, 1993, págs. 390-420. DOI: 10.1007/3-540-48071-4_28. URL: https://doi.org/10.1007/3-540-48071-4_28.

[57] Manuel Blum, Paul Feldman y Silvio Micali. «Non-Interactive Zero-Knowledge and Its Applications». En: *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing (STOC '88)*. ACM, 1988, págs. 103-112. DOI: 10.1145/62212.62222. URL: https://doi.org/10.1145/62212.62222.

[58] Amos Fiat y Adi Shamir. «How to Prove Yourself: Practical Solutions to Identification and Signature Problems». En: *Advances in Cryptology — CRYPTO '86*. Vol. 263. Lecture Notes in Computer Science. Springer, 1987, págs. 186-194. DOI: 10.1007/3-540-47721-7_12. URL: https://doi.org/10.1007/3-540-47721-7_12.

[59] David Bernhard, Olivier Pereira y Bogdan Warinschi. «How Not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios». En: *Advances in Cryptology — ASIACRYPT 2012*. Vol. 7658. Lecture Notes in Computer Science. Springer, 2012, págs. 626-643. DOI: 10.1007/978-3-642-34961-4_38. URL: https://doi.org/10.1007/978-3-642-34961-4_38.

[60] Mihir Bellare y Phillip Rogaway. «Random Oracles Are Practical: A Paradigm for Designing Efficient Protocols». En: *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS '93)*. ACM, 1993, págs. 62-73. DOI: 10.1145/168588.168596. URL: https://doi.org/10.1145/168588.168596.

[61] Bryan Parno, Jon Howell, Craig Gentry y Mariana Raykova. «Pinocchio: Nearly Practical Verifiable Computation». En: *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, págs. 238-252. DOI: 10.1109/SP.2013.47. URL: https://doi.org/10.1109/SP.2013.47.

[62] Stefanos Chaliasos, Jens Ernstberger, David Theodore, David Wong, Mohammad Jahanara y Benjamin Livshits. *SoK: What Don't We Know? Understanding Security Vulnerabilities in SNARKs*. arXiv:2402.15293 [cs.CR]. 2024. URL: https://arxiv.org/abs/2402.15293.

[63] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer y Madars Virza. «SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge». En: *Advances in Cryptology — CRYPTO 2013*. Vol. 8043. Lecture Notes in Computer Science. Springer, 2013, págs. 90-108. DOI: 10.1007/978-3-642-40084-1_6. URL: https://doi.org/10.1007/978-3-642-40084-1_6.

[64] Jacob T. Schwartz. «Fast Probabilistic Algorithms for Verification of Polynomial Identities». En: *Journal of the ACM* 27.4 (1980), págs. 701-717. DOI: 10.1145/322217.322225. URL: https://doi.org/10.1145/322217.322225.

[65] Richard Zippel. «Probabilistic Algorithms for Sparse Polynomials». En: *Symbolic and Algebraic Computation (EUROSAM '79)*. Vol. 72. Lecture Notes in Computer Science. Springer, 1979, págs. 216-226. DOI: 10.1007/3-540-09519-5_73. URL: https://doi.org/10.1007/3-540-09519-5_73.

[66] Aniket Kate, Gregory M. Zaverucha y Ian Goldberg. «Constant-Size Commitments to Polynomials and Their Applications». En: *Advances in Cryptology — ASIACRYPT 2010*. Vol. 6477. Lecture Notes in Computer Science. Springer, 2010, págs. 177-194. DOI: 10.1007/978-3-642-17373-8_11. URL: https://doi.org/10.1007/978-3-642-17373-8_11.

[67] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh y Michael Riabzev. «Fast Reed-Solomon Interactive Oracle Proofs of Proximity». En: *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Vol. 107. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, 14:1-14:17. DOI: 10.4230/LIPIcs.ICALP.2018.14. URL: https://doi.org/10.4230/LIPIcs.ICALP.2018.14.

[68] Nir Bitansky, Ran Canetti, Alessandro Chiesa y Eran Tromer. «From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again». En: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS '12)*. ACM, 2012, págs. 326-349. DOI: 10.1145/2090236.2090263. URL: https://doi.org/10.1145/2090236.2090263.

[69] Jens Groth. «On the Size of Pairing-Based Non-interactive Arguments». En: *Advances in Cryptology — EUROCRYPT 2016*. Vol. 9666. Lecture Notes in Computer Science. Springer, 2016, págs. 305-326. DOI: 10.1007/978-3-662-49896-5_11. URL: https://doi.org/10.1007/978-3-662-49896-5_11.

[70] iden3. *snarkjs: zkSNARK Implementation in JavaScript and WASM*. Repositorio de código fuente. 2018. URL: https://github.com/iden3/snarkjs.

[71] Ariel Gabizon, Zachary J. Williamson y Oana Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive, Paper 2019/953. 2019. URL: https://eprint.iacr.org/2019/953.

[72] Aztec Labs. *Barretenberg Documentation*. Documentación oficial del backend de pruebas Barretenberg. 2025. URL: https://barretenberg.aztec.network/docs/.

[73] Sean Bowe, Ariel Gabizon y Ian Miers. *Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model*. Cryptology ePrint Archive, Paper 2017/1050. 2017. URL: https://eprint.iacr.org/2017/1050.

[74] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh y Michael Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. Cryptology ePrint Archive, Paper 2018/046. 2018. URL: https://eprint.iacr.org/2018/046.

[75] Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh y Dawn Song. «zkBridge: Trustless Cross-chain Bridges Made Practical». En: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. ACM, 2022, págs. 3003-3017. DOI: 10.1145/3548606.3560652. URL: https://doi.org/10.1145/3548606.3560652.

[76] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio y Jordi Baylina. «Circom: A Circuit Description Language for Building Zero-Knowledge Applications». En: *IEEE Transactions on Dependable and Secure Computing* 20.6 (2023), págs. 4733-4751. DOI: 10.1109/TDSC.2022.3232813. URL: https://doi.org/10.1109/TDSC.2022.3232813.

[77] iden3. *Circom 2 Documentation*. Documentación oficial del compilador circom, versión 2.2. 2025. URL: https://docs.circom.io/.

[78] iden3. *circomlib: Library of Basic Circuits for Circom*. Repositorio de código fuente. 2018. URL: https://github.com/iden3/circomlib.

[79] iden3. *Circom Release Notes*. Archivo RELEASES.md del repositorio oficial del compilador circom. 2025. URL: https://github.com/iden3/circom/blob/master/RELEASES.md.

[80] Shankara Pailoor, Yanju Chen, Franklyn Wang, Clara Rodríguez, Jacob Van Geffen, Jason Morton, Michael Chu, Brian Gu, Yu Feng e Işıl Dillig. «Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs». En: *Proceedings of the ACM on Programming Languages* 7.PLDI (2023), págs. 1510-1532. DOI: 10.1145/3591282. URL: https://doi.org/10.1145/3591282.

[81] Hongbo Wen, Jon Stephens, Yanju Chen, Kostas Ferles, Shankara Pailoor, Kyle Charbonnet, Işıl Dillig y Yu Feng. «Practical Security Analysis of Zero-Knowledge Proof Circuits». En: *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024, págs. 1471-1487. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/wen.

[82] 0xPARC ZK-ID Working Group. *circom-pairing: Proof-of-Concept Implementations of BLS12-381 Pairings in Circom*. Repositorio de código fuente. 2022. URL: https://github.com/yi-sun/circom-pairing.

[83] noir-lang. *noir: The Noir Programming Language, etiqueta v1.0.0-beta.22*. Repositorio oficial, incluye la documentación de ACIR, ACVM y Brillig en acvm-repo. Consultado el 10 de octubre de 2026. 2026. URL: https://github.com/noir-lang/noir/tree/v1.0.0-beta.22.

[84] noir-lang. *Noir Documentation, versión 1.0.0-beta.22*. Fuente de la documentación en la etiqueta v1.0.0-beta.22. Consultada el 10 de octubre de 2026. 2026. URL: https://github.com/noir-lang/noir/tree/v1.0.0-beta.22/docs/docs.

[85] noir-lang. *@noir-lang/noir_js, versión 1.0.0-beta.22*. Paquete npm. 2026. URL: https://www.npmjs.com/package/@noir-lang/noir_js/v/1.0.0-beta.22.

[86] Aztec Labs. *Barretenberg, etiqueta v5.0.0-nightly.20260522*. Código fuente y documentación en el repositorio aztec-packages. Consultado el 10 de octubre de 2026. 2026. URL: https://github.com/AztecProtocol/aztec-packages/tree/v5.0.0-nightly.20260522/barretenberg.

[87] Aztec Labs. *@aztec/bb.js, versión 5.0.0-nightly.20260522*. Paquete npm, incluye el binario bb de la misma versión. 2026. URL: https://www.npmjs.com/package/@aztec/bb.js/v/5.0.0-nightly.20260522.

[88] Ariel Gabizon y Zachary J. Williamson. *plookup: A Simplified Polynomial Protocol for Lookup Tables*. Cryptology ePrint Archive, Paper 2020/315. 2020. URL: https://eprint.iacr.org/2020/315.

[89] Ulrich Haböck. *Multivariate Lookups Based on Logarithmic Derivatives*. Cryptology ePrint Archive, Paper 2022/1530. 2022. URL: https://eprint.iacr.org/2022/1530.

[90] Aztec Protocol. *Ignition Verification: Repository to Verify Contributions to the AZTEC Ignition Ceremony*. Repositorio de código fuente. 2020. URL: https://github.com/AztecProtocol/ignition-verification.

[91] Succinct Labs. *SP1: repositorio oficial, etiqueta v6.2.2*. Incluye la biblioteca SLOP y los crates del ejecutor, del sistema de pruebas Hypercube, del SDK y de la zkVM. Consultado el 10 de octubre de 2026. 2026. URL: https://github.com/succinctlabs/sp1/tree/v6.2.2.

[92] Succinct Labs. *SP1 Documentation (SP1 V6)*. Documentación en línea; las páginas consultadas no indican una versión de parche. Consultada el 10 de octubre de 2026. 2026. URL: https://docs.succinct.xyz/docs/sp1/introduction.

[93] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, versión 20240411*. Especificación oficial. 2024. URL: https://github.com/riscv/riscv-isa-manual/releases/tag/20240411.

[94] Succinct Labs. *sp1-contracts: contratos verificadores de SP1, etiqueta v6.1.0*. Repositorio de código fuente. 2026. URL: https://github.com/succinctlabs/sp1-contracts/tree/v6.1.0.

[95] Succinct Labs. *SP1 Hypercube Security Model*. Documentación de SP1 V6, sección de seguridad. Consultada el 11 de octubre de 2026. 2026. URL: https://docs.succinct.xyz/docs/sp1/security/security-model.

[96] Shahar Papini y Ulrich Haböck. *Improving Logarithmic Derivative Lookups Using GKR*. Cryptology ePrint Archive, Paper 2023/1284. 2023. URL: https://eprint.iacr.org/2023/1284.

[97] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh y Ron D. Rothblum. *Jagged Polynomial Commitments (or: How to Stack Multilinears)*. Cryptology ePrint Archive, Paper 2025/917. Revisión menor de la publicación en EUROCRYPT 2026. 2025. URL: https://eprint.iacr.org/2025/917.

[98] Hadas Zeilberger, Binyi Chen y Ben Fisch. *BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes*. Cryptology ePrint Archive, Paper 2023/1705. Revisión menor de la publicación en CRYPTO 2024. 2023. URL: https://eprint.iacr.org/2023/1705.

[99] Consensys. *gnark: High-Performance zk-SNARKs in Go*. Repositorio de código fuente. 2026. URL: https://github.com/Consensys/gnark.

[100] RISC Zero. *risc0: RISC Zero zero-knowledge VM*. Repositorio de código fuente. Consultado el 24 de octubre de 2026. 2026. URL: https://github.com/risc0/risc0.

[101] RISC Zero. *RISC Zero Developer Docs: Proof System*. Documentación oficial. Consultada el 24 de octubre de 2026. 2026. URL: https://dev.risczero.com/proof-system/.

[102] Lior Goldberg, Shahar Papini y Michael Riabzev. *Cairo – a Turing-complete STARK-friendly CPU architecture*. Cryptology ePrint Archive, Paper 2021/1063. 2021. URL: https://eprint.iacr.org/2021/1063.

[103] StarkWare. *The Cairo Programming Language*. Documentación oficial. Consultada el 24 de octubre de 2026. 2026. URL: https://www.starknet.io/cairo-book/.

[104] Zcash. *The halo2 Book*. Documentación oficial. Consultada el 24 de octubre de 2026. 2026. URL: https://zcash.github.io/halo2/.

[105] Sean Bowe, Jack Grigg y Daira Hopwood. *Recursive Proof Composition without a Trusted Setup*. Cryptology ePrint Archive, Paper 2019/1021. 2019. URL: https://eprint.iacr.org/2019/1021.

[106] Privacy Scaling Explorations. *halo2: variante de Privacy Scaling Explorations*. Repositorio de código fuente. Consultado el 24 de octubre de 2026. 2026. URL: https://github.com/privacy-scaling-explorations/halo2.

[107] Polygon Zero. *Plonky3*. Repositorio de código fuente. Consultado el 24 de octubre de 2026. 2026. URL: https://github.com/Plonky3/Plonky3.

[108] Aligned. *About Aligned*. Documentación oficial. Consultada el 24 de octubre de 2026. 2026. URL: https://docs.alignedlayer.com/.

[109] Aligned. *Archived documentation: Aligned Verification Layer*. Documentación archivada del repositorio aligned_layer. Consultada el 24 de octubre de 2026. 2026. URL: https://github.com/yetanotherco/aligned_layer/blob/testnet/docs/archive/README.md.

[110] Succinct Labs. *Succinct Prover Network: Key Terms*. Documentación oficial. Consultada el 24 de octubre de 2026. 2026. URL: https://docs.succinct.xyz/docs/protocol/key-terms.

[111] FPUNA-ZK-Bridge. *zk-guarani-bridge-sp1*. Repositorio de código fuente del presente trabajo. Versión de trabajo del 31 de octubre de 2026, posterior a la revisión b97de71. 2026. URL: https://github.com/FPUNA-ZK-Bridge/zk-guarani-bridge-sp1.

[112] José Sebastián Figueredo Camé y Francisco Sebastián Riquelme Godoy. «Diseño e implementación de un zkBridge para la interoperabilidad entre blockchains y la transferencia segura de activos digitales». Proyecto de grado. San Lorenzo, Paraguay: Facultad Politécnica, Universidad Nacional de Asunción, 2026.

[113] FPUNA-ZK-Bridge. *guarani-bridge-vanilla*. Repositorio de código fuente. Consultado el 24 de octubre de 2026. 2026. URL: https://github.com/FPUNA-ZK-Bridge/guarani-bridge-vanilla.

[114] OpenZeppelin. *OpenZeppelin Contracts, versión 5.3.0: ERC20.sol*. Implementación del estándar ERC-20 en la que se basa GuaraniToken. 2025. URL: https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v5.3.0/contracts/token/ERC20/ERC20.sol.

[115] Armando Faz-Hernández, Sam Scott, Nick Sullivan, Riad S. Wahby y Christopher A. Wood. *Hashing to Elliptic Curves*. RFC 9380, Internet Research Task Force (IRTF). 2023. DOI: 10.17487/RFC9380. URL: https://www.rfc-editor.org/rfc/rfc9380.

[116] Micah Zoltu. *EIP-2718: Typed Transaction Envelope*. Ethereum Improvement Proposals. 2020. URL: https://eips.ethereum.org/EIPS/eip-2718.

[117] FPUNA-ZK-Bridge. *sp1-sync-committee*. Programa de consenso del presente trabajo para SP1. Versión de trabajo del 31 de octubre de 2026, posterior a la revisión a69f1e1. 2026. URL: https://github.com/FPUNA-ZK-Bridge/sp1-sync-committee.

[118] FPUNA-ZK-Bridge. *tx-inclusion-verifier-sp1*. Programa de inclusión del presente trabajo para SP1. Versión de trabajo del 31 de octubre de 2026, posterior a la revisión c2180f0. 2026. URL: https://github.com/FPUNA-ZK-Bridge/tx-inclusion-verifier-sp1.

APÉNDICE

A.1. A

PENDIENTES

— **Abstract** [Necesario - Crítico]
Ubicación: Preliminares, después del Resumen.
Traducción al inglés del título, el resumen y las palabras clave. El archivo actual contiene solo el texto de plantilla.

— **Agradecimientos** [Sugerencia - Recomendado]
Ubicación: Preliminares, después de la Dedicatoria.
Texto de agradecimientos. El archivo actual contiene solo el título.

1.7 Estructura del documento [Sugerencia - Recomendado]
Ubicación: al final del Capítulo 1, después de Delimitación.
Párrafo breve que describe el contenido de cada capítulo.

2.5.1 Interoperabilidad y concepto de puente [Necesario - Importante]
Ubicación: al inicio de la Sección 2.5 Puentes.
Definición de interoperabilidad y de puente entre cadenas, diferencia entre transferencia de activos y paso de mensajes, y componentes genéricos de un puente (contratos en origen y destino, y agente que transmite la información).

2.5.2 Modelos de transferencia de activos [Necesario - Importante]
Ubicación: después de Interoperabilidad y concepto de puente.
Esquemas lock-and-mint, burn-and-mint, lock-and-unlock con reservas de liquidez e intercambios atómicos. Incluye una tabla comparativa y una figura del flujo lock-and-mint con relayer de confianza.

2.5.3 Modelos de verificación y supuestos de confianza [Necesario - Crítico]
Ubicación: después de Modelos de transferencia de activos.
Clasificación según quién verifica el estado de la cadena de origen. Verificación externa (relayer único, multifirma, comités y MPC), verificación optimista con vigilantes, verificación nativa mediante clientes ligeros y clientes ligeros con pruebas de conocimiento cero. Incluye una tabla con supuesto de confianza, latencia y costo de cada modelo.

2.5.4 Incidentes de seguridad [Necesario - Crítico]
Ubicación: después de Modelos de verificación y supuestos de confianza.
Casos documentados como Ronin, Wormhole, Nomad, Harmony Horizon y Multichain, con fecha, monto, causa y modelo de verificación afectado. Incluye una tabla. Respalda la afirmación de la introducción sobre los puentes como componente más atacado.

2.5.5 Puentes basados en pruebas de conocimiento cero [Necesario - Importante]
Ubicación: después de Incidentes de seguridad.
Panorama de clientes ligeros verificados con pruebas de conocimiento cero, como zkBridge y los clientes ligeros de Ethereum basados en el comité de sincronización. Debe ser breve y remitir a la Sección 4.2 para el análisis de zkBridge.

2.5.6 Requisitos del puente propuesto [Sugerencia - Recomendado]
Ubicación: al final de la Sección 2.5 Puentes.
Síntesis de los requisitos que se derivan del capítulo y enlace con el Capítulo 4.

4.5 Metodología de comparación [Necesario - Crítico]
Ubicación: después de Diseño e implementación en SP1.
Comprobaciones que acredita cada implementación y alcance comparable, métricas (tiempo de generación de las pruebas, memoria, gas y complejidad de desarrollo), correspondencia entre las etapas de Noir y de SP1, hardware, versiones y número de repeticiones. Los costos se reportan en tres ejes, ya anunciados en la Sección 2.4.3. El primero es el gas de ejecución de `update()` del cliente ligero, que se paga una vez por anclaje y se comparte entre todos los bloqueos que ancla, y de `release()`. El segundo es el costo monetario en Optimism Sepolia, calculado como `gasUsed` × `effectiveGasPrice` + `l1Fee` y convertido a ETH y USD con la fecha del precio. El tercero es el costo de generar las pruebas fuera de la cadena, en tiempo, memoria máxima y, si corresponde, costo de la máquina en la nube, por circuito y agregado. Cada medición debe identificar la revisión del código de Noir y de SP1 con que se obtuvo, y ambas revisiones deben verificar el recibo y exigir el éxito de la transacción de bloqueo. Las mediciones anteriores a esa verificación corresponden a otro enunciado y no son comparables.

4.6 Resultados y discusión [Necesario - Crítico]
Ubicación: después de Metodología de comparación.
Mediciones de ambas implementaciones, despliegue en la red de destino, costo de verificación en gas, discusión frente a la hipótesis y tratamiento de las limitaciones señaladas en la Sección 4.1.3.

5.1 Conclusiones [Necesario - Crítico]
Ubicación: al inicio del Capítulo 5.
Conclusiones respecto de cada objetivo específico.

5.2 Verificación de la hipótesis [Necesario - Crítico]
Ubicación: después de Conclusiones.
Respuesta a la pregunta de investigación sobre las ventajas de SP1 frente a Noir.

5.3 Trabajos futuros [Necesario - Importante]
Ubicación: después de Verificación de la hipótesis.
Posibles extensiones, como el despliegue en redes principales, la verificación de la finalidad plena o la incorporación de otras cadenas.

A.1 Código fuente de los circuitos [Sugerencia - Recomendado]

Ubicación: en el Apéndice, reemplazando la sección de plantilla.

Fragmentos relevantes de los circuitos en Noir y de los programas en SP1.

A.2 Entorno y datos de las mediciones [Sugerencia - Recomendado]

Ubicación: en el Apéndice, después de Código fuente de los circuitos.

Especificaciones del equipo, versiones de las herramientas y tablas completas de las mediciones.